

A T M E
College of Engineering



Department of Computer Science and Engineering

Notes of Lesson Academic Year: 2024- 2025

**SUBJECT TITLE AND CODE : SOFTWARE ENGINEERING AND
PROJECT MANAGEMENT (BCS501)
SEMESTER : V**



INSTITUTIONAL MISSION AND VISION

Objectives

- To provide quality education and groom top-notch professionals, entrepreneurs and leaders for different fields of engineering, technology and management.
- To open a Training-R & D-Design-Consultancy cell in each department, gradually introduce doctoral and postdoctoral programs, encourage basic & applied research in areas of social relevance, and develop the institute as a center of excellence.
- To develop academic, professional and financial alliances with the industry as well as the academia at national and transnational levels.
- To develop academic, professional and financial alliances with the industry as well as the academia at national and transnational levels.
- To cultivate strong community relationships and involve the students and the staff in local community service.
- To constantly enhance the value of the educational inputs with the participation of students, faculty, parents and industry.

Vision

- Development of academically excellent, culturally vibrant, socially responsible and globally competent human resources.

Mission

- To keep pace with advancements in knowledge and make the students competitive and capable at the global level.
- To create an environment for the students to acquire the right physical, intellectual, emotional and moral foundations and shine as torch bearers of tomorrow's society.
- To strive to attain ever-higher benchmarks of educational excellence.

Department of Computer Science And Engineering

Vision of the Department

To develop highly talented individuals in computer science and engineering to deal with real world challenges in industry, education, research and society.

Mission of the Department

- To inculcate professional behavior, strong ethical values, innovative research capabilities and leadership abilities in the young minds & to provide a teaching environment that emphasizes depth, originality and critical thinking.
- Motivate students to put their thoughts and ideas adoptable by industry or to pursue higher studies leading to research.

Program Educational Objectives (PEO'S):

1. Empower students with a strong basis in the mathematical, scientific and engineering fundamentals to solve computational problems and to prepare them for employment, higher learning and R&D.
2. Gain technical knowledge, skills and awareness of current technologies of computer science engineering and to develop an ability to design and provide novel engineering solutions for software/hardware problems through entrepreneurial skills.
3. Exposure to emerging technologies and work in teams on interdisciplinary projects with effective communication skills and leadership qualities.
4. Ability to function ethically and responsibly in a rapidly changing environment by applying innovative ideas in the latest technology, to become effective professionals in Computer Science to bear a life-long career in related areas.

Program Specific Outcomes (PSOs)

PSO1: Ability to apply skills in the field of algorithms, database design, web design, cloud computing and data analytics.

PSO2: Apply knowledge in the field of computer networks for building network and internet based applications.

COURSE MODULE FOR THE SESSION 2023- 2024 (EVEN)

Course Syllabi with CO's

Faculty Name/s :			Academic Year: 2023 - 2024			
Department: Computer Science& Engineering						
Course Code	Course Title	Core/Elective	Contact Hours			Total Hrs/ Sessions
			L	T	P	
21CS61	SOFTWARE ENGINEERING & PROJECT MANAGEMENT	Core	2	2	-	40
Topics Covered as per Syllabus						
<u>Module-1</u>						
Software and Software Engineering: The nature of Software, The unique nature of WebApps, Software Engineering, The software Process, The software Engineering practice, The software myths, How it all starts						
Process Models: A generic process model, Process assessment and improvement, Prescriptive process models, Waterfall model, Incremental process models, Evolutionary process models, Concurrent models, Specialized process models..						
<u>Module-2</u>						
Understanding Requirements: Requirements Engineering, Establishing the ground work, Eliciting Requirements, Developing use cases, Building the requirements model, Negotiating Requirements, Validating Requirements.						
Requirements Modeling Scenarios, Information and Analysis classes: Requirement Analysis, Scenario based modeling, UML models that supplement the Use Case, Data modeling Concepts class Based Modeling.						
<u>Module-3</u>						
AGILE DEVELOPMENT: What is Agility?, Agility and the cost of change. What is an agile Process?, Extreme Programming (XP), Other Agile Process Models, A tool set for Agile process						
Principles that guide practice: Software Engineering Knowledge, Core principles, Principles that guide each framework activity.						
<u>Module-4</u>						
Introduction to Project Management:						
Introduction, Project and Importance of Project Management, Contract Management, Activities Covered by Software Project Management, Plans, Methods and Methodologies, Some ways of categorizing Software Projects, Stakeholders, Setting Objectives, Business Case, Project Success						

and Failure, Management and Management Control, Project Management life cycle, Traditional versus Modern Project Management Practices.

Module-5

Software Quality:

Introduction, The place of software quality in project planning, Importance of software quality, Defining software quality, quality models, ISO 9126, product and process metrics, product versus process quality management, Quality Management systems, process capability models, techniques to enhance software quality, testing, Software reliability, quality plans.

List of Text Books

1. Roger S. Pressman: Software Engineering-A Practitioners approach, 7th Edition, Tata McGraw Hill.
2. Bob Hughes, Mike Cotterell, Rajib Mall: Software Project Management, 6th Edition, McGraw Hill Education, 2018.

Reference:

1. Pankaj Jalote: An Integrated Approach to Software Engineering, Wiley India.

List of URLs, Text Books, Notes, Multimedia Content, etc

1. https://onlinecourses.nptel.ac.in/noc20_cs68/preview
2. https://www.youtube.com/watch?v=WxkP5KR_Emk&list=PLrjkTql3jnm9b5nrggx7Pt1G4UAHeFlJ
3. <http://elearning.vtu.ac.in/econtent/CSE.php>
4. <http://elearning.vtu.ac.in/econtent/courses/video/CSE/15CS42.html>
5. <https://nptel.ac.in/courses/128/106/128106012/> (DevOps)

Course Outcomes	After studying this course, students will be able to												
	CO 1. Understand the activities involved in software engineering and analyze the role of various process models												
	CO 2. Explain the basics of object-oriented concepts and build a suitable class model using modelling techniques												
	CO 3. Describe various software testing methods and to understand the importance of agile methodology and DevOps												
	CO 4. Illustrate the role of project planning and quality management in software development												
	CO 5. Understand the importance of activity planning and different planning models												

The Correlation of Course Outcomes (CO's) and Program Outcomes (PO's)

Subject Code:	21CS61	TITLE: Software Engineering and Project Management					Faculty Name:						
List of Course Outcomes	Program Outcomes												Total
	PO-1	PO-2	PO-3	PO-4	PO-5	PO-6	PO-7	PO-8	PO-9	PO-10	PO-11	PO-12	
CO-1	2	3	3	-	-	3	3	-	-	-	-	-	14
CO-2	2	3	3	-	3	-	-		-	-	-	-	11
CO-3	2	3	-	-	2	-	-	-	-	-	-	-	07
CO-4	2	-	-	-	3	-	-	-	3	2	3	3	16
CO-5	2	-	-	-	-	-	-	-	2	-	2	2	08
Total	10	9	6	-	8	3	3	-	5	2	5	5	56

Note: 3 = Strong Contribution 2 = Average Contribution 1 = Weak Contribution 0 = No Contribution

The Correlation of Course Outcomes (CO's) and Program Outcomes (PO's)

Subject Code:	BCS501	TITLE: Software Engineering and Project Management	Faculty Name:	
List of Course Outcomes	Program Outcomes			Total
	PSO-1	PSO-2		
CO-1	1	-		1
CO-2	2	-		2
CO-3	2	-		2
CO-4	3	-		3
CO-5	3	-		3
Total	11	-		11

MODULE-1

1.1 Introduction

This module gives the overview for the need of software engineering, software crisis. It gives the difference between professional software development and personal software development, states the responsibilities of software engineering. Different software process models are discussed. Process activities, requirement engineering are explained. Classification of functional and non-functional requirements is highlighted. Requirement engineering is explained in detail.

1.2 Objective

- Understand the concepts of software processes and software process models
- Know about the fundamental process activities of software requirements engineering, software development, testing, and evolution

1.3 Need for Software Engineering

- We can't run the modern world without software. National infrastructures and utilities are controlled by computer-based systems and most electrical products include a computer and controlling software. Industrial manufacturing and distribution is completely computerized, as is the financial system. Entertainment, including the music industry, computer games, and

film and television, is software intensive. Therefore, software engineering is essential for the functioning of national and international societies.

- Software systems are abstract and intangible. This simplifies software engineering, as there are no natural limits to the potential of software. However, because of the lack of physical constraints, software systems can quickly become extremely complex, difficult to understand, and expensive to change.
- Developing an organizational information system is completely different from developing a controller for a scientific instrument. Neither of these systems has much in common with a graphics-intensive computer game. All of these applications need software engineering; they do not all need the same software engineering techniques.

1.4 Software Crisis

Software crisis is a term used in the early days of computer science for the difficulty of writing useful and efficient computer programs in the required time.

Software crisis is common due to software failures. Software failures are a consequence of two factors:

- **Increasing demands:**As new software engineering techniques help us to build larger, more complex systems, the demands change. Existing software engineering methods cannot cope and new software engineering techniques have to be developed to meet new these new demands.
- **Low expectations:**It is relatively easy to write computer programs without using software engineering methods and techniques. Consequently, their software is often more expensive and less reliable than it should be. We need better software engineering education and training to address this problem.

1.5 Professional Software Development

- Vast majority of software development is a professional activity where software is developed for specific business purposes. Professional software, intended for use by someone apart from its developer, is usually developed by teams rather than individuals. It is maintained and changed throughout its life.
- Software engineering is intended to support professional software development, rather than individual programming. It includes techniques that support program specification, design, and evolution, none of which are normally relevant for personal software development.
- When we are talking about software engineering, software is not just the programs themselves but also all associated documentation and configuration data that is required to make these programs operate correctly. A professionally developed software system is often more than a single program. The system usually consists of a number of separate programs and configuration files that are used to set up these programs.
- If you are writing software that other people will use and other engineers will change then you usually have to provide additional information as well as the code of the program.

Software engineers are concerned with developing software products (i.e., software which can be sold to a customer). There are two kinds of software products.

- **Generic products:**These are stand-alone systems that are produced by a development organization and sold on the open market to any customer who is able to buy them. Examples of this type of product include software for PCs such as databases, word processors.

- **Customized(or bespoke) products:**These are systems that are commissioned by a particular customer. A software contractor develops the software especially for that customer. Examples of this type of software include control systems for electronic devices.

An important difference between these types of software is that, in generic products, the organization that develops the software controls the software specification. For custom products, the specification is usually developed and controlled by the organization that is buying the software. The software developers must work to that specification.

More and more systems are now being built with a generic product as a base, which is then adapted to suit the requirements of a customer. Enterprise Resource Planning (ERP) systems, such as the SAP system, are the best examples of this approach.

When we talk about the quality of professional software, we have to take into account that the software is used and changed by people apart from its developers. Quality is therefore not just concerned with what the software does.

The specific set of attributes that you might expect from a software system obviously depends on its application.

Software engineering

Software engineering is an engineering discipline that is concerned with all aspects of software production from the early stages of system specification through to maintaining the system after it has gone into use. In this definition, there are two key phrases:

- **Engineering discipline:**Engineers make things work. They apply theories, methods, and tools where these are appropriate. However, they use them selectively and always try to discover solutions to problems even when there are no applicable theories and methods. Engineers also recognize that they must work to organizational and financial constraints so they look for solutions within these constraints.
- **All aspects of software production:**Software engineering is not just concerned with the technical processes of software development. It also includes activities such as software project management and the development of tools, methods, and theories to support software production.

Software engineers adopt a systematic and organized approach to their work, as this is often the most effective way to produce high-quality software.

Software engineering is important for two reasons:

- More and more, individuals and society rely on advanced software systems. We need to be able to produce reliable and trustworthy systems economically and quickly.
- It is usually cheaper, in the long run, to use software engineering methods and techniques for software systems rather than just write the programs as if it was a personal programming project.

The systematic approach that is used in software engineering is sometimes called a software process. A software process is a sequence of activities that leads to the production of a software product. There are four fundamental activities that are common to all software processes. These activities are:

- **Software specification:** where customers and engineers define the software that is to be produced and the constraints on its operation.
- **Software development:** where the software is designed and programmed.
- **Software validation:** where the software is checked to ensure that it is what the customer requires.
- **Software evolution:** where the software is modified to reflect changing customer and market requirements.

There are many different types of software. There is no universal software engineering method or technique that is applicable for all of these. However, there are three general issues that affect many different types of software:

- **Heterogeneity:** Increasingly, systems are required to operate as distributed systems across networks that include different types of computer and mobile devices. The challenge here is to develop techniques for building dependable software that is flexible enough to cope with this heterogeneity.
- **Business and social change:** Business and society are changing incredibly quickly as emerging economies develop and new technologies become available. Many traditional software engineering techniques are time consuming and delivery of new systems often takes longer than planned. They need to evolve so that the time required for software to deliver value to its customers is reduced.
- **Security and trust:** As software is intertwined with all aspects of our lives, it is essential that we can trust that software. We have to make sure that malicious users cannot attack our software and that information security is maintained.

Software engineering diversity

The most significant factor in determining which software engineering methods and techniques are most important is the type of application that is being developed. There are many different types of application including:

- **Stand-alone applications:** These are application systems that run on a local computer, such as a PC. They include all necessary functionality and do not need to be connected to a network. Examples of such applications are office applications on a PC, CAD programs, photo manipulation software, etc.
- **Interactive transaction-based applications:** These are applications that execute on a remote computer and that are accessed by users from their own PCs or terminals. Obviously, these include web applications such as e-commerce applications where you can interact with a remote system to buy goods and services. Interactive applications often incorporate a large data store that is accessed and updated in each transaction.
- **Embedded control systems:** These are software control systems that control and manage hardware devices. Examples of embedded systems include the software in a mobile (cell) phone, software that controls anti-lock braking in a car, and software in a microwave oven to control the cooking process.
- **Batch processing systems:** These are business systems that are designed to process data in large batches. They process large numbers of individual inputs to create corresponding outputs. Examples of batch systems include periodic billing systems, such as phone billing systems, and salary payment systems.
- **Entertainment systems:** These are systems that are primarily for personal use and which are intended to entertain the user. The quality of the user interaction offered is the most important distinguishing characteristic of entertainment systems.
- **Systems for modeling and simulation:** These are systems that are developed by scientists and engineers to model physical processes or situations, which include many, separate, interacting objects. These are often computationally intensive and require high-performance parallel systems for execution.
- **Data collection systems:** These are systems that collect data from their environment using a set of sensors and send that data to other systems for processing. The software has to interact with sensors and often is installed in a hostile environment such as inside an engine or in a remote location.

- **Systems of systems:** These are systems that are composed of a number of other software systems.

Software engineering and the Web

The development of the World Wide Web has had a profound effect on all of our lives. Initially, the Web was primarily a universally accessible information store and it had little effect on software systems. Around 2000, the Web started to evolve and more and more functionality was added to browsers. This meant that web-based systems could be developed where, instead of a special-purpose user interface, these systems could be accessed using a web browser.

Instead of writing software and deploying it on users' PCs, the software was deployed on a web server. This made it much cheaper to change and upgrade the software, as there was no need to install the software on every PC. It also reduced costs, as user interface development is particularly expensive.

The next stage in the development of web-based systems was the notion of web services. Web services are software components that deliver specific, useful functionality and which are accessed over the Web. Applications are constructed by integrating these web services, which may be provided by different companies. In principle, this linking can be dynamic so that an application may use different web services each time that it is executed.

In the last few years, the notion of 'software as a service' has been developed. It has been proposed that software will not normally run on local computers but will run on 'computing clouds' that are accessed over the Internet. Users do not buy software but pay according to how much the software is use.

The advent of the web, therefore, has led to a significant change in the way that business software is organized. Before the web, business applications were mostly monolithic, single programs running on single computers or computer clusters. Business applications are not programmed from scratch but involve extensive reuse of components and programs.

This radical change in software organization has, obviously, led to changes in the ways that web-based systems are engineered. For example:

- Software reuse has become the dominant approach for constructing web-based systems.
- It is now generally recognized that it is impractical to specify all the requirements for such systems in advance. Web-based systems should be developed and delivered incrementally.
- User interfaces are constrained by the capabilities of web browsers.

1.6 Software engineering ethics

There are areas where standards of acceptable behavior are not bound by laws but by the more tenuous notion of professional responsibility. Some of these are:

- **Confidentiality:** You should normally respect the confidentiality of your employers or clients irrespective of whether or not a formal confidentiality agreement has been signed.
- **Competence:** You should not misrepresent your level of competence. You should not knowingly accept work that is outside your competence.
- **Intellectual property rights:** You should be aware of local laws governing the use of intellectual property such as patents and copyright. You should be careful to ensure that the intellectual property of employers and clients is protected.
- **Computer misuse:** You should not use your technical skills to misuse other people's computers. Computer misuse ranges from relatively trivial (game playing on an employer's machine, say) to extremely serious (dissemination of viruses or other malware).

1.7 Case studies

An insulin pump control system

An insulin pump is a medical system that simulates the operation of the pancreas (an internal organ). The software controlling this system is an embedded system, which collects information from a sensor and controls a pump that delivers a controlled dose of insulin to a user.

A software-controlled insulin delivery system might work by using a microsensor embedded in the patient to measure some blood parameter that is proportional to the sugar level. This is then sent to the pump controller. This controller computes the sugar level and the amount of insulin that is needed. It then sends signals to a miniaturized pump to deliver the insulin via a permanently attached needle.

Figure 1 shows the hardware components and organization of the insulin pump.

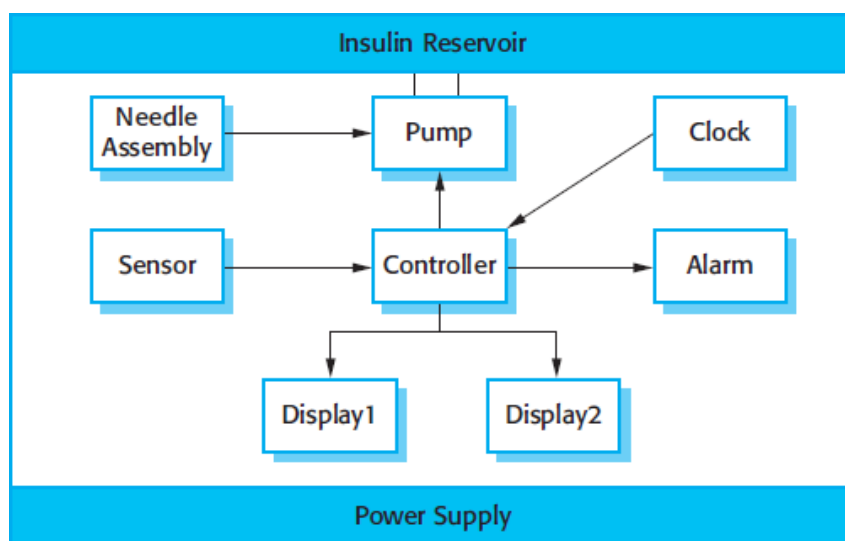


Figure 1: Insulin pump hardware

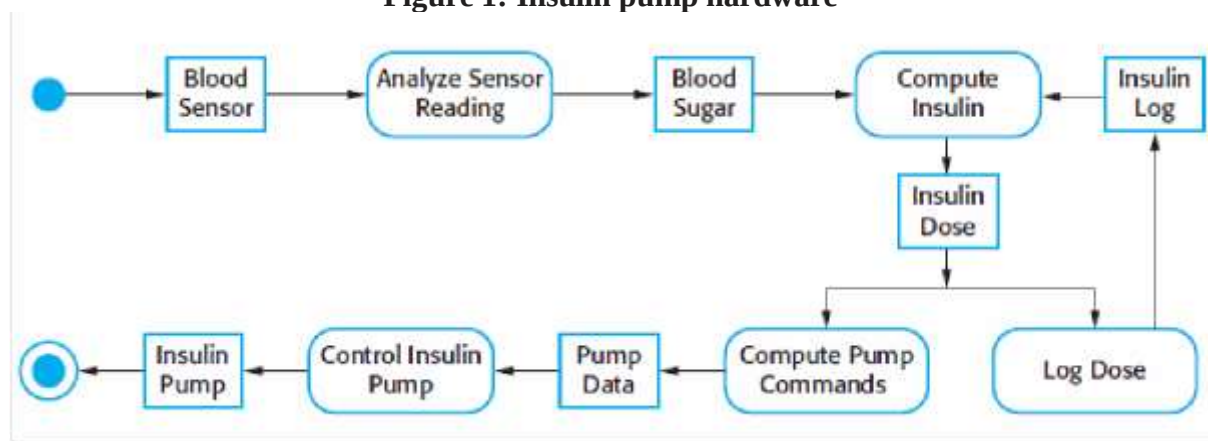


Figure 2: Activity model of the insulin pump

Figure 2 is a UML activity model that illustrates how the software transforms an input blood sugar level to a sequence of commands that drive the insulin pump.

Clearly, this is a safety-critical system. If the pump fails to operate or does not operate correctly, then the user's health may be damaged or they may fall into a coma because their blood

sugar levels are too high or too low. There are, therefore, two essential high-level requirements that this system must meet:

- The system shall be available to deliver insulin when required.
- The system shall perform reliably and deliver the correct amount of insulin to counteract the current level of blood sugar.

A patient information system for mental health care

The MHC-PMS (Mental Health Care-Patient Management System) is an information system that is intended for use in clinics. It makes use of a centralized database of patient information but has also been designed to run on a PC, so that it may be accessed and used from sites that do not have secure network connectivity. When the local systems have secure network access, they use patient information in the database but they can download and use local copies of patient records when they are disconnected.

Figure 3 illustrates the organization of the MHC-PMS. The MHC-PMS has two overall goals:

- To generate management information that allows health service managers to assess performance against local and government targets.
- To provide medical staff with timely information to support the treatment of patients.

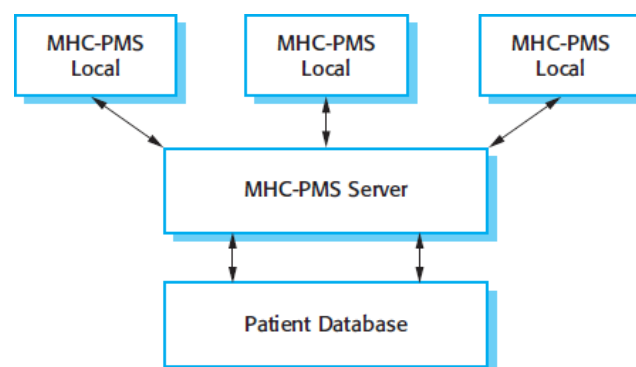


Figure 3: The organization of the MHC-PMS

Where patients are dangerous, they may need to be ‘sectioned’—confined to a secure hospital for treatment and observation.

The key features of the system are:

- **Individual care management:** Clinicians can create records for patients, edit the information in the system, view patient history, etc. The system supports data summaries so that doctors who have not previously met a patient can quickly learn about the key problems and treatments that have been prescribed.
- **Patient monitoring:** The system regularly monitors the records of patients that are involved in treatment and issues warnings if possible problems are detected. Therefore, if a patient has not seen a doctor for some time, a warning may be issued. One of the most important elements of the monitoring system is to keep track of patients who have been sectioned and to ensure that the legally required checks are carried out at the right time.
- **Administrative reporting:** The system generates monthly management reports showing the number of patients treated at each clinic, the number of patients who have entered and left the care system, number of patients sectioned, the drugs prescribed and their costs, etc.

Two different laws affect the system. These are laws on data protection that govern the confidentiality of personal information and mental health laws that govern the compulsory detention

of patients deemed to be a danger to themselves or others. One of the aims of the MHC-PMS is to ensure that staff always acts in accordance with the law and that their decisions are recorded for judicial review if necessary.

A wilderness weather station

Wilderness weather stations are part of a larger system (Figure 4), which is a weather information system that collects data from weather stations and makes it available to other systems for processing. The systems in Figure 4 are:

- **The weather station system:** This is responsible for collecting weather data, carrying out some initial data processing, and transmitting it to the data management system.
- **The data management and archiving system:** This system collects the data from all of the wilderness weather stations, carries out data processing and analysis, and archives the data in a form that can be retrieved by other systems, such as weather forecasting systems.
- **The station maintenance system:** This system can communicate by satellite with all wilderness weather stations to monitor the health of these systems and provide reports of problems. It can update the embedded software in these systems. In the event of system problems, this system can also be used to remotely control a wilderness weather system.

In Figure 4, I have used the UML package symbol to indicate that each system is a collection of components and have identified the separate systems, using the UML stereotype «system». The associations between the packages indicate there is an exchange of information but, at this stage, there is no need to define them in any more detail.

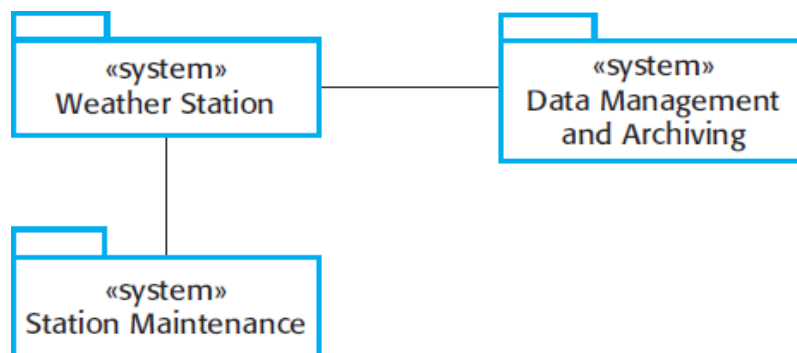


Figure 4: The weather station's environment

The station software is therefore not just concerned with data collection. It must also:

- Monitor the instruments, power, and communication hardware and report faults to the management system.
- Manage the system power, ensuring that batteries are charged whenever the environmental conditions permit but also that generators are shut down in potentially damaging weather conditions, such as high wind.
- Allow for dynamic reconfiguration where parts of the software are replaced with new versions and where backup instruments are switched into the system in the event of system failure.

1.8 Software process models

Software process model is a simplified representation of a software process.

The process models are:

- **The waterfall model:** This takes the fundamental process activities of specification, development, validation, and evolution and represents them as separate process phases such as requirements specification, software design, implementation, testing, and so on.
- **Incremental development:** This approach interleaves the activities of specification, development, and validation. The system is developed as a series of versions (increments), with each version adding functionality to the previous version.
- **Spiral model:** Here, the software process is represented as a spiral, rather than a sequence of activities with some backtracking from one activity to another.

1.8 The waterfall Model

This model is illustrated in Figure 5. Because of the cascade from one phase to another, this model is known as the ‘waterfall model’ or software life cycle. The waterfall model is an example of a plan-driven process—in principle, you must plan and schedule all of the process activities before starting work on them.

The principal stages of the waterfall model directly reflect the fundamental development activities:

- **Requirements analysis and definition:** The system’s services, constraints, and goals are established by consultation with system users. They are then defined in detail and serve as a system specification.
- **System and software design:** The systems design process allocates the requirements to either hardware or software systems by establishing an overall system architecture. Software design involves identifying and describing the fundamental software system abstractions and their relationships.
- **Implementation and unit testing:** During this stage, the software design is realized as a set of programs or program units. Unit testing involves verifying that each unit meets its specification.

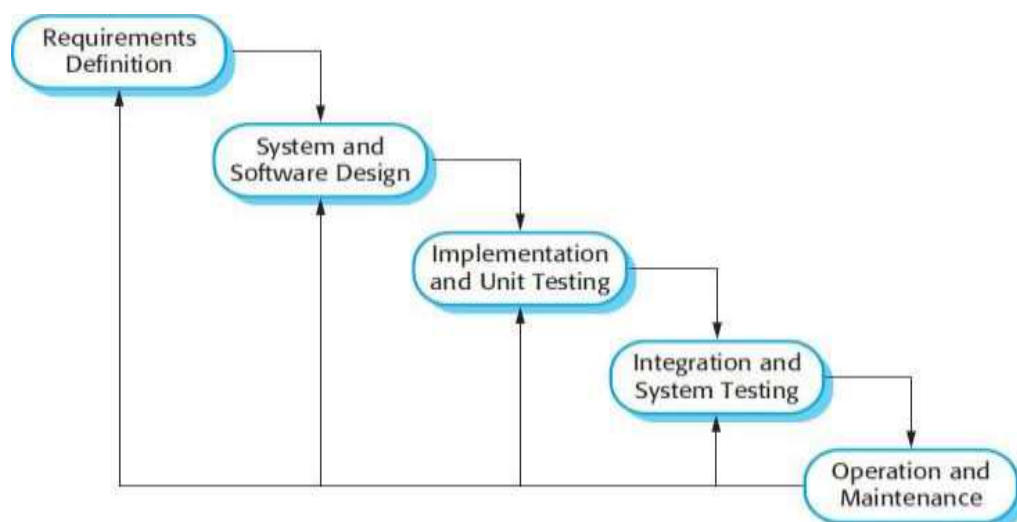


Figure 5: The Waterfall Model

- **Integration and system testing:** The individual program units or programs are integrated and tested as a complete system to ensure that the software requirements have been met. After testing, the software system is delivered to the customer.
- **Operation and maintenance:** Normally (although not necessarily), this is the longest life cycle phase. The system is installed and put into practical use. Maintenance involves correcting errors which were not discovered in earlier stages of the life cycle, improving the implementation of system units and enhancing the system's services as new requirements are discovered.

Disadvantages of waterfall model

- In principle, the result of each phase is one or more documents that are approved ('signed off'). The following phase should not start until the previous phase has finished. In practice, these stages overlap and feed information to each other.
- The software process is not a simple linear model but involves feedback from one phase to another. Documents produced in each phase may then have to be modified to reflect the changes made.
- Because of the costs of producing and approving documents, iterations can be costly and involve significant rework. Therefore, after a small number of iterations, it is normal to freeze parts of the development, such as the specification, and to continue with the later development stages. This premature freezing of requirements may mean that the system won't do what the user wants.
- During the final life cycle phase (operation and maintenance) the software is put into use. Errors and omissions in the original software requirements are discovered. Program and design errors emerge and the need for new functionality is identified. Making these changes (software maintenance) may involve repeating previous process stages.
- Its major problem is the inflexible partitioning of the project into distinct stages. Commitments must be made at an early stage in the process, which makes it difficult to respond to changing customer requirements.

In principle, the waterfall model should only be used when the requirements are well understood and unlikely to change radically during system development.

1.9 Incremental development

Incremental development is based on the idea of developing an initial implementation, exposing this to user comment and evolving it through several versions until an adequate system has been developed (Figure 6). Specification, development, and validation activities are interleaved rather than separate, with rapid feedback across activities.

It's better than a waterfall approach for most business, e-commerce, and personal systems. By developing the software incrementally, it is cheaper and easier to make changes in the software as it is being developed. Each increment or version of the system incorporates some of the functionality that is needed by the customer. Generally, the early increments of the system include the most important or most urgently required functionality. This means that the customer can evaluate the system at a relatively early stage in the development to see if it delivers what is required. If not, then only the current increment has to be changed and, possibly, new functionality defined for later increments.

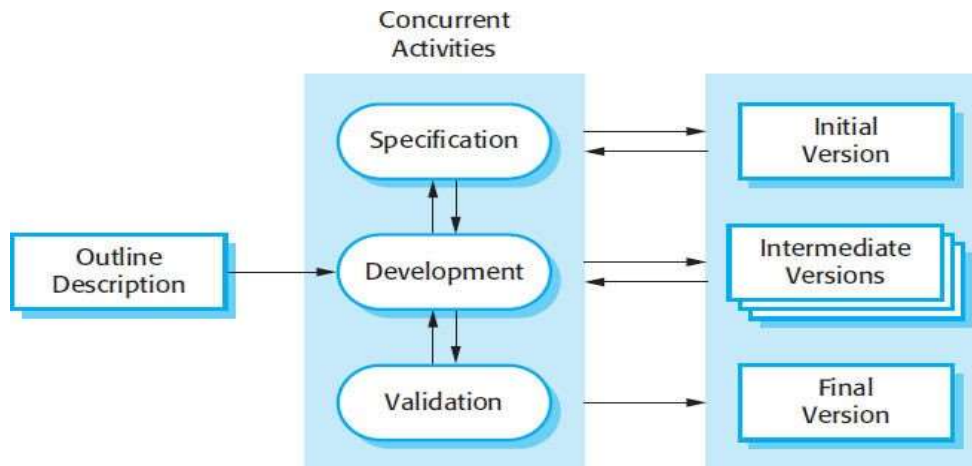


Figure 6: Incremental development

Incremental development has three important benefits, compared to the waterfall model:

- The cost of accommodating changing customer requirements is reduced.
- It is easier to get customer feedback on the development work that has been done.
- More rapid delivery and deployment of useful software to the customer is possible, even if all of the functionality has not been included.

Disadvantages of incremental model

- The process is not visible. Managers need regular deliverables to measure progress.
- System structure tends to degrade as new increments are added. Unless time and money is spent on refactoring to improve the software, regular change tends to corrupt its structure. Incorporating further software changes becomes increasingly difficult and costly.

1.10 Spiral model

The spiral model is shown in Figure 7. Here, the software process is represented as a spiral, rather than a sequence of activities with some backtracking from one activity to another. Each loop in the spiral represents a phase of the software process. Thus, the innermost loop might be concerned with system feasibility, the next loop with requirements definition, the next loop with system design, and so on. The spiral model combines change avoidance with change tolerance. It assumes that changes are a result of project risks and includes explicit risk management activities to reduce these risks.

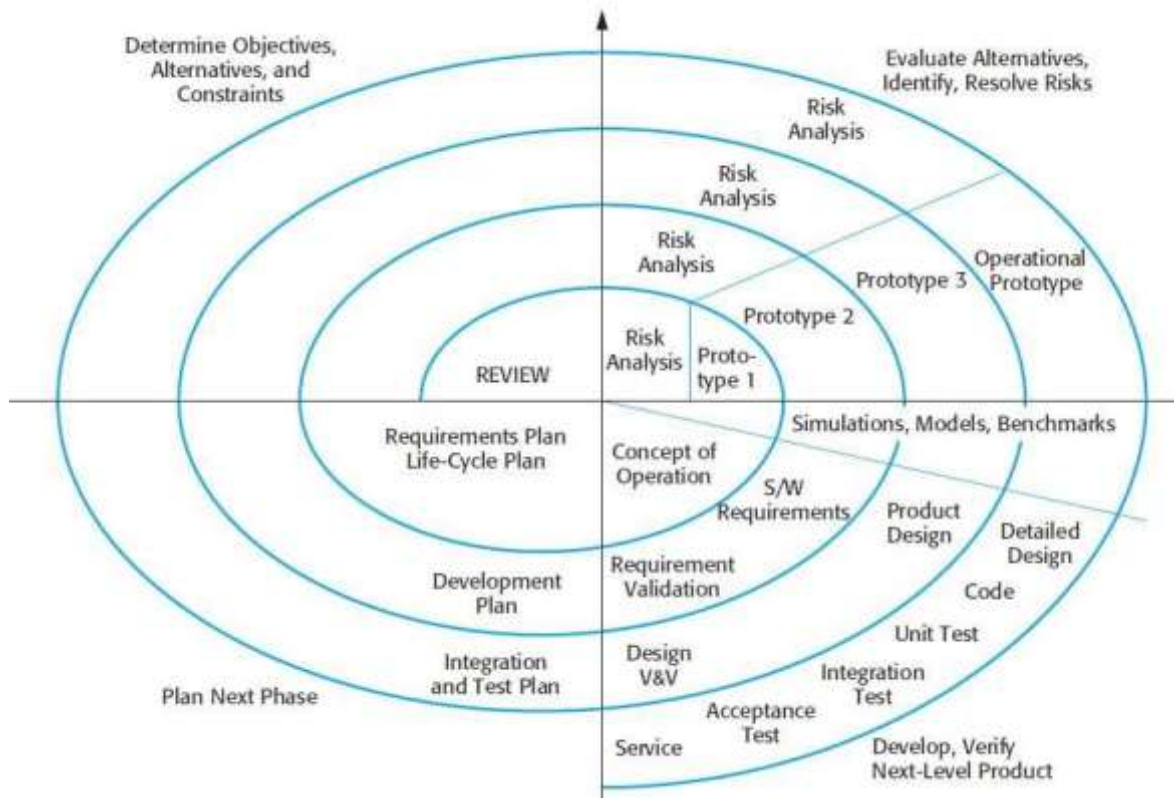


Figure 7: spiral model of the software process

Each loop in the spiral is split into four sectors:

- **Objective setting:** Specific objectives for that phase of the project are defined. Constraints on the process and the product are identified and a detailed management plan is drawn up. Project risks are identified. Alternative strategies, depending on these risks, may be planned.
- **Risk assessment and reduction:** For each of the identified project risks, a detailed analysis is carried out. Steps are taken to reduce the risk.
- **Development and validation:** After risk evaluation, a development model for the system is chosen.
- **Planning:** The project is reviewed and a decision made whether to continue with a further loop of the spiral. If it is decided to continue, plans are drawn up for the next phase of the project.

The main difference between the spiral model and other software process models is its explicit recognition of risk.

1.11 Process Activities

The four basic process activities of specification, development, validation, and evolution are organized differently in different development processes.

Software specification

Software specification or requirements engineering is the process of understanding and defining what services are required from the system and identifying the constraints on the system's operation and development. The requirements engineering process (Figure 8) aims to produce an agreed requirements document that specifies system satisfying stakeholder requirements.

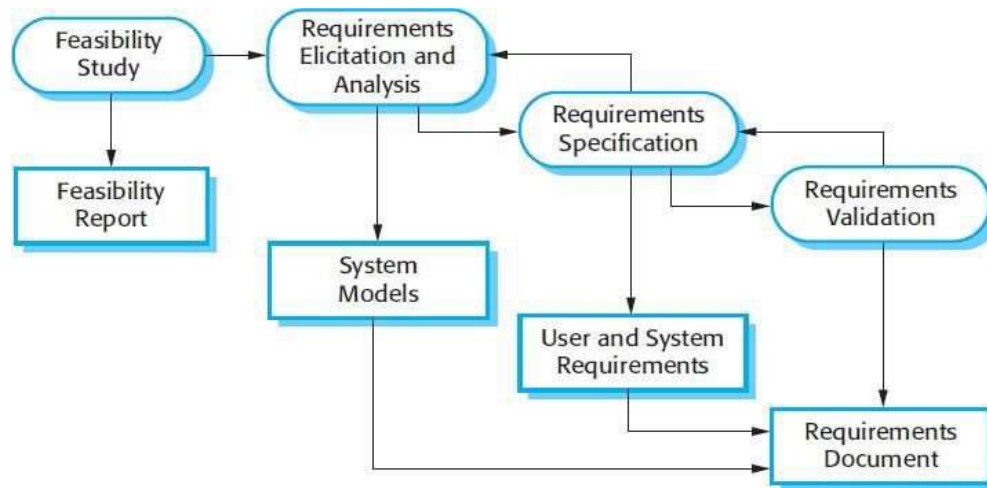


Figure 8: The requirements engineering process

There are four main activities in the requirements engineering process:

- **Feasibility study:** An estimate is made of whether the identified user needs may be satisfied using current software and hardware technologies. The study considers whether the proposed system will be cost-effective from a business point of view and if it can be developed within existing budgetary constraints.
- **Requirements elicitation and analysis:** This is the process of deriving the system requirements through observation of existing systems, discussions with potential users and procurers, task analysis, and so on.
- **Requirements specification:** Requirements specification is the activity of translating the information gathered during the analysis activity into a document that defines a set of requirements. Two types of requirements may be included in this document. User requirements are abstract statements of the system requirements for the customer and end-user of the system; system requirements are a more detailed description of the functionality to be provided.
- **Requirements validation:** This activity checks the requirements for realism, consistency, and completeness. During this process, errors in the requirements document are inevitably discovered. It must then be modified to correct these problems.

Software design and implementation

The implementation stage of software development is the process of converting a system specification into an executable system. It always involves processes of software design and programming.

Information about the platform is an essential input to the design process, as designers must decide how best to integrate it with the software's environment. The requirements specification is a description of the functionality the software must provide and its performance and dependability requirements. If the system is to process existing data, then the description of that data may be

included in the platform specification; otherwise, the data description must be an input to the design process so that the system data organization to be defined.

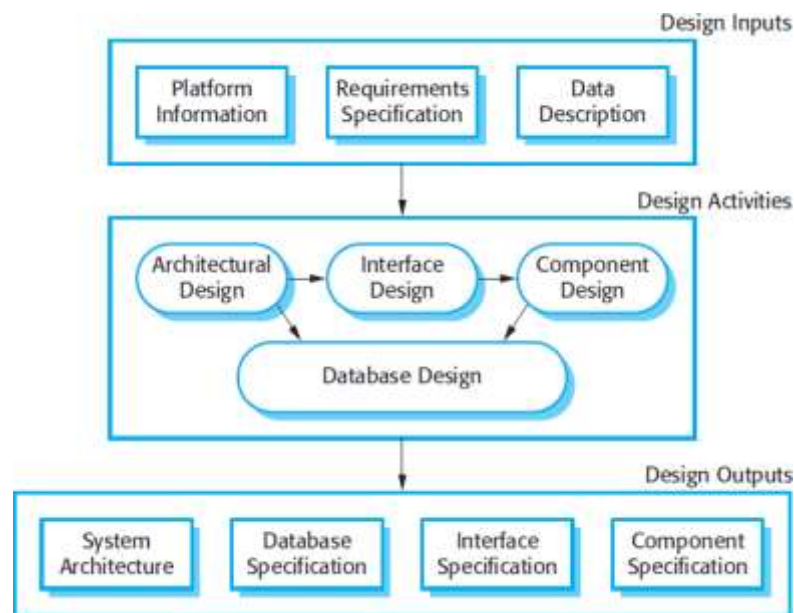


Figure 9: A general model of the design process

Figure 9 shows four activities that may be part of the design process for information systems:

- **Architectural design:** where you identify the overall structure of the system, the principal components (sometimes called sub-systems or modules), their relationships, and how they are distributed.
- **Interface design:** where you define the interfaces between system components. This interface specification must be unambiguous. With a precise interface, a component can be used without other components having to know how it is implemented.
- **Component design:** where you take each system component and design how it will operate. This may be a simple statement of the expected functionality to be implemented, with the specific design left to the programmer.
- **Database design:** where you design the system data structures and how these are to be represented in a database. Again, the work here depends on whether an existing database is to be reused or a new database is to be created.

These activities lead to a set of design outputs, which are also shown in Figure 9.

Software Validation

Software validation or, more generally, verification and validation (V&V) is intended to show that a system both conforms to its specification and that it meets the expectations of the system customer.

Figure 10 shows a three-stage testing process in which system components are tested then the integrated system is tested and, finally, the system is tested with the customer's data.

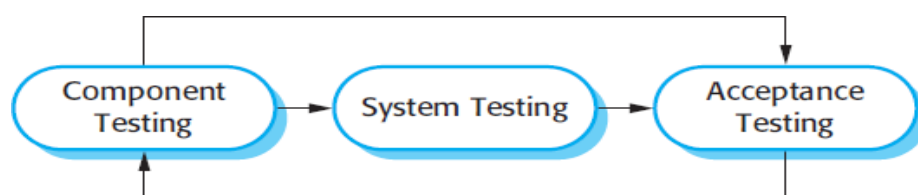


Figure 10: Stages of testing

The stages in the testing process are:

- **Development testing:** The components making up the system are tested by the people developing the system. Each component is tested independently, without other system components.
- **System testing:** System components are integrated to create a complete system. This process is concerned with finding errors that result from unanticipated interactions between components and component interface problems.
- **Acceptance testing:** This is the final stage in the testing process before the system is accepted for operational use. The system is tested with data supplied by the system customer rather than with simulated test data.

Software evolution

The flexibility of software systems is one of the main reasons why more and more software is being incorporated in large, complex systems. Once a decision has been made to manufacture hardware, it is very expensive to make changes to the hardware design. However, changes can be made to software at any time during or after the system development. Even extensive changes are still much cheaper than corresponding changes to system hardware.

1.12 Requirement Engineering

The requirements for a system are the descriptions of what the system should do— the services that it provides and the constraints on its operation. These requirements reflect the needs of customers for a system that serves a certain purpose such as controlling a device, placing an order, or finding information. The process of finding out, analyzing, documenting and checking these services and constraints is called requirements engineering (RE).

- **User requirements** are statements, in a natural language plus diagrams, of what services the system is expected to provide to system users and the constraints under which it must operate.
- **System requirements** are more detailed descriptions of the software system's functions, services, and operational constraints. The system requirements document (sometimes called a functional specification) should define exactly what is to be implemented. It may be part of the contract between the system buyer and the software developers.

1.13 Functional and non-functional requirements

Software system requirements are often classified as functional requirements or non-functional requirements:

- **Functional requirements:** These are statements of services the system should provide, how the system should react to particular inputs, and how the system should behave in particular situations. In some cases, the functional requirements may also explicitly state what the system should not do.
- **Non-functional requirements:** These are constraints on the services or functions offered by the system. They include timing constraints, constraints on the development process, and constraints imposed by standards. Non-functional requirements often apply to the system as a whole, rather than individual system features or services.

Functional requirements

The functional requirements for a system describe what the system should do. These requirements depend on the type of software being developed, the expected users of the software, and the general approach taken by the organization when writing requirements. When expressed as user requirements, functional requirements are usually described in an abstract way that can be understood by system users.

Functional system requirements vary from general requirements covering what the system should do to very specific requirements reflecting local ways of working or an organization's existing systems.

Functional user requirements define specific facilities to be provided by the system. These have been taken from the user requirements document and they show that functional requirements may be written at different levels of detail.

Imprecision in the requirements specification is the cause of many software engineering problems. It is natural for a system developer to interpret an ambiguous requirement in a way that simplifies its implementation. New requirements have to be established and changes made to the system. Of course, this delays system delivery and increases costs.

In principle, the functional requirements specification of a system should be both complete and consistent. Completeness means that all services required by the user should be defined. Consistency means that requirements should not have contradictory definitions. In practice, for large, complex systems, it is practically impossible to achieve requirements consistency and completeness. One reason for this is that it is easy to make mistakes and omissions when writing specifications for complex systems. Another reason is that there are many stakeholders in a large system. A stakeholder is a person or role that is affected by the system in some way. Stakeholders have different and often inconsistent needs.

Non-functional requirements

Non-functional requirements, as the name suggests, are requirements that are not directly concerned with the specific services delivered by the system to its users. They may relate to emergent system properties such as reliability, response time, and store occupancy. Alternatively, they may define constraints on the system implementation such as the capabilities of I/O devices or the data representations used in interfaces with other systems.

Non-functional requirements, such as performance, security, or availability, usually specify or constrain characteristics of the system as a whole. However, failing to meet a non-functional requirement can mean that the whole system is unusable.

The implementation of these requirements may be diffused throughout the system. There are two reasons for this:

- Non-functional requirements may affect the overall architecture of a system rather than the individual components.
- A single non-functional requirement, such as a security requirement, may generate a number of related functional requirements that define new system services that are required. In addition, it may also generate requirements that restrict existing requirements.

Figure 11 is a classification of non-functional requirements.

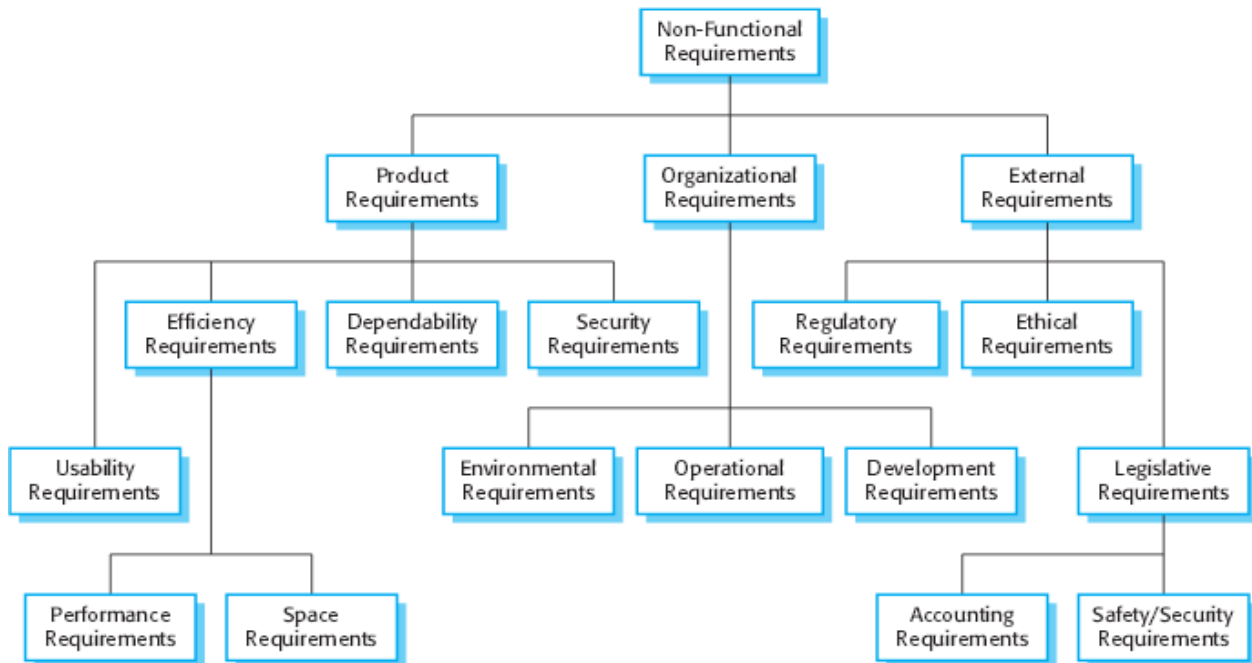


Figure 11: Types of non-functional requirement

- **Product requirements:** These requirements specify or constrain the behavior of the software. Examples include performance requirements on how fast the system must execute and how much memory it requires, reliability requirements that set out the acceptable failure rate, security requirements, and usability requirements.
- **Organizational requirements:** These requirements are broad system requirements derived from policies and procedures in the customer's and developer's organization. Examples include operational process requirements that define how the system will be used, development process requirements that specify the programming language.
- **External requirements:** This broad heading covers all requirements that are derived from factors external to the system and its development process. These may include regulatory requirements that set out what must be done for the system to be approved for use by a regulator, such as a central bank; legislative requirements that must be followed to ensure that the system operates within the law; and ethical requirements that ensure that the system will be acceptable to its users and the general public.

Common problems with non-functional requirements

- Users or customers often propose these requirements as general goals, such as ease of use, the ability of the system to recover from failure, or rapid user response. Goals set out good intentions but cause problems for system developers as they leave scope for interpretation and subsequent dispute once the system is delivered.
- In practice, customers for a system often find it difficult to translate their goals into measurable requirements.
- Non-functional requirements often conflict and interact with other functional or non-functional requirements.
- It is difficult, in practice, to separate functional and non-functional requirements in the requirements document.

1.14 The software requirements document

The software requirements document (sometimes called the software requirements specification or SRS) is an official statement of what the system developers should implement. It should include both the user requirements for a system and a detailed specification of the system requirements. Sometimes, the user and system requirements are integrated into a single description. In other cases, the user requirements are defined in an introduction to the system requirements specification. If there are a large number of requirements, the detailed system requirements may be presented in a separate document. Requirements documents are essential when an outside contractor is developing the software system.

The requirements document has a diverse set of users, ranging from the senior management of the organization that is paying for the system to the engineers responsible for developing the software. Figure 12 shows possible users of the document and how they use it.

The diversity of possible users means that the requirements document has to be a compromise between communicating the requirements to customers, defining the requirements in precise detail for developers and testers, and including information about possible system evolution. The level of detail that you should include in a requirements document depends on the type of system that is being developed and the development process used.

One possible organization for a requirements document that is based on an IEEE standard for requirements documents. According to that, the organization on requirement document is as follows: Preface, introduction, glossary, user requirements definition, system architecture, system requirements specification, system models, system evolution, appendices, and index.

Naturally, the information that is included in a requirements document depends on the type of software being developed and the approach to development that is to be used.

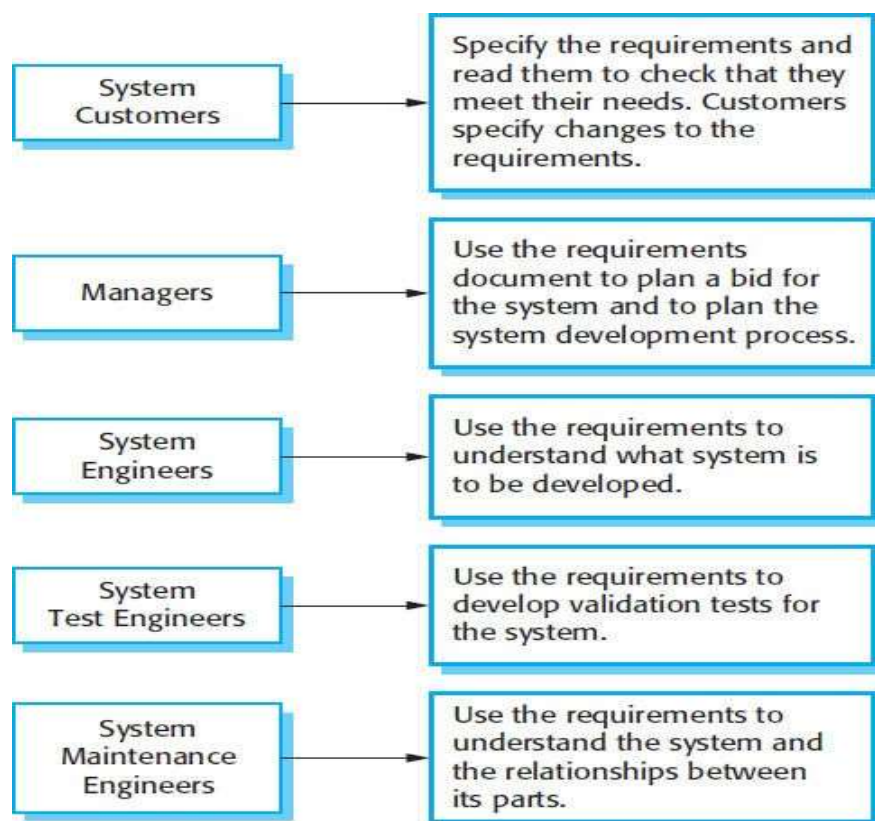


Figure 12: Users of arequirements document

1.15 Requirement specification

Requirements specification is the process of writing down the user and system requirements in a requirements document. Ideally, the user and system requirements should be clear, unambiguous, easy to understand, complete, and consistent.

The user requirements for a system should describe the functional and non-functional requirements so that they are understandable by system users who don't have detailed technical knowledge. Ideally, they should specify only the external behaviour of the system. The requirements document should not include details of the system architecture or design. Consequently, if you are writing user requirements, you should not use software jargon, structured notations, or formal notations. You should write user requirements in natural language, with simple tables, forms, and intuitive diagrams.

System requirements are expanded versions of the user requirements that are used by software engineers as the starting point for the system design. They add detail and explain how the user requirements should be provided by the system. They may be used as part of the contract for the implementation of the system and should therefore be a complete and detailed specification of the whole system.

Ideally, the system requirements should simply describe the external behaviour of the system and its operational constraints. They should not be concerned with how the system should be designed or implemented.

User requirements are almost always written in natural language supplemented by appropriate diagrams and tables in the requirements document. System requirements may also be written in natural language but other notations based on forms, graphical system models, or mathematical system models can also be used.

Natural language specification

Natural language has been used to write requirements for software since the beginning of software engineering. It is expressive, intuitive, and universal. It is also potentially vague, ambiguous, and its meaning depends on the background of the reader.

To minimize misunderstandings when writing natural language requirements, I recommend that you follow some simple guidelines:

- Invent a standard format and ensure that all requirement definitions adhere to that format. Standardizing the format makes omissions less likely and requirements easier to check.
- Use language consistently to distinguish between mandatory and desirable requirements. Mandatory requirements are requirements that the system must support and are usually written using 'shall'. Desirable requirements are not essential and are written using 'should'.
- Use text highlighting (bold, italic, or colour) to pick out key parts of the requirement.
- Do not assume that readers understand technical software engineering language. It is easy for words like 'architecture' and 'module' to be misunderstood. You should, therefore, avoid the use of jargon, abbreviations, and acronyms.
- Whenever possible, you should try to associate a rationale with each user requirement. The rationale should explain why the requirement has been included.

Structured specification

Structured natural language is a way of writing system requirements where the freedom of the requirements writer is limited and all requirements are written in a standard way. This approach maintains most of the expressiveness and understand-ability of natural language but ensures that

some uniformity is imposed on the specification. Structured language notations use templates to specify system requirements. The specification may use programming language constructs to show alternatives and iteration, and may highlight key elements using shading or different fonts.

To use a structured approach to specifying system requirements, you define one or more standard templates for requirements and represent these templates as structured forms.

When a standard form is used for specifying functional requirements, the following information should be included:

- A description of the function or entity being specified.
- A description of its inputs and where these come from.
- A description of its outputs and where these go to.
- Information about the information that is needed for the computation or other entities in the system that are used (the 'requires' part).
- A description of the action to be taken.
- If a functional approach is used, a pre-condition setting out what must be true before the function is called, and a post-condition specifying what is true after the function is called.
- A description of the side effects (if any) of the operation.

Using structured specifications removes some of the problems of natural language specification.

1.16 Requirements engineering process

Requirements engineering processes may include four high-level activities. These focus on assessing if the system is useful to the business (feasibility study), discovering requirements (elicitation and analysis), converting these requirements into some standard form (specification), and checking that the requirements actually define the system that the customer wants (validation).

Figure 13 shows this interleaving. The activities are organized as an iterative process around a spiral, with the output being a system requirements document. The amount of time and effort devoted to each activity in each iteration depends on the stage of the overall process and the type of system being developed. Early in the process, most effort will be spent on understanding high-level business and non-functional requirements, and the user requirements for the system. Later in the process, in the outer rings of the spiral, more effort will be devoted to eliciting and understanding the detailed system requirements.

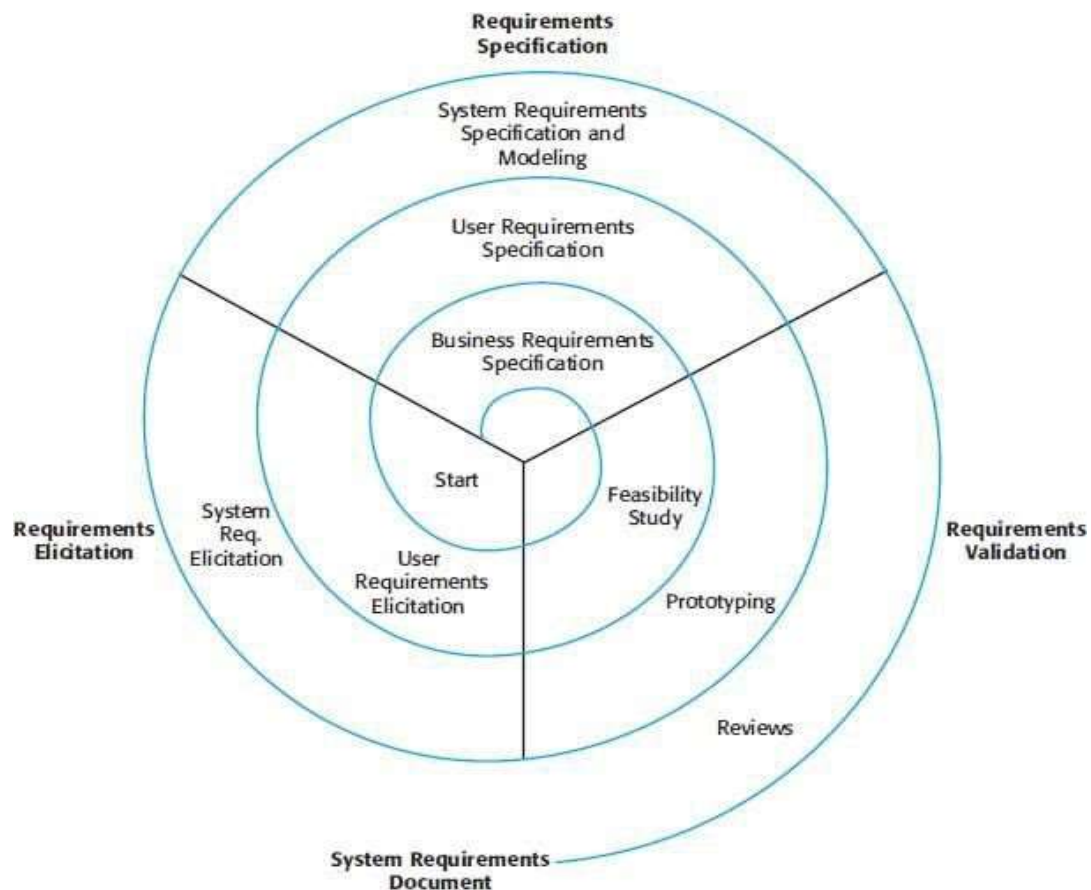


Figure 12: A spiral view of the requirements engineering process

This spiral model accommodates approaches to development where the requirements are developed to different levels of detail. The number of iterations around the spiral can vary so the spiral can be exited after some or all of the user requirements have been elicited.

Some people consider requirements engineering to be the process of applying a structured analysis method, such as object-oriented analysis. The set of models describes the behaviour of the system and is annotated with additional information.

In virtually all systems, requirements change. The people involved develop a better understanding of what they want the software to do; the organization buying the system changes; modifications are made to the system's hardware, software, and organizational environment. The process of managing these changing requirements is called requirements management.

1.17 Requirements elicitation and analysis

After an initial feasibility study, the next stage of the requirements engineering process is requirements elicitation and analysis. In this activity, software engineers work with customers and system end-users to find out about the application domain, what services the system should provide, the required performance of the system, hardware constraints, and so on.

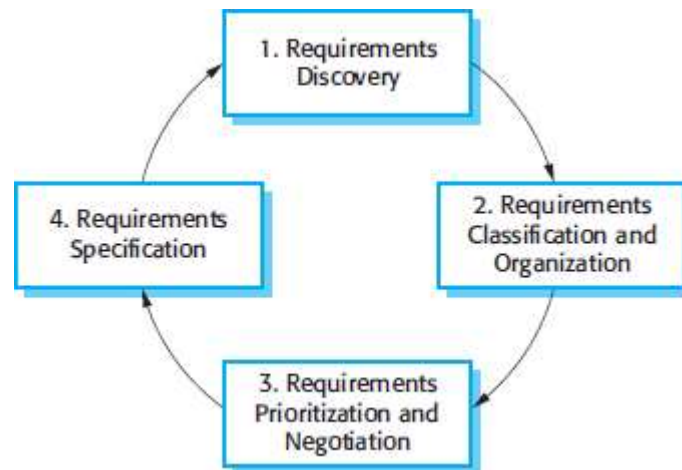


Figure 13: The requirements elicitation and analysis process

Requirements elicitation and analysis may involve a variety of different kinds of people in an organization. A system stakeholder is anyone who should have some direct or indirect influence on the system requirements. Stakeholders include end-users who will interact with the system and anyone else in an organization who will be affected by it. Other system stakeholders might be engineers who are developing or maintaining other related systems, business managers, domain experts, and trade union representatives.

A process model of the elicitation and analysis process is shown in Figure 13. The process activities are:

- **Requirements discovery:** This is the process of interacting with stakeholders of the system to discover their requirements. Domain requirements from stakeholders and documentation are also discovered during this activity.
- **Requirements classification and organization:** This activity takes the unstructured collection of requirements, groups related requirements, and organizes them into coherent clusters.
- **Requirements prioritization and negotiation:** Inevitably, when multiple stakeholders are involved, requirements will conflict. This activity is concerned with prioritizing requirements and finding and resolving requirements conflicts through negotiation. Usually, stakeholders have to meet to resolve differences and agree on compromise requirements.

Eliciting and understanding requirements from system stakeholders is a difficult process for several reasons:

- Stakeholders often don't know what they want from a computer system except in the most general terms.
- Stakeholders in a system naturally express requirements in their own terms and with implicit knowledge of their own work.
- Different stakeholders have different requirements and they may express these in different ways.
- Political factors may influence the requirements of a system. Managers may demand specific system requirements because these will allow them to increase their influence in the organization.
- The economic and business environment in which the analysis takes place is dynamic.

Requirements discovery

Requirements discovery (sometime called requirements elicitation) is the process of gathering information about the required system and existing systems, and distilling the user and system requirements from this information. Sources of information during the requirements discovery phase include documentation, system stakeholders, and specifications of similar systems. You interact with stakeholders through interviews and observation and you may use scenarios and prototypes to help stakeholders understand what the system will be like.

Stakeholders range from end-users of a system through managers to external stakeholders such as regulators, who certify the acceptability of the system. In addition to system stakeholders, we have already seen that requirements may also come from the application domain and from other systems that interact with the system being specified. All of these must be considered during the requirements elicitation process.

These different requirements sources (stakeholders, domain, systems) can all be represented as system viewpoints with each viewpoint showing a subset of the requirements for the system. Different viewpoints on a problem see the problem in different ways. However, their perspectives are not completely independent but usually overlap so that they have common requirements. You can use these viewpoints to structure both the discovery and the documentation of the system requirements.

Interviewing

Formal or informal interviews with system stakeholders are part of most requirements engineering processes. In these interviews, the requirements engineering team puts questions to stakeholders about the system that they currently use and the system to be developed. Requirements are derived from the answers to these questions.

Interviews may be of two types:

- **Closed interviews:** where the stakeholder answers a pre-defined set of questions.
- **Open interviews:** in which there is no pre-defined agenda. The requirements engineering team explores a range of issues with system stakeholders and hence develop a better understanding of their needs.

It can be difficult to elicit domain knowledge through interviews for two reasons:

- All application specialists use terminology and jargon that are specific to a domain. It is impossible for them to discuss domain requirements without using this terminology. They normally use terminology in a precise and subtle way that is easy for requirements engineers to misunderstand.
- Some domain knowledge is so familiar to stakeholders that they either find it difficult to explain or they think it is so fundamental that it isn't worth mentioning.

Effective interviewers have two characteristics:

- They are open-minded, avoid pre-conceived ideas about the requirements, and are willing to listen to stakeholders. If the stakeholder comes up with surprising requirements, then they are willing to change their mind about the system.
- They prompt the interviewee to get discussions going using a springboard question, a requirements proposal, or by working together on a prototype system. Saying to people 'tell me what you want' is unlikely to result in useful information. They find it much easier to talk in a defined context rather than in general terms.

Scenarios

People usually find it easier to relate to real-life examples rather than abstract descriptions. They can understand and criticize a scenario of how they might interact with a software system. Requirements engineers can use the information gained from this discussion to formulate the actual system requirements.

Scenarios can be particularly useful for adding detail to an outline requirements description. Each scenario usually covers one or a small number of possible interactions. Different forms of scenarios are developed and they provide different types of information at different levels of detail about the system.

Scenario-based elicitation involves working with stakeholders to identify scenarios and to capture details to be included in these scenarios. Scenarios may be written as text, supplemented by diagrams, screen shots, etc. Alternatively, a more structured approach such as event scenarios or use cases may be used.

Use cases

Use cases are the fundamental feature of the unified modeling language. In their simplest form, a use case identifies the actors involved in an interaction and names the type of interaction. This is then supplemented by additional information describing the interaction with the system. The additional information may be a textual description or one or more graphical models such as UML sequence or state charts.

Use cases are documented using a high-level use case diagram. The set of use cases represents all of the possible interactions that will be described in the system requirements. Actors in the process, who may be human or other systems, are represented as stick figures. Each class of interaction is represented as a named ellipse. Lines link the actors with the interaction. Optionally, arrowheads may be added to lines to show how the interaction is initiated. This is illustrated in Figure 14, which shows some of the use cases for the patient information system.

Scenarios and use cases are effective techniques for eliciting requirements from stakeholders who interact directly with the system. The UML is a de facto standard for object-oriented modeling, so use cases and use case-based elicitation are now widely used for requirements elicitation.

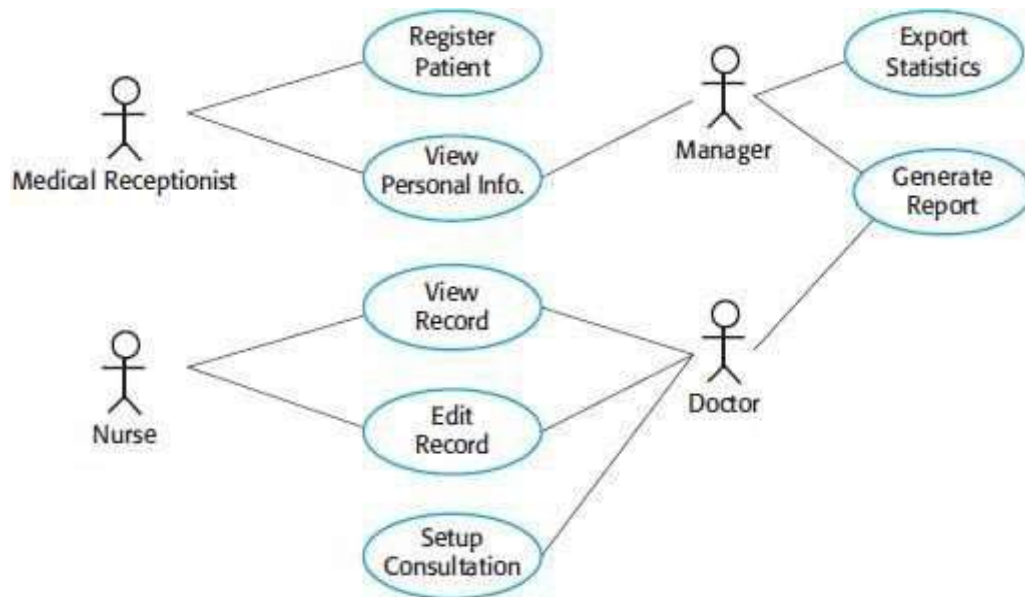


Figure 14: Use cases for the MHC-PMS

Ethnography

Software systems do not exist in isolation. They are used in a social and organizational context and software system requirements may be derived or constrained by that context. Satisfying these social and organizational requirements is often critical for the success of the system. One reason why many software systems are delivered but never used is that their requirements do not take proper account of how the social and organizational context affects the practical operation of the system.

Ethnography is an observational technique that can be used to understand operational processes and help derive support requirements for these processes. An analyst immerses himself or herself in the working environment where the system will be used. The day-to-day work is observed and notes made of the actual tasks in which participants are involved. The value of ethnography is that it helps discover implicit system requirements that reflect the actual ways that people work, rather than the formal processes defined by the organization.

Ethnography is particularly effective for discovering two types of requirements:

- Requirements that are derived from the way in which people actually work, rather than the way in which process definitions say they ought to work.
- Requirements that are derived from cooperation and awareness of other people's activities.

1.18 Requirements validation

Requirements validation is the process of checking that requirements actually define the system that the customer really wants. It overlaps with analysis as it is concerned with finding problems with the requirements. Requirements validation is important because errors in a requirements document can lead to extensive rework costs when these problems are discovered during development or after the system is in service.

During the requirements validation process, different types of checks should be carried out on the requirements in the requirements document. These checks include:

- **Validity checks:** A user may think that a system is needed to perform certain functions. However, further thought and analysis may identify additional or different functions that are required. Systems have diverse stakeholders with different needs and any set of requirements is inevitably a compromise across the stakeholder community.
- **Consistency checks:** Requirements in the document should not conflict. That is, there should not be contradictory constraints or different descriptions of the same system function.
- **Completeness checks:** The requirements document should include requirements that define all functions and the constraints intended by the system user.
- **Realism checks:** Using knowledge of existing technology, the requirements should be checked to ensure that they can actually be implemented. These checks should also take account of the budget and schedule for the system development.
- **Verifiability:** To reduce the potential for dispute between customer and contractor, system requirements should always be written so that they are verifiable. This means that you should be able to write a set of tests that can demonstrate that the delivered system meets each specified requirement.

There are a number of requirements validation techniques that can be used individually or in conjunction with one another:

- **Requirements reviews:** The requirements are analyzed systematically by a team of reviewers who check for errors and inconsistencies.
- **Prototyping:** In this approach to validation, an executable model of the system in question is demonstrated to end-users and customers. They can experiment with this model to see if it meets their real needs.
- **Test-case generation:** Requirements should be testable. If the tests for the requirements are devised as part of the validation process, this often reveals requirements problems. If a test is difficult or impossible to design, this usually means that the requirements will be difficult to implement and should be reconsidered.

1.19 Requirements management

The requirements for large software systems are always changing. During the software process, the stakeholders' understanding of the problem is constantly changing (Figure 15). The system requirements must then also evolve to reflect this changed problem view.

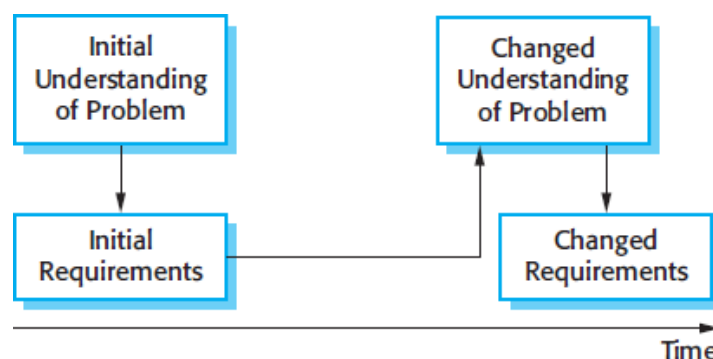


Figure 15: Requirements evolution

Once a system has been installed and is regularly used, new requirements inevitably emerge. It is hard for users and system customers to anticipate what effects the new system will have on their

business processes and the way that work is done. Once end-users have experience of a system, they will discover new needs and priorities. There are several reasons why change is inevitable:

- The business and technical environment of the system always changes after installation.
- The people who pay for a system and the users of that system are rarely the same people.
- Large systems usually have a diverse user community, with many users having different requirements and priorities that may be conflicting or contradictory.
- Requirements management is the process of understanding and controlling changes to system requirements.

Requirements management planning

Planning is an essential first stage in the requirements management process. The planning stage establishes the level of requirements management detail that is required. During the requirements management stage, you have to decide on:

- **Requirements identification:** Each requirement must be uniquely identified so that it can be cross-referenced with other requirements and used in traceability assessments.
- **A change management process:** This is the set of activities that assess the impact and cost of changes.
- **Traceability policies:** These policies define the relationships between each requirement and between the requirements and the system design that should be recorded.
- **Tool support:** Requirements management involves the processing of large amounts of information about the requirements. Tools that may be used range from specialist requirements management systems to spreadsheets and simple database systems.

Requirements management needs automated support and the software tools for this should be chosen during the planning phase. You need tool support for:

- **Requirements storage:** The requirements should be maintained in a secure, managed data store that is accessible to everyone involved in the requirements engineering process.
- **Change management:** The process of change management (Figure 16) is simplified if active tool support is available.
- **Traceability management:** As discussed above, tool support for traceability allows related requirements to be discovered. Some tools are available which use natural language processing techniques to help discover possible relationships between requirements.

Requirements change management

Requirements change management (Figure 16) should be applied to all proposed changes to a system's requirements after the requirements document has been approved. Change management is essential because you need to decide if the benefits of implementing new requirements are justified by the costs of implementation. The advantage of using a formal process for change management is that all change proposals are treated consistently and changes to the requirements document are made in a controlled way.

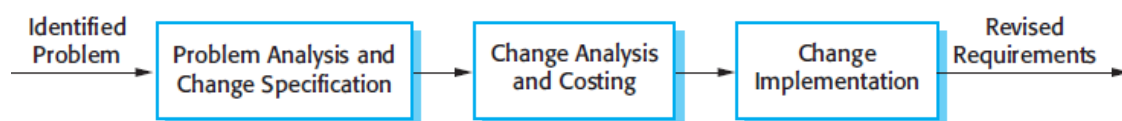


Figure 16: Requirements change management

There are three principal stages to a change management process:

- **Problem analysis and change specification:**The process starts with an identified requirements problem or, sometimes, with a specific change proposal. During this stage, the problem or the change proposal is analyzed to check that it is valid.
- **Change analysis and costing:**The effect of the proposed change is assessed using traceability information and general knowledge of the system requirements. The cost of making the change is estimated both in terms of modifications to the requirements document and, if appropriate, to the system design and implementation. Once this analysis is completed, a decision is made whether or not to proceed with the requirements change.
- **Change implementation:**The requirements document and, where necessary, the system design and implementation, are modified. You should organize the requirements document so that you can make changes to it without extensive rewriting or reorganization.

*****XXX*****

All the Best

Module 2: OBJECT-ORIENTED MODELING AND DESIGN

What is Object Oriented

Object OO means that we organize software as a collection of discrete objects (that incorporate both data structure and behavior).

There are four **aspects (characteristics)** required by an OO approach

- Identity.
- Classification.
- Inheritance.
- Polymorphism

Identity means that data is quantized into discrete, distinguishable entities called objects.

E.g. for objects: personal computer, bicycle, queen in chess etc.

Objects can be concrete (such as a file in a file system) or conceptual (such as scheduling policy in a multiprocessing OS).

Classification means that objects with the same data structure (attribute) and behavior (operations) are grouped into a class.

Each object is said to be an instance of its class.

Development refers to the software life cycle: Analysis, Design and Implementation. The essence of OO Development is the *identification* and *organization* of application concepts, rather than their final representation in a programming language.

OO methodology

Here it presents a process for OO development and a graphical notation for representing OO concepts.

System conception: Software development begins with business analysis or users conceiving an application and formulating tentative requirements.

Analysis: The analyst scrutinizes and rigorously restates the requirements from the system conception by constructing models.

Domain Model- a description of real world objects reflected within the system.

Application Model- a description of parts of the application system itself that are visible to the user.

Three models

Class Model—for the objects in the system & their relationships. It describes the static structure of the objects in the system and their relationships.

Class model contains class diagrams- a graph whose nodes are classes and arcs are relationships among the classes.

State model—for the life history of objects. It describes the aspects of an object that change over time.

It specifies and implements control with state diagrams-a graph whose nodes are states and whose arcs are transition between states caused by events.

OO THEMES

Abstraction:

Abstraction lets you focus on essential aspects of an application while ignoring details i.e focusing on what an object is and does, before deciding how to implement it.

Encapsulation (information hiding)

It separates the external aspects of an object (that are accessible to other objects) from the internal implementation details (that are hidden from other objects)

Sharing

OO techniques provide sharing at different levels.

Inheritance of both data structure and behaviour lets sub classes share common code.

OO development not only lets you share information within an application, but also offers the prospect of reusing designs and code on future projects.

Emphasis on the essence of an object

OO technology stresses what an object is, rather than how it is used.

OO development places a greater emphasis on data structure and a lesser emphasis on procedure structure than functional-decomposition methodologies.

Synergy

Identity, classification, polymorphism and inheritance characterize OO languages.

Evidence for usefulness of OO development

OO techniques used for developing compilers, graphics, user interfaces, databases, CAD systems, simulators, control systems and other applications.

Object Modelling Technique introduced in 1991.

The popularity of OO modelling led to new problem- alternative notations. The notations expressed similar ideas but had different symbols, confusing developers and making communication difficult.

In 1996 object management group (OMG) issued a request for proposal for a standard OO modelling notations.

Model: A model is an abstraction of something for the purpose of understanding it before building it.

A model omits non essential details, it is easier to manipulate than the original entity.

MODELLING

Designers build many kinds of models for various purposes before constructing things. Models serve several purposes

Class Modeling

CONTENTS:

1. Object and Class Concepts.
2. Link and Association Concepts
3. Generalization and Inheritance
4. A Sample Class Model
5. Navigation of Class Models
6. Summary

Class Modeling:

Captures the static structure of a system by characterizing the objects in the system, the relationships between the objects, and the attributes and operations for each class

1. Object and Class Concepts:

Object: is a concept, abstraction, or a thing with identity that has meaning for an application Eg. two apples) each have identity and are distinguishable.

Class: Describes a group of objects with the same properties(attributes), behavior (operations), kinds of relationship, and Semantics. (Eg: *Person, company, Process and Window*)

Class Diagram: Provide a Graphical notation for modeling classes and their relationships, thereby describing possible objects as shown in figure 1.

Object Diagram: Shows individual Objects and their relationships.

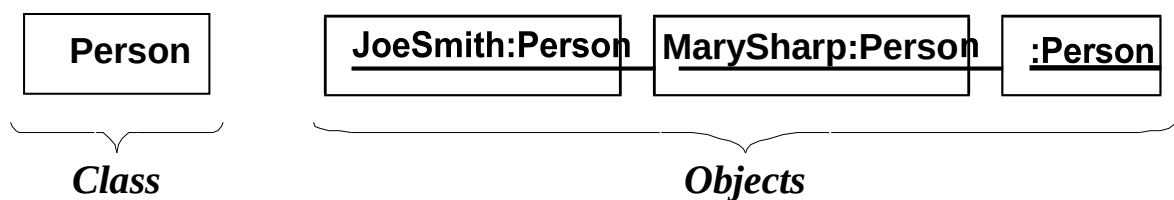


Figure 1: A Class and Objects: Objects and Classes are the focus of Class Modeling

Values: A Value is a piece of data. **Attributes:** named property of class. These elaborate classes as shown in figure 2.

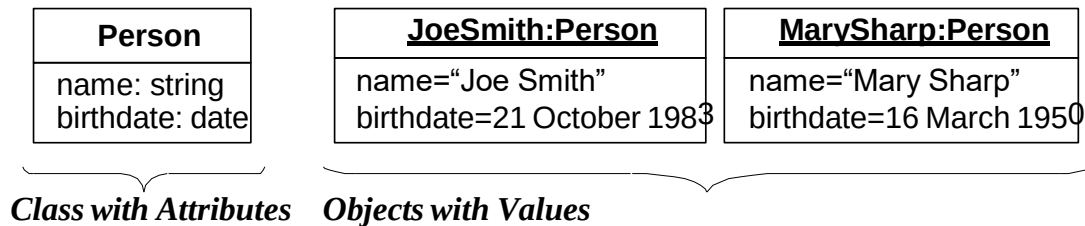


Figure 2: Attributes and Values.

Operations: is a function or procedure that may be applied to or by objects in a class as shown in figure 3.

Methods: is a implementation of an operation for a class.

Polymorphism: same operation applied to many different class.

Signature: all methods must have same signature ie *print* should not have *filename* as argument for one method and *filepointer* as argument for other.

Feature: generic word for either an attribute or operation.

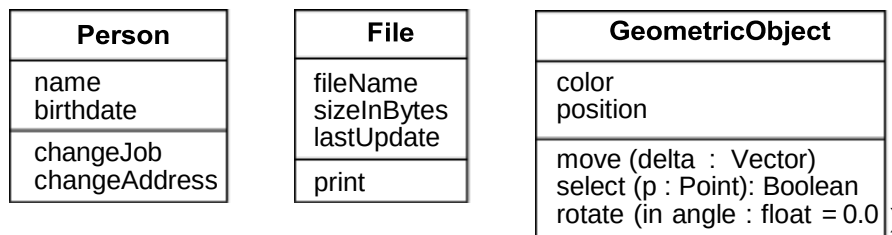


Figure 3 : Operations : function that may be applied to or by objects in a class.

Summary of notations for classes: Figure 4 summarizes the notations for classes.

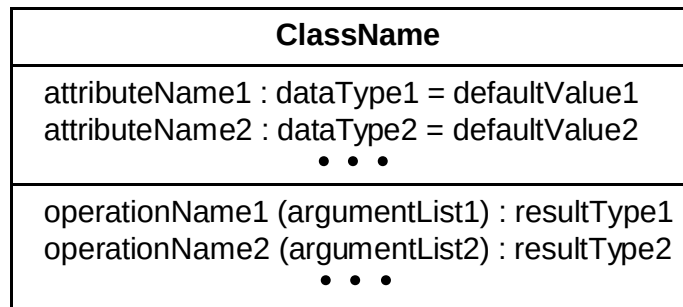


Figure 4: Summary of modeling notations for classes: A Box represents a class and may have as many as three components.

The compartments in the box contain, from top to bottom: class name, list of attributes, and list of operations. Optional details are type, default values, argument list and result type may follow each operation name.

2. Link and Association Concepts:

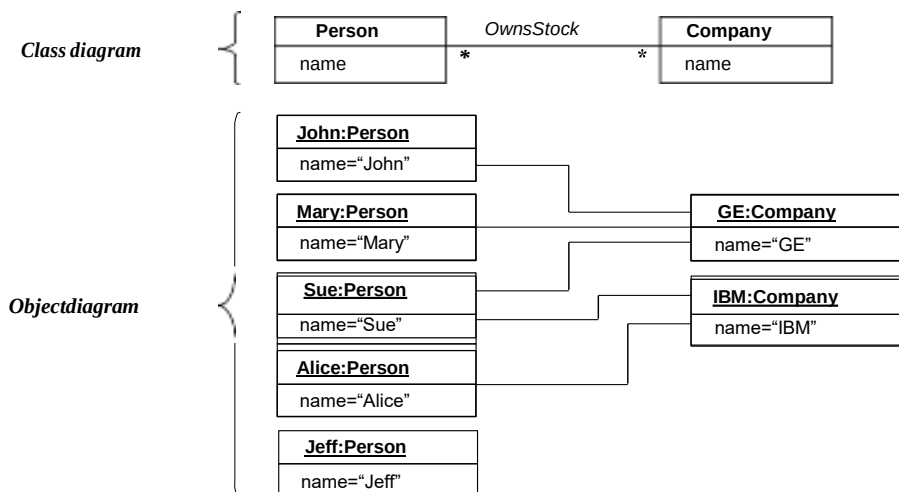


Figure 5: Many to Many Associations

Link: is a physical / conceptual connection among objects most links relate two objects, but some links relate 3 or more object. It is an instance of association as shown in figure 5 above.

Association: is a description of a group of links with common structure and common semantics as in the class diagram shown in figure 5.

Multiplicity: Specifies the number of instances of one class that may relate to single instance of other class. Similar to Cardinality ratio in DBMS as shown in the following figures 6,7,8.

UML specifies: “1”: exactly one,
 “1..*” (one or more)
 “3..5” (three to five inclusive)

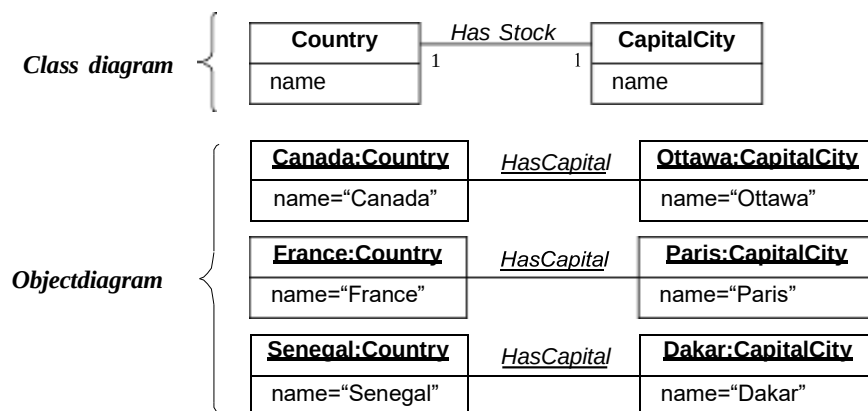


Figure 6: One-to-one Association

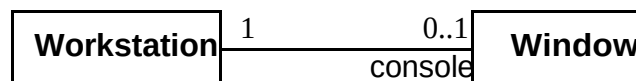


Figure 7: Zero-to-one multiplicity



Figure 8: Association Vs Link

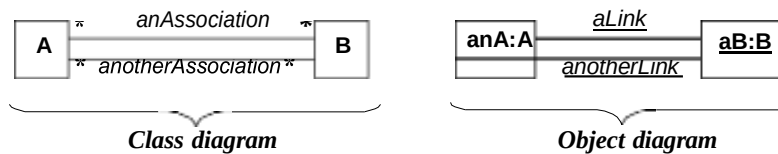


Figure: Multiple association to model multiple links

Figure 9 Association Vs Links. Multiple associations to model multiple links between the same objects

Association End names:

- Association End names is an important concept in UML.
- Association ends can not only be assigned multiplicity they can also be named. In a problem description they are generally identified by nouns

Association end: can give names to the association link.

Use of associations end name: to name links b/w object of same class.

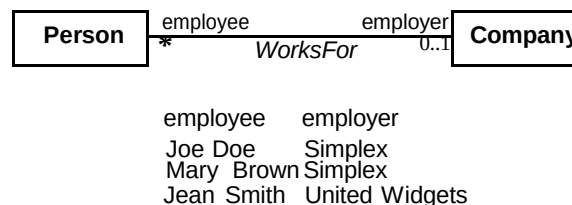


Figure 10: Association End names. Each end of an association can have a name.

Importance of association end names as in figure 11.

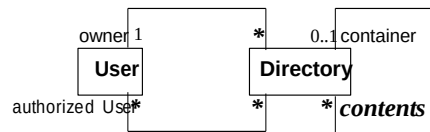


Figure 11: association end names are necessary for association between two objects of same class

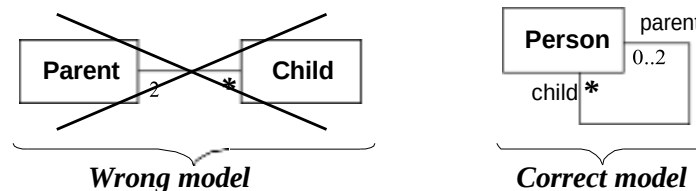


Figure 12: Association end names: Use association end names to model multiple references to the same class

Ordering: association which follows some priority eg. Windows opened on screen follow on explicit order only top most windows is visible on any part of screen
 Ordered collection of elements with duplicates not allowed .ref figure 13



Figure 13 Ordering an object for an association end. Ordering sometimes occurs for “many” multiplicity

Bags and sequences:

- Bag: collection of elements with duplicates allowed.
- Sequence: ordered collection of elements with duplicate allowed eg. An itinerary is a sequence of airports and the same airport can be visited more than once.
- Sequence is ordered bag both allow duplicates, {ordered} and {sequence} is same only difference is sequence allows duplicates as shown in figure 14.

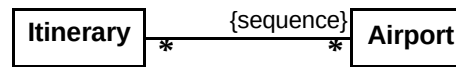


Figure 14: An example of a sequence. A itinerary may visit multiple airports so you should use { sequence} and not { ordered}

Association Classes:

- It is an association that is also a class.
- Like the links of an association, the instances of an association class derive identity from instances of the constituent classes.
- You can find association classes by looking for adverbs in a problem statement or by abstracting values. ref figure 15,16

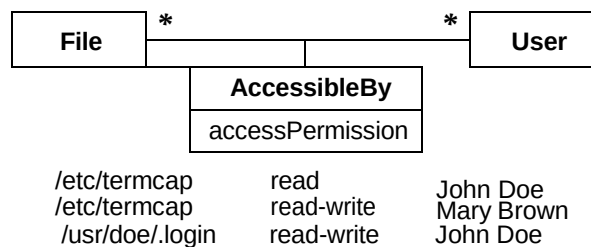


Figure 15: The links of an association can have attributes.

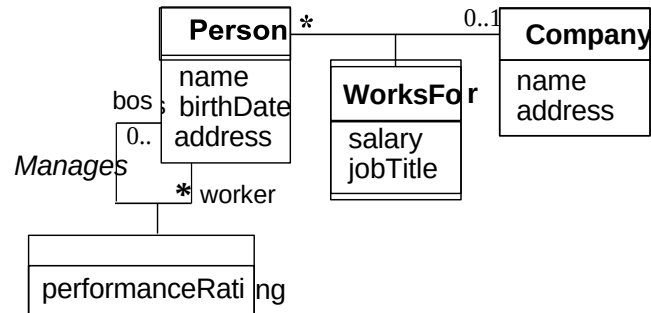


Figure 16: Association classes. Attributes may also occur for one-to-many and one-to-one associations

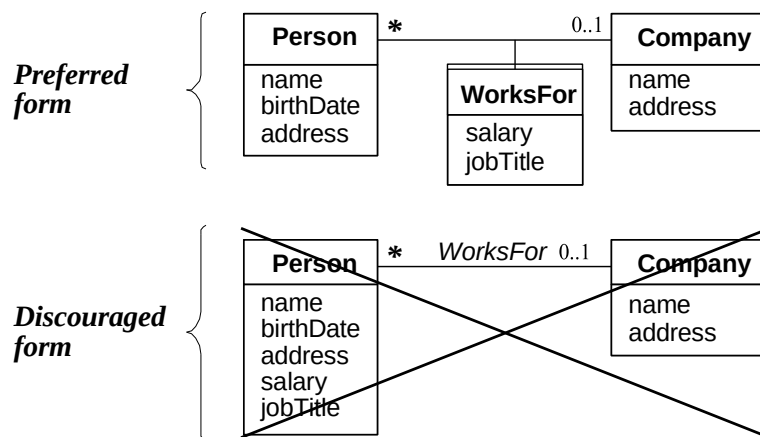


Figure 17: Proper use of association class: Do not fold attributes of an association into a class

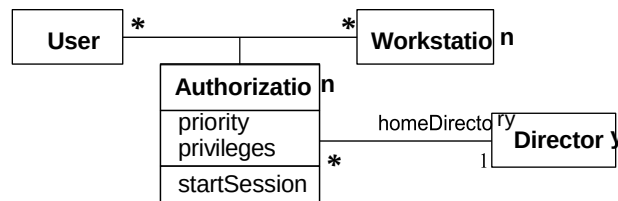


Figure 18: An association class participating in an association.

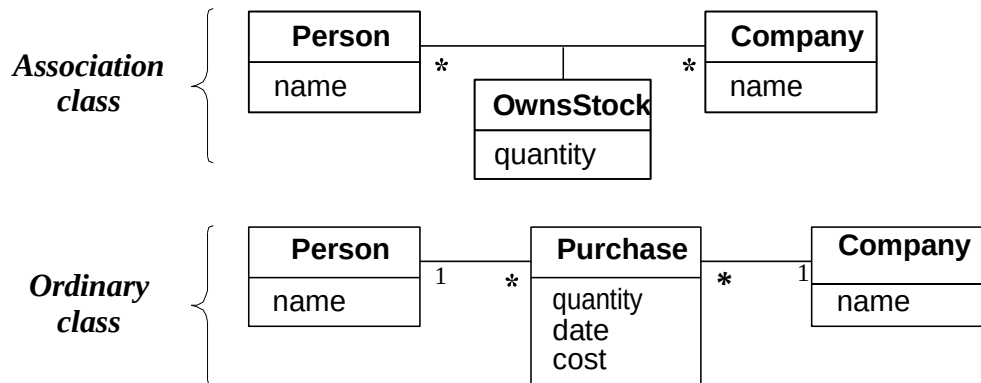


Figure 19: Association class Vs. Ordinary class An association class is much different from an ordinary class

Qualified Associations:

- An association in which an attribute called the qualifier disambiguates the objects for a “many” association end.
- Job of qualifier is to reduce the many association to one. Can only work for m-m and 1-m
- Also facilitates traversal of class models
- Stock exchange gives special unique symbol to each company
- Using qualified association we can make traversal easy.
- Ref Figure 20,21.

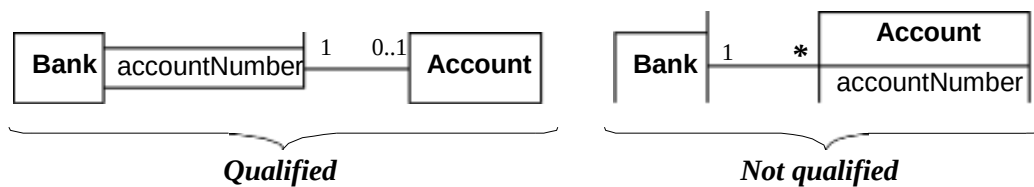
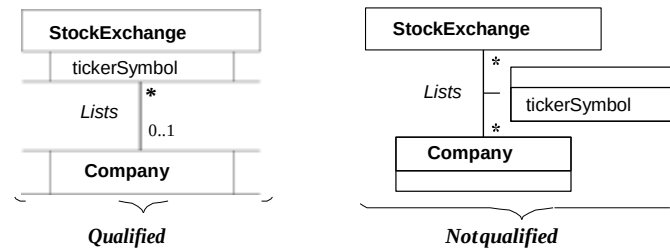


Figure 20: Qualified association. Qualification increases the precision of a model.



3.

Figure 21 : Qualified association. Qualification also facilitates traversal of class models.

3. Generalization and inheritance

- Generalization is the relationship between a class (the super class) and one or more variations of the class (sub class)
- Super class holds the common attributes, operations and associations. Subclass adds specific attributes
- Each subclass inherits features of super class Ancestor and descendents
- Refer figure 22.
- Figure 23 shows the inheritance definition for the case study of describing graphical figures.

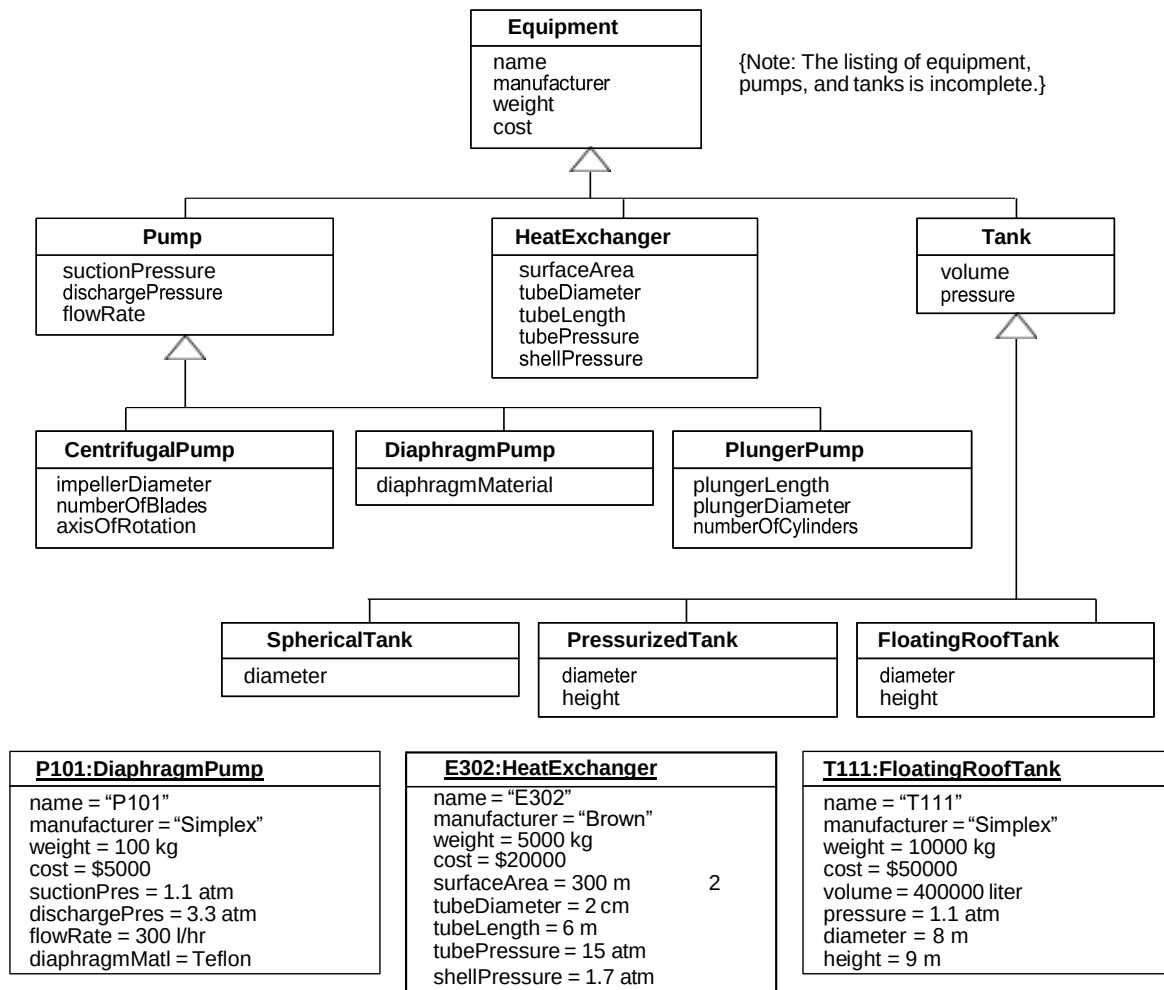


Figure 22: A multilevel Inheritance hierarchy with instances: Generalization organizes classes by their similarities and differences, structuring the description of objects.

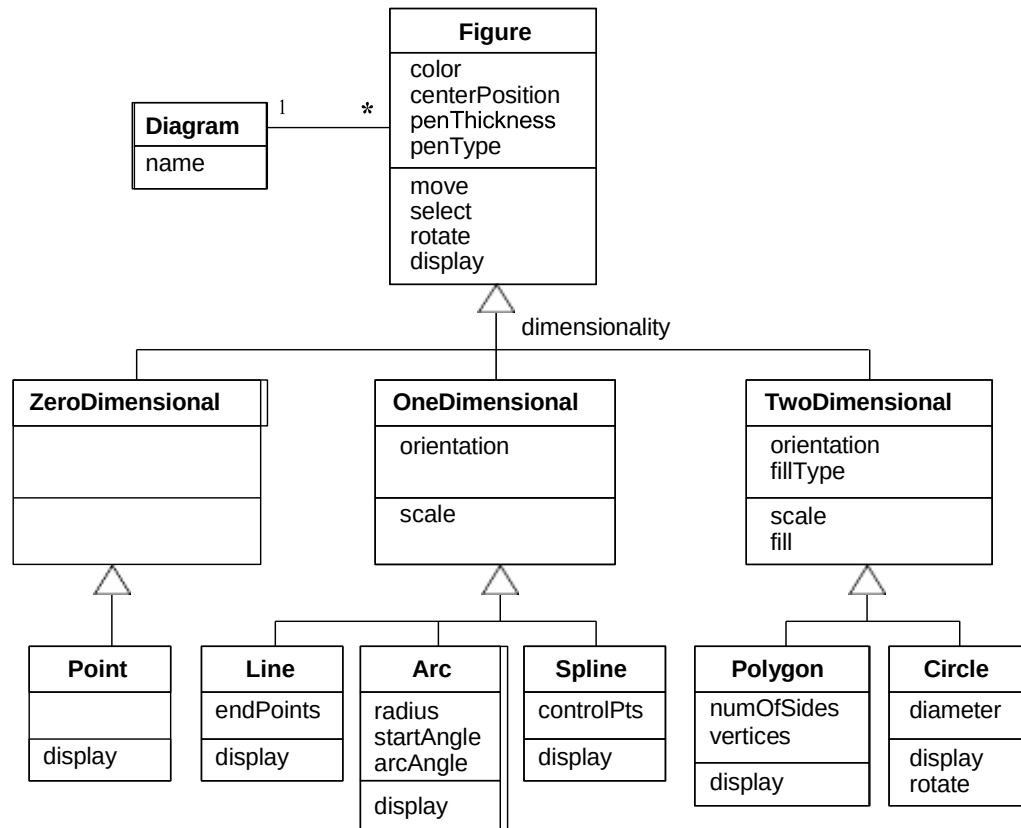


Figure 23: Inheritance for graphic figures. Each subclass inherits the attributes, operations, and associations of its super classes.

Use of Generalization:

- Serves three purposes:
- Support for polymorphism. (call at super class level automatically resolved)
- Second purpose is to structure the description of objects.(a taxonomy is formed)
- Third purpose is to enable reuse of code.

The terms generalization, specialization and inheritance all refer to aspects of the same idea.

- Generalization: derives from the fact that the superclass generalizes the subclasses.
- Specialization: refers to the fact that the subclasses refine or specialize the Superclass.
- Inheritance: is the mechanism for sharing attributes, operations, and associations via generalization / specialization relationship.

Overriding Features:

- A subclass may override a super class feature by defining a feature with the same name.
- We can override methods and default values of attributes.
- Never override the signature, or form, of a feature.

4. A Sample Class Model: figure 24

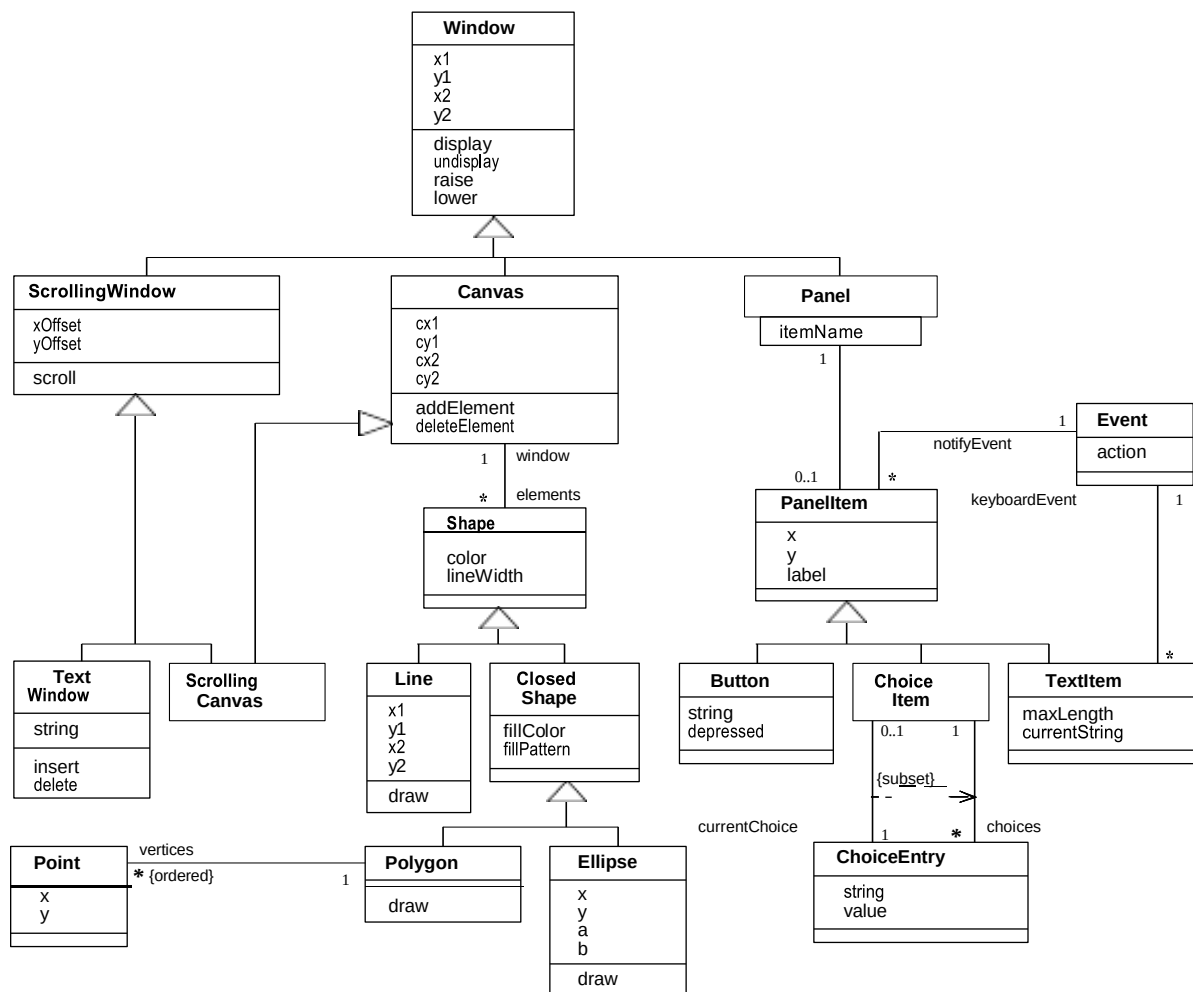


Figure 24. Class model of a windowing system.

The figure 24 shows a class model of a workstation window management system. This model is greatly simplified – a real model would require a number of pages- but it illustrates many class modeling constructs and shows how they fit together.

This simple mode gives a flavor of the use of class modeling.

5. Navigation of class model

Until now we have seen how class model can express the structure of an application. Now we will in this section see how they can also express the behavior of navigating among classes. Navigation is important because it lets you express the behavior of navigating among classes. Navigation is important because it lets you exercise a model and uncover hidden flaws and omissions so that you can repair them. You can perform navigation manually (an informal technique) or write navigation expressions.

Consider the simple model for credit card accounts: we can pose a variety of questions against the model.

- How many credit card account customer has?
- What transaction occurred in a time interval for a customer?
- The UML incorporates a language that can express these kinds of questions- the Object Constraint Language (OCL)

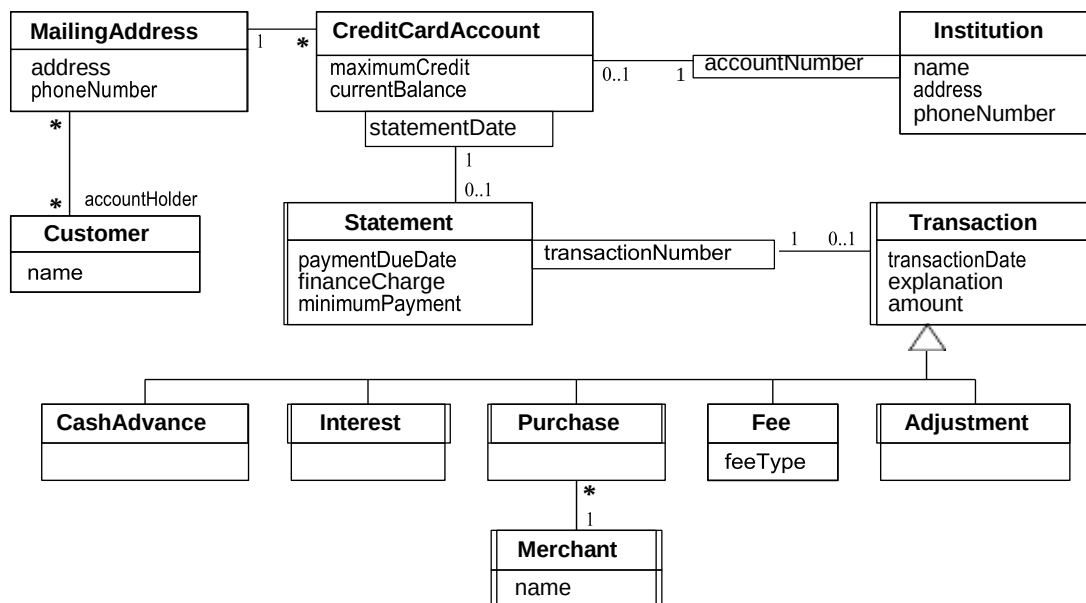


Figure 24: Class model for managing credit card accounts.

Object constraint language :

- Used to answer question pertaining to a class model
- Attribute syntax *aCreditCardAccount.maximumCredit* takes a *CreditcardAccount* object and finds the value of *maximumCredit*
- Operations : The syntax for a collection operation is the source object collection, followed by *by ->* and then the operation.
- Simple association: A third use of the dot notation is to traverse an association to a target end. The target end may be indicated by an association end name or, where there is no ambiguity, a class name. In the example *aCustomer.MailingAddress* yields a set of addresses for a customer.
- Qualified association. A qualifier lets you make a more precise traversal.
- Association classes: Given a link of an association class, you can find the constituent objects. Alternatively, given a constituent object, you can find the multiple links of an association class.
- Generalizations: Traversal of a generalization hierarchy is implicit for the OCL notation.
- Filters: There is often a need to filter the objects in a set. The OCL has several kinds of filters, the most common of which is the *select* operation.

Examples of OCL Expressions:

What transactions occurred for a credit card account within a time interval ?

```
aCreditCardAccount.Statement.Transaction-->
select (aStartDate <= transactionDate and
transactiondate <= anEndDate )
```

The above expression traverses from a *CreditCardAccount* object to *Statement* and then to *Transaction*, resulting in a set of transactions. (Traversal of the two associations results in a set, rather than a bag, because both associations are one-to-many.) Then we use the OCL *select* operator (a collection operator) to find the transactions within the time interval bounded by *aStartDate* and *anEndDate*.

Summary:

- Object and Class Concepts.
- Link and Association Concepts
- Generalization and Inheritance
- A Sample Class Model
- Navigation of Class Models

Exercise:

1. Prepare an Object diagram for an imaginary round tripe you took last weekend to London. Include at least one instance of each class. Fortunately, direct flights on a hyper sonic plane were available. A friend went with you but decided to stay a while and is still there. Captain Johnson was your pilot on both flights. You had a different seat each way, but noticed it was on the same plane because of a distinctive dent in the tail section. Students should indicate unknown values with a “?”.

Answer:

Figure A3.10 shows an object diagram that corresponds to the exercise statement. Note that most attribute values are left unspecified by the problem statement, yet the object diagram is still unambiguous. All objects are clearly identified by their attribute values and links to other objects. The exercise states that “you took a round trip between cities last weekend”; we make the assumption that this is also a round trip between airports. The two seats connected by the dotted line may be the same object.

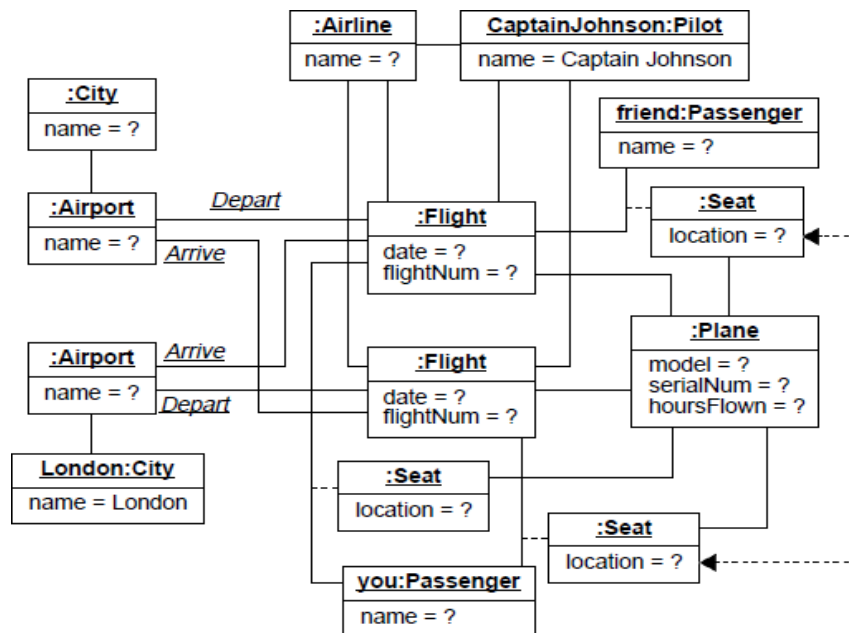


Figure A3.10 Object diagram for an air transportation system

2. Prepare a class diagram for the dining philosopher problem. There are 5 philosophers and 5 forks around a circular table. Each philosopher has access to 2 forks, one on either side. Each fork is shared by 2 philosophers. Each fork may be either on the table or in use by one philosopher. A philosopher must have 2 forks to eat.

Answer:

Figure A3.34 shows a class diagram for the dining philosopher's problem. The one-to-one associations describe the relative locations of philosophers and forks. The *InUse* association describes who is using forks. Other representations are possible, depending on your viewpoint. An object diagram may help you better understand this problem

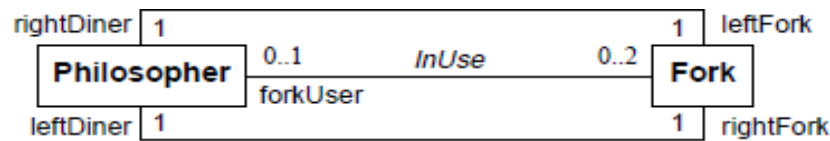


Figure A3.34 Class diagram for the dining philosopher problem

MODULE 3

CONTENTS

- 2.1 Introduction
- 2.2 Objective
- 2.3 System Modeling
- 2.4 Context Models
- 2.5 Interaction models
- 2.6 Structural models
- 2.7 Behavioral models
- 2.8 Model-driven engineering
- 2.9 The Rational Unified Process
- 2.10 Software Design and Implantation
- 2.11 Object-oriented design using the UML
- 2.12 Design patterns
- 2.13 Implementation issues
- 2.14 Open source development
- 2.15 Assignment questions
- 2.16 Outcome
- 2.17 Further Reading

2.1 Introduction

The different system models are discussed in detail. The context model, structural model, interaction models and behavioural model are explained in detail with required diagrams. An overview of object oriented design using the UML is given. Different implementation issues are discussed. Idea of open source development is highlighted.

2.2 Objective

- Understand why different types of model are required and the fundamental system modeling perspectives of context, interaction, structure, and behaviour
- Have been introduced to some of the diagram types in the Unified Modeling Language (UML) and how these diagrams may be used in system modelling
- Understand how the Rational Unified Process integrates good software engineering practice to create adaptable software processes

2.3 System Modeling

System modeling is the process of developing abstract models of a system, with each model presenting a different view or perspective of that system. System modelling has generally come to mean representing the system using some kind of graphical notation, which is now almost always based on notations in the Unified Modeling Language (UML).

You may develop models of both the existing system and the system to be developed:

- Models of the existing system are used during requirements engineering. They help clarify what the existing system does and can be used as a basis for discussing its strengths and weaknesses. These then lead to requirements for the new system.
- Models of the new system are used during requirements engineering to help explain the proposed requirements to other system stakeholders. Engineers use these models to discuss design proposals and to document the system for implementation.

The most important aspect of a system model is that it leaves out detail. A model is an abstraction of the system being studied rather than an alternative representation of that system. Ideally, a representation of a system should maintain all the information about the entity being represented. An abstraction deliberately simplifies and picks out the most salient characteristics.

You may develop different models to represent the system from different perspectives. For example:

- **An external perspective**, where you model the context or environment of the system.
- **An interaction perspective**, where you model the interactions between a system and its environment or between the components of a system.
- **A structural perspective**, where you model the organization of a system or the structure of the data that is processed by the system.
- **A behavioral perspective**, where you model the dynamic behavior of the system and how it responds to events.

However, a survey in 2007 (Erickson and Siau, 2007) showed that most users of the UML thought that five diagram types could represent the essentials of a system:

- **Activity diagrams**, which show the activities involved in a process or in data processing.
- **Use case diagrams**, which show the interactions between a system and its environment.
- **Sequence diagrams**, which show interactions between actors and the system and between system components.
- **Class diagrams**, which show the object classes in the system and the associations between these classes.
- **State diagrams**, which show how the system reacts to internal and external events.

When developing system models, you can often be flexible in the way that the graphical notation is used. You do not always need to stick rigidly to the details of a notation. The detail and rigor of a model depends on how you intend to use it. There are three ways in which graphical models are commonly used:

- As a means of facilitating discussion about an existing or proposed system.
- As a way of documenting an existing system.
- As a detailed system description that can be used to generate a system implementation.

2.4 Context Models

At an early stage in the specification of a system, you should decide on the system boundaries. This involves working with system stakeholders to decide what functionality should be included in the system and what is provided by the system's environment. You should look at possible overlaps in functionality with existing systems and decide where new functionality should be implemented. These decisions should be made early in the process to limit the system costs and the time needed for understanding the system requirements and design.

In some cases, the boundary between a system and its environment is relatively clear. In other cases, there is more flexibility, and you decide what constitutes the boundary between the system and its environment during the requirements engineering process.

Once some decisions on the boundaries of the system have been made, part of the analysis activity is the definition of that context and the dependencies that a system has on its environment. Normally, producing a simple architectural model is the first step in this activity.

Figure 1 is a simple context model that shows the patient information system and the other systems in its environment. From Figure 1, you can see that the MHC-PMS is connected to an appointments system and a more general patient record system with which it shares data. The system is also connected to systems for management reporting and hospital bed allocation and a statistics system that collects information for research. Finally, it makes use of a prescription system to generate prescriptions for patients' medication.

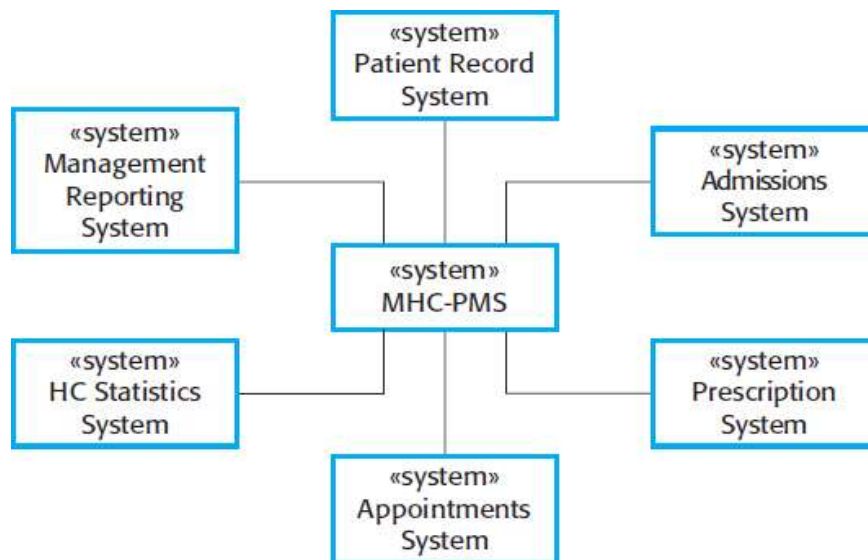


Figure 1: The context of the MHC-PMS

Context models normally show that the environment includes several other automated systems. However, they do not show the types of relationships between the systems in the environment and the system that is being specified. Therefore, simple context models are used along with other models, such as business process models. These describe human and automated processes in which particular software systems are used.

Figure 2 is a model of an important system process that shows the processes in which the MHC-PMS is used.

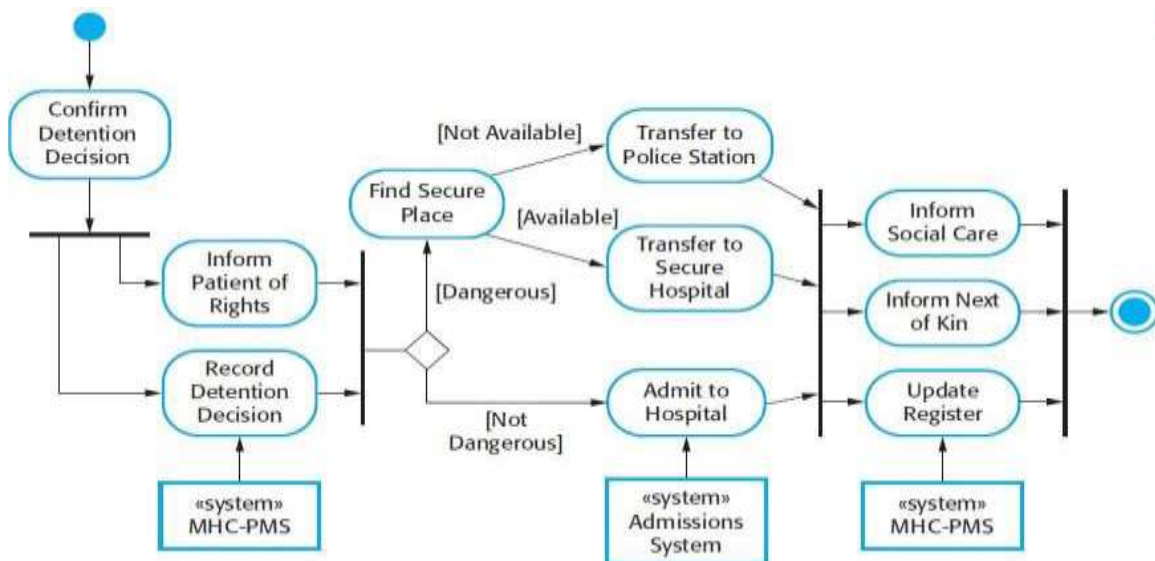


Figure 2: Process model of involuntary detention

Figure 2 is a UML activity diagram. Activity diagrams are intended to show the activities that make up a system process and the flow of control from one activity to another. The start of a process is indicated by a filled circle; the end by a filled circle inside another circle. Rectangles with round corners represent activities, that is, the specific sub-processes that must be carried out. You may include objects in activity charts. In Figure 2, I have shown the systems that are used to support different processes. I have indicated that these are separate systems using the UML stereotype feature.

In a UML activity diagram, arrows represent the flow of work from one activity to another. A solid bar is used to indicate activity coordination. When the flow from more than one activity leads to a solid bar then all of these activities must be complete before progress is possible. When the flow from a solid bar leads to a number of activities, these may be executed in parallel. Therefore, in Figure 2, the activities to inform social care and the patient's next of kin, and to update the detention register may be concurrent.

Arrows may be annotated with guards that indicate the condition when that flow is taken. In Figure 2, you can see guards showing the flows for patients who are dangerous and not dangerous to society. Patients who are dangerous to society must be detained in a secure facility. However, patients who are suicidal and so are a danger to themselves may be detained in an appropriate ward in a hospital.

2.5 Interaction models

All systems involve interaction of some kind. This can be user interaction, which involves user inputs and outputs, interaction between the system being developed and other systems or interaction between the components of the system. Modeling user interaction is important as it helps to identify user requirements. Modeling system to system interaction highlights the communication problems that may arise. Modeling component interaction helps us understand if a proposed system structure is likely to deliver the required system performance and dependability.

Two related approaches to interaction modelling are:

- **Use case modeling**, which is mostly used to model interactions between a system and external actors (users or other systems).
- **Sequence diagrams**, which are used to model interactions between system components, although external agents may also be included.

Use case modeling

A use case can be taken as a simple scenario that describes what a user expects from a system. Each use case represents a discrete task that involves external interaction with a system. In its simplest form, a use case is shown as an ellipse with the actors involved in the use case represented as stick figures. Figure 3 shows a use case from the MHC-PMS that represents the task of uploading data from the MHC-PMS to a more general patient record system. This more general system maintains summary data about a patient rather than the data about each consultation, which is recorded in the MHC-PMS.



Figure 3: Transferdata use case

Notice that there are two actors in this use case: the operator who is transferring the data and the patient record system. The stick figure notation was originally developed to cover human interaction but it is

also now used to represent other external systems and hardware. Formally, use case diagrams should use lines without arrows as arrows in the UML indicate the direction of flow of messages. Obviously, in a use case messages pass in both directions. However, the arrows in Figure 3 are used informally to indicate that the medical receptionist initiates the transaction and data is transferred to the patient record system.

Use case diagrams give a fairly simple overview of an interaction so you have to provide more detail to understand what is involved. This detail can either be a simple textual description, a structured description in a table, or a sequence diagram as discussed below. You chose the most appropriate format depending on the use case and the level of detail that you think is required in the model. Figure 4 shows a tabular description of the 'Transfer data' use case.

MHC-PMS: Transfer data	
Actors	Medical receptionist, patient records system (PRS)
Description	A receptionist may transfer data from the MHC-PMS to a general patient record database that is maintained by a health authority. The information transferred may either be updated personal information (address, phone number, etc.) or a summary of the patient's diagnosis and treatment.
Data	Patient's personal information, treatment summary
Stimulus	User command issued by medical receptionist
Response	Confirmation that PRS has been updated
Comments	The receptionist must have appropriate security permissions to access the patient information and the PRS.

Figure 4: Tabular description of the 'Transfer data' use case

Composite use case diagrams show a number of different use cases. Sometimes, it is possible to include all possible interactions with a system in a single composite use case diagram. However, this may be impossible because of the number of use cases. In such cases, you may develop several diagrams, each of which shows related use cases. For example, Figure 5 shows all of the use cases in the MHC-PMS in which the actor 'Medical Receptionist' is involved.

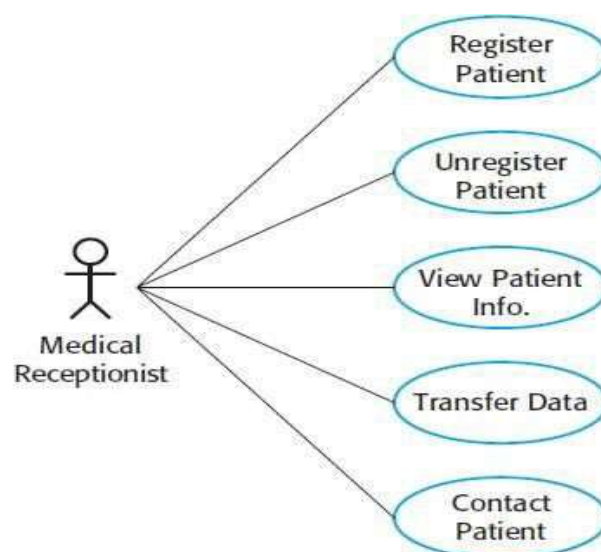


Figure 5: Use cases involving the role 'medical receptionist'

Sequence diagrams

Sequence diagrams in the UML are primarily used to model the interactions between the actors and the objects in a system and the interactions between the objects themselves. As the name implies, a sequence diagram shows the sequence of interactions that take place during a particular use case or use case instance. Figure 6 is an example of a sequence diagram that illustrates the basics of the notation. This diagram models the interactions involved in the View patient information use case, where a medical receptionist can see some patient information.

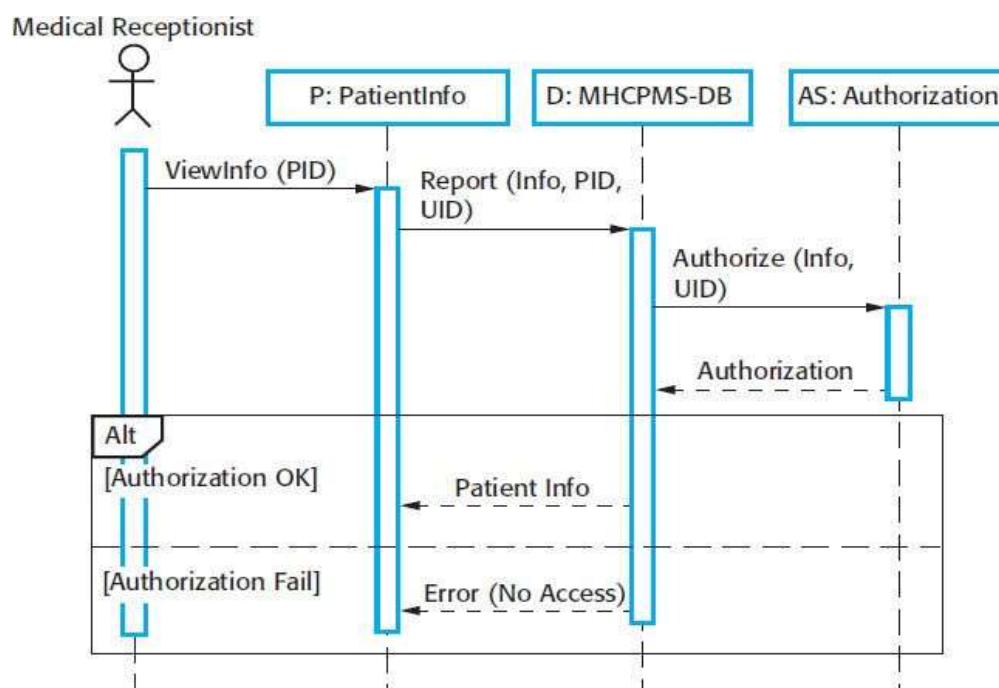


Figure 6: Sequence diagram for View patient information

The objects and actors involved are listed along the top of the diagram, with a dotted line drawn vertically from these. Interactions between objects are indicated by annotated arrows. The rectangle on the dotted lines indicates the lifeline of the object concerned (i.e., the time that object instance is involved in the computation). You read the sequence of interactions from top to bottom. The annotations on the arrows indicate the calls to the objects, their parameters, and the return values. In this example, I also show the notation used to denote alternatives. A box named alt is used with the conditions indicated in square brackets.

You can read Figure 6 as follows:

- The medical receptionist triggers the ViewInfo method in an instance P of the PatientInfo object class, supplying the patient's identifier, PID. P is a user interface object, which is displayed as a form showing patient information.
- The instance P calls the database to return the information required, supplying the receptionist's identifier to allow security checking (at this stage, we do not care where this UID comes from).
- The database checks with an authorization system that the user is authorized for this action.
- If authorized, the patient information is returned and a form on the user's screen is filled in. If authorization fails, then an error message is returned.

2.6 Structural models

Structural models of software display the organization of a system in terms of the components that make up that system and their relationships. Structural models may be static models, which show the structure of the system design or dynamic models, which show the organization of the system when it is executing. You create structural models of a system when you are discussing and designing the system architecture.

Class diagrams

Class diagrams are used when developing an object-oriented system model to show the classes in a system and the associations between these classes. Loosely, an object class can be thought of as a general definition of one kind of system object. An association is a link between classes that indicates that there is a relationship between these classes. Consequently, each class may have to have some knowledge of its associated class.

Class diagrams in the UML can be expressed at different levels of detail. When you are developing a model, the first stage is usually to look at the world, identify the essential objects, and represent these as classes. The simplest way of writing these is to write the class name in a box. You can also simply note the existence of an association by drawing a line between classes. For example, Figure 7 is a simple class diagram showing two classes: Patient and Patient Record with an association between them.

In Figure 7, I illustrate a further feature of class diagrams—the ability to show how many objects are involved in the association. In this example, each end of the association is annotated with a 1, meaning that there is a 1:1 relationship between objects of these classes. That is, each patient has exactly one record and each record maintains information about exactly one patient. As you can see from later examples, other multiplicities are possible. You can define that an exact number of objects are involved or, by using a *, as shown in Figure 8, that there are an indefinite number of objects involved in the association.



Figure 7: UML classes and association

Figure 8 develops this type of class diagram to show that objects of class Patient are also involved in relationships with a number of other classes. In this example, I show that you can name associations to give the reader an indication of the type of relationship that exists. The UML also allows the role of the objects participating in the association to be specified.

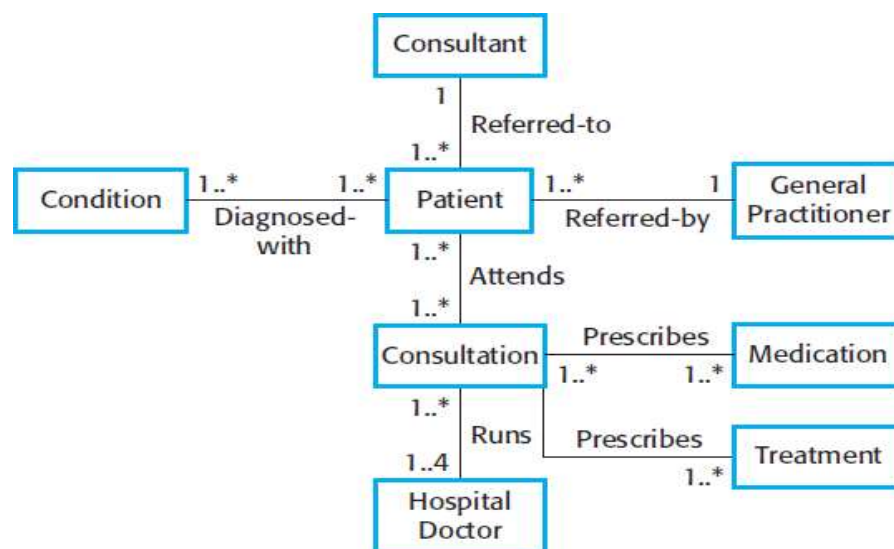


Figure 8: Classes and associations in the MHC-PMS

When showing the associations between classes, it is convenient to represent these classes in the simplest possible way. To define them in more detail, you add information about their attributes (the characteristics of an object) and operations (the things that you can request from an object).

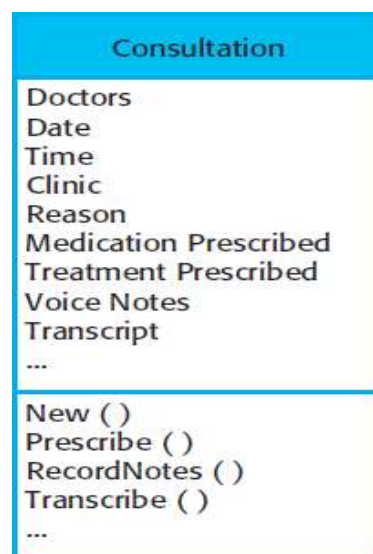


Figure 9: The consultation class

In the UML, you show attributes and operations by extending the simple rectangle that represents a class. This is illustrated in Figure 9 where:

- The name of the object class is in the top section.
- The class attributes are in the middle section. This must include the attribute names and, optionally, their types.
- The operations (called methods in Java and other OO programming languages) associated with the object class are in the lower section of the rectangle.

Generalization

Generalization is an everyday technique that we use to manage complexity. Rather than learn the detailed characteristics of every entity that we experience, we place these entities in more general classes (animals, cars, houses, etc.) and learn the characteristics of these classes. This allows us to infer that different members of these classes have some common characteristics (e.g., squirrels and rats are rodents). We can make general statements that apply to all class members (e.g., all rodents have teeth for gnawing).

In modeling systems, it is often useful to examine the classes in a system to see if there is scope for generalization. This means that common information will be maintained in one place only. This is good design practice as it means that, if changes are proposed, then you do not have to look at all classes in the system to see if they are affected by the change. In object-oriented languages, such as Java, generalization is implemented using the class inheritance mechanisms built into the language.

The UML has a specific type of association to denote generalization, as illustrated in Figure 10. The generalization is shown as an arrowhead pointing up to the more general class. This shows that general practitioners and hospital doctors can be generalized as doctors and that there are three types of Hospital Doctor—those that have just graduated from medical school and have to be supervised (Trainee Doctor); those that can work unsupervised as part of a consultant's team (Registered Doctor); and consultants, who are senior doctors with full decision-making responsibilities.

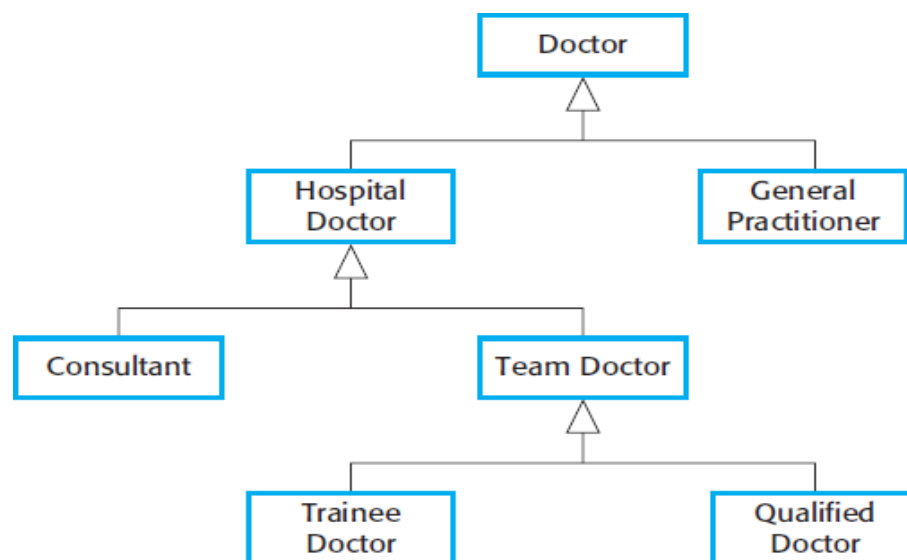


Figure 10: A generalization hierarchy

In a generalization, the attributes and operations associated with higher-level classes are also associated with the lower-level classes. In essence, the lower-level classes are subclasses that inherit the attributes and operations from their superclasses. These lower-level classes then add more specific attributes and operations.

Figure 11, which shows part of the generalization hierarchy that I have extended with class attributes. The operations associated with the class Doctor are intended to register and de-register that doctor with the MHC-PMS.

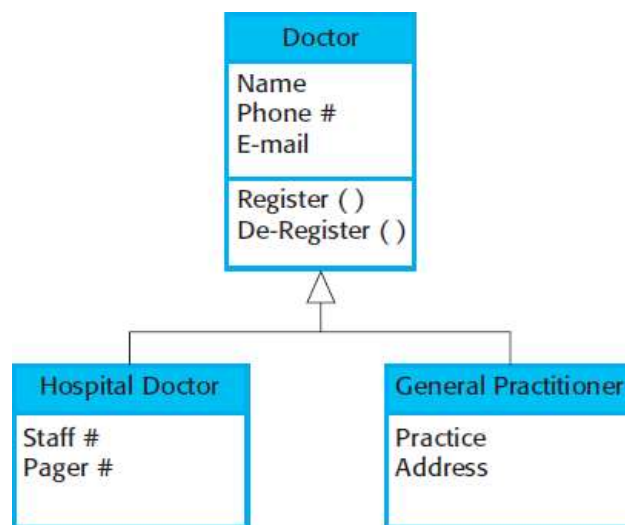


Figure 11: A generalization hierarchy with added detail

Aggregation

Objects in the real world are often composed of different parts. The UML provides a special type of association between classes called aggregation that means that one object (the whole) is composed of other objects (the parts). To show this, we use a diamond shape next to the class that represents the whole. This is shown in Figure 12, which shows that a patient record is a composition of Patient and an indefinite number of Consultations.

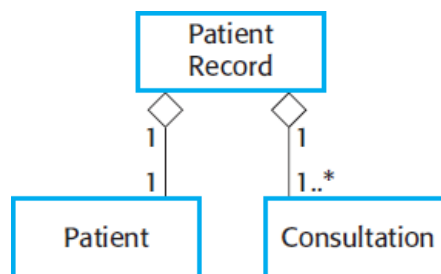


Figure 12: The aggregation association

2.7 Behavioral models

Behavioral models are models of the dynamic behavior of the system as it is executing. They show what happens or what is supposed to happen when a system responds to a stimulus from its environment. You can think of these stimuli as being of two types:

- **Data** Some data arrives that has to be processed by the system.
- **Events** Some event happens that triggers system processing. Events may have associated data but this is not always the case.

Data-driven modeling

Data-driven models show the sequence of actions involved in processing input data and generating an associated output. They are particularly useful during the analysis of requirements as they can be used to show end-to-end processing in a system. That is, they show the entire sequence of actions that take place from an input being processed to the corresponding output, which is the system's response.

In the 1970s, structured methods such as DeMarco's Structured Analysis (DeMarco, 1978) introduced data-flow diagrams (DFDs) as a way of illustrating the processing steps in a system. The UML does not support data-flow diagrams as they were originally proposed and used for modeling data processing. The reason for this is that DFDs focus on system functions and do not recognize system objects. However, because data-driven systems are so common in business, UML 2.0 introduced activity diagrams, which are similar to data-flow diagrams. For example, Figure 13 shows the chain of processing involved in the insulin pump software. In this diagram, you can see the processing steps (represented as activities) and the data flowing between these steps (represented as objects).

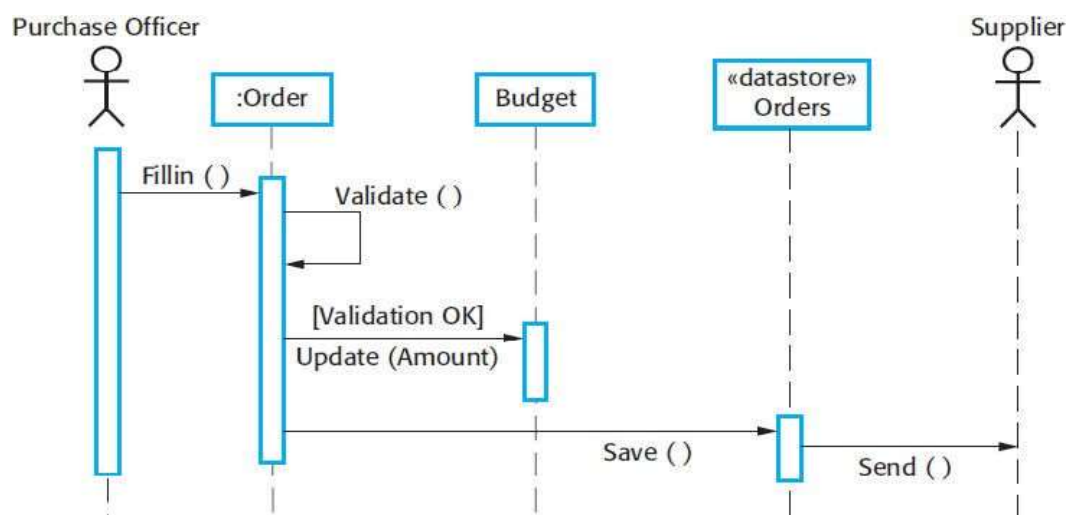


Figure 13: An activitymodel of the insulinpump's operation

An alternative way of showing the sequence of processing in a system is to use UML sequence diagrams. You have seen how these can be used to model interaction but, if you draw these so that messages are only sent from left to right, then they show the sequential data processing in the system. Figure 14 illustrates this, using a sequence model of the processing of an order and sending it to a supplier. Sequence models highlight objects in a system, whereas data-flow diagrams highlight the functions.

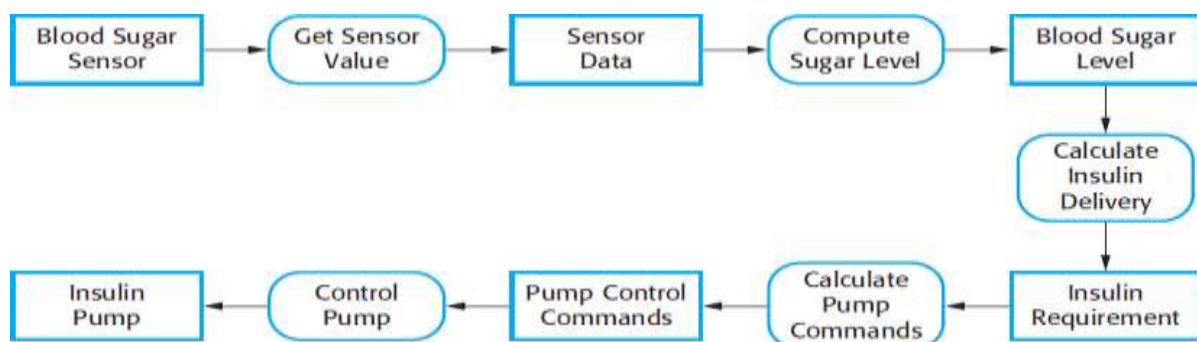


Figure 14: Order processing

Event-driven modelling

Event-driven modeling shows how a system responds to external and internal events. It is based on the assumption that a system has a finite number of states and that events (stimuli) may cause a transition from one state to another. The UML supports event-based modeling using state diagrams. State diagrams show system states and events that cause transitions from one state to another. They do not show the flow of data within the system but may include additional information on the computations carried out in each state.

In UML state diagrams, rounded rectangles represent system states. They may include a brief description (following 'do') of the actions taken in that state. The labeled arrows represent stimuli that force a transition from one state to another. You can indicate start and end states using filled circles, as in activity diagrams.

From Figure 15, you can see that the system starts in a waiting state and responds initially to either the full-power or the half-power button. Users can change their mind after selecting one of these and press the other button. The time is set and, if the door is closed, the Start button is enabled. Pushing this button starts the oven operation and cooking takes place for the specified time. This is the end of the cooking cycle and the system returns to the waiting state.

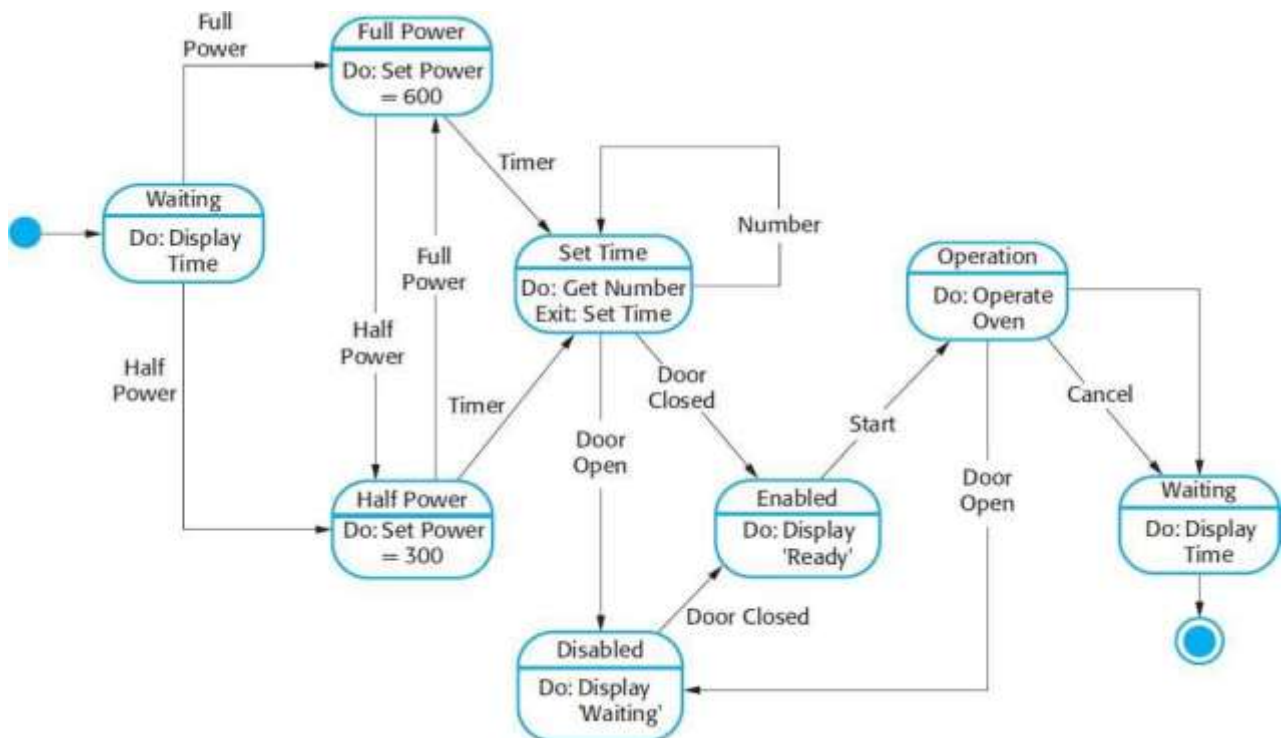


Figure 15: State diagram of a microwave oven

The problem with state-based modeling is that the number of possible states increases rapidly. For large system models, therefore, you need to hide detail in the models. One way to do this is by using the notion of a superstate that encapsulates a number of separate states. This superstate looks like a single state on a high-level model but is then expanded to show more detail on a separate diagram. To illustrate this concept, consider the Operation state in Figure 15. This is a superstate that can be expanded, as illustrated in Figure 16.

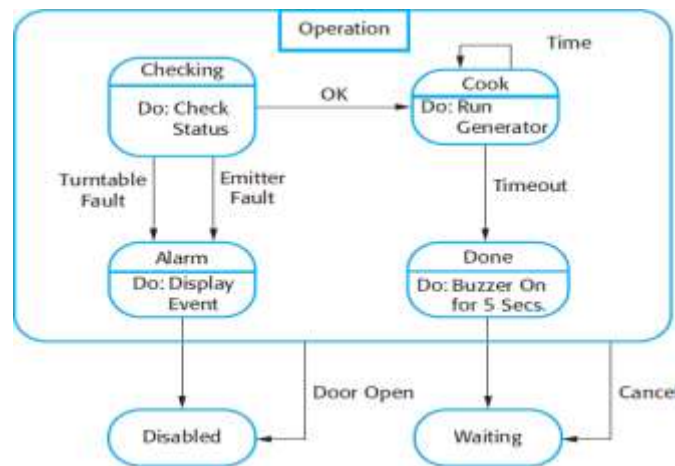


Figure 16: Microwave oven operation

2.8 Model-driven engineering

Model-driven engineering (MDE) is an approach to software development where models rather than programs are the principal outputs of the development process. The programs that execute on a hardware/software platform are then generated automatically from the models. Proponents of MDE argue that this raises the level of abstraction in software engineering so that engineers no longer have to be concerned with programming language details or the specifics of execution platforms.

Model-driven engineering and model-driven architecture are often seen as the same thing. However, I think that MDE has a wider scope than MDA. MDA focuses on the design and implementation stages of software development whereas MDE is concerned with all aspects of the software engineering process.

The main arguments for and against MDE are:

- **For MDE:** Model-based engineering allows engineers to think about systems at a high level of abstraction, without concern for the details of their implementation. This reduces the likelihood of errors, speeds up the design and implementation process, and allows for the creation of reusable, platform-independent application models. By using powerful tools, system implementations can be generated for different platforms from the same model. Therefore, to adapt the system to some new platform technology, it is only necessary to write a translator for that platform. When this is available, all platform-independent models can be rapidly rehosted on the new platform.
- **Against MDE:** As I discussed earlier in this chapter, models are a good way of facilitating discussions about a software design. However, it does not always follow that the abstractions that are supported by the model are the right abstractions for implementation. So, you may create informal design models but then go on to implement the system using an off-the-shelf, configurable package.

Model-driven architecture

Model-driven architecture is a model-focused approach to software design and implementation that uses a sub-set of UML models to describe a system. Here, models at different levels of abstraction are created.

The MDA method recommends that three types of abstract system model should be produced:

- **A computation independent model (CIM)** that models the important domain abstractions used in the system. CIMs are sometimes called domain models. You may develop several different CIMs, reflecting different views of the system.

- **A platform independent model (PIM)** that models the operation of the system without reference to its implementation.
- **Platform specific models (PSM)** which are transformations of the platform independent model with a separate PSM for each application platform.

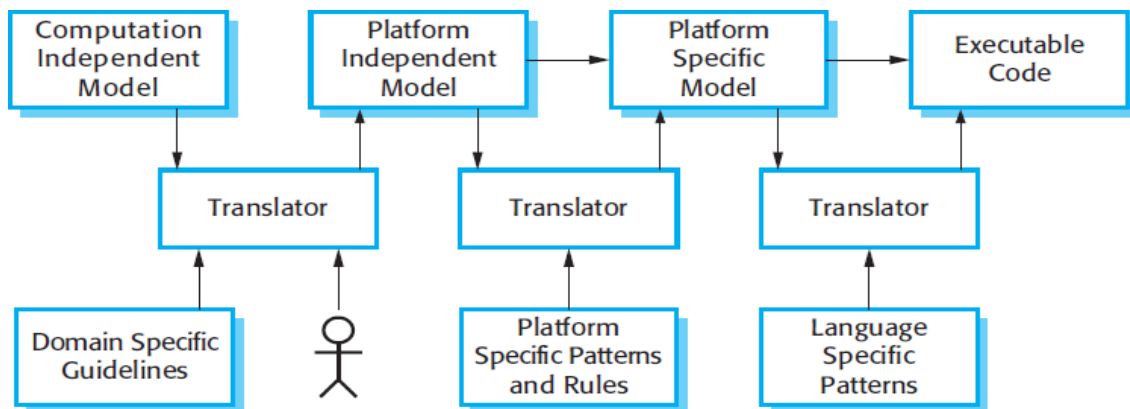


Figure 17: MDA transformations

As I have said, transformations between these models may be defined and applied automatically by software tools. This is illustrated in Figure 17, which also shows a final level of automatic transformation. A transformation is applied to the PSM to generate executable code that runs on the designated software platform.

At the time of writing, automatic CIM to PIM translation is still at the research prototype stage. It is unlikely that completely automated translation tools will be available in the near future. Human intervention, indicated by a stick figure in Figure 17, will be needed for the foreseeable future. CIMs are related and part of the translation process may involve linking concepts in different CIMs. For example, the concept of a role in a security CIM may be mapped onto the concept of a staff member in a hospital CIM. Mellor and Balcer (2002) give the name ‘bridges’ to the information that supports mapping from one CIM to another.

The translation of PIMs to PSMs is more mature and several commercial tools are available that provide translators from PIMs to common platforms such as Java and J2EE. These rely on an extensive library of platform-specific rules and patterns to convert the PIM to the PSM. There may be several PSMs for each PIM in the system. If a software system is intended to run on different platforms (e.g., J2EE and .NET), then it is only necessary to maintain the PIM. The PSMs for each platform are automatically generated. This is illustrated in Figure 18.

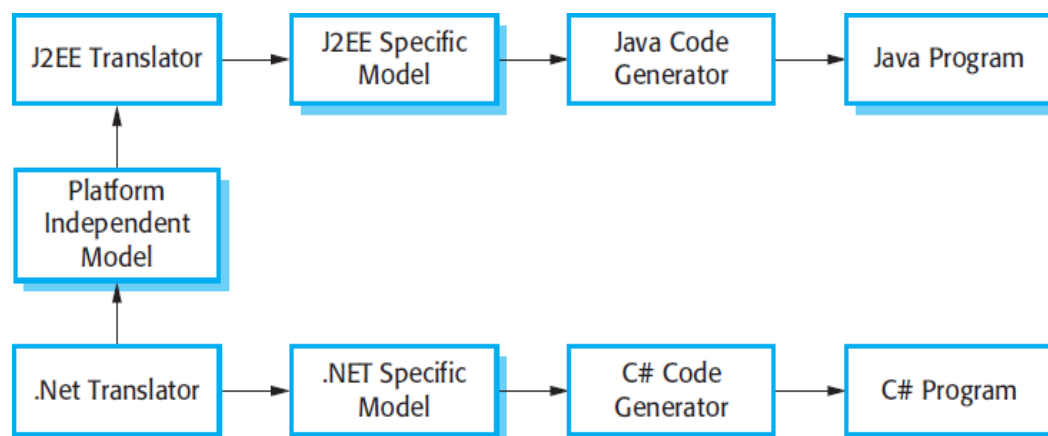


Figure 18: Multiple platform-specific models

Although MDA-support tools include platform-specific translators, it is often the case that these will only offer partial support for the translation from PIMs to PSMs. In the vast majority of cases, the execution environment for a system is more than the standard execution platform (e.g., J2EE, .NET, etc.). It also includes other application systems, application libraries that are specific to a company, and user interface libraries. As these vary significantly from one company to another, standard tool support is not available. Therefore, when MDA is introduced, special purpose translators may have to be created that take the characteristics of the local environment into account. In some cases (e.g., for user interface generation), completely automated PIM to PSM translation may be impossible.

Executable UML

The fundamental notion behind model-driven engineering is that completely automated transformation of models to code should be possible. To achieve this, you have to be able to construct graphical models whose semantics are well defined. You also need a way of adding information to graphical models about the ways in which the operations defined in the model are implemented. This is possible using a subset of UML 2, called Executable UML or xUML.

To create an executable sub-set of UML, the number of model types has therefore been dramatically reduced to three key model types:

- Domain models identify the principal concerns in the system. These are defined using UML class diagrams that include objects, attributes, and associations.
- Class models, in which classes are defined, along with their attributes and operations.
- State models, in which a state diagram is associated with each class and is used to describe the lifecycle of the class.

The dynamic behavior of the system may be specified declaratively using the object constraint language (OCL) or may be expressed using UML's action language. The action language is like a very high-level programming language where you can refer to objects and their attributes and specify actions to be carried out.

2.9 The Rational Unified Process

RUP is a good example of a hybrid process model. It brings together elements from all of the generic process models, illustrates good practice in specification and design and supports prototyping and incremental delivery.

The RUP recognizes that conventional process models present a single view of the process. In contrast, the RUP is normally described from three perspectives:

- A dynamic perspective, which shows the phases of the model over time.
- A static perspective, which shows the process activities that are enacted.
- A practice perspective, which suggests good practices to be used during the process.

Dynamic Perspective

Figure 19 shows the phases in the RUP. These are:

- **Inception:** The goal of the inception phase is to establish a business case for the system. You should identify all external entities (people and systems) that will interact with the system and define these interactions. You then use this information to assess the contribution that the system makes to the business. If this contribution is minor, then the project may be cancelled after this phase.

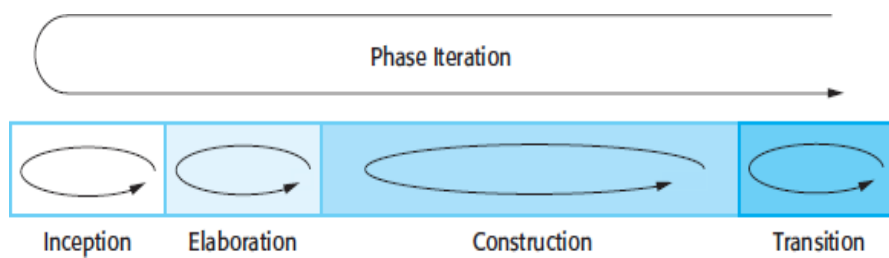


Figure 19: Phases in the Rational Unified Process

- **Elaboration:** The goals of the elaboration phase are to develop an understanding of the problem domain, establish an architectural framework for the system, develop the project plan, and identify key project risks.
- **Construction:** The construction phase involves system design, programming, and testing. Parts of the system are developed in parallel and integrated during this phase.
- **Transition:** The final phase of the RUP is concerned with moving the system from the development community to the user community and making it work in a real environment.

Static perspective

The static view of the RUP focuses on the activities that take place during the development process. These are called workflows in the RUP description.

Workflow	Description
Business modelling	The business processes are modelled using business use cases.
Requirements	Actors who interact with the system are identified and use cases are developed to model the system requirements.
Analysis and design	A design model is created and documented using architectural models, component models, object models, and sequence models.
Implementation	The components in the system are implemented and structured into implementation sub-systems. Automatic code generation from design models helps accelerate this process.
Testing	Testing is an iterative process that is carried out in conjunction with implementation. System testing follows the completion of the implementation.
Deployment	A product release is created, distributed to users, and installed in their workplace.
Configuration and change management	This supporting workflow manages changes to the system (see Chapter 25).
Project management	This supporting workflow manages the system development (see Chapters 22 and 23).
Environment	This workflow is concerned with making appropriate software tools available to the software development team.

Figure 20: Static workflows in the Rational Unified Process

Practice perspective

The practice perspective on the RUP describes good software engineering practices that are recommended for use in systems development. Six fundamental best practices are recommended:

- **Develop software iteratively:** Plan increments of the system based on customer priorities and develop the highest-priority system features early in the development process.
- **Manage requirements:** Explicitly document the customer's requirements and keep track of changes to these requirements. Analyze the impact of changes on the system before accepting them.
- **Use component-based architectures:** Structure the system architecture into components, as discussed earlier in this chapter.
- **Visually model software:** Use graphical UML models to present static and dynamic views of the software.
- **Verify software quality:** Ensure that the software meets the organizational quality standards.
- **Control changes to software:** Manage changes to the software using a change management system and configuration management procedures and tools.

2.10 Software Design and Implantation

Software design and implementation is the stage in the software engineering process at which an executable software system is developed. For some simple systems, software design and implementation is software engineering, and all other activities are merged with this process. However, for large systems, software design and implementation is only one of a set of processes (requirements engineering, verification and validation, etc.) involved in software engineering.

One of the most important implementation decisions that has to be made at an early stage of a software project is whether or not you should buy or build the application software. In a wide range of domains, it is now possible to buy off-the-shelf systems (COTS) that can be adapted and tailored to the user's requirements.

2.11 Object-oriented design using the UML

An object-oriented system is made up of interacting objects that maintain their own local state and provide operations on that state. The representation of the state is private and cannot be accessed directly from outside the object. Object-oriented design processes involve designing object classes and the relationships between these classes. These classes define the objects in the system and their interactions. When the design is realized as an executing program, the objects are created dynamically from these class definitions.

Object-oriented systems are easier to change than systems developed using functional approaches. Objects include both data and operations to manipulate that data. They may therefore be understood and modified as stand-alone entities. Changing the implementation of an object or adding services should not affect other system objects.

Because objects are associated with things, there is often a clear mapping between real-world entities (such as hardware components) and their controlling objects in the system. This improves the understandability, and hence the maintainability, of the design.

To develop a system design from concept to detailed, object-oriented design, there are several things that you need to do:

- Understand and define the context and the external interactions with the system.
- Design the system architecture.

- Identify the principal objects in the system.
- Develop design models.
- Specify interfaces.

System context and interactions

The first stage in any software design process is to develop an understanding of the relationships between the software that is being designed and its external environment. This is essential for deciding how to provide the required system functionality and how to structure the system to communicate with its environment. Understanding of the context also lets you establish the boundaries of the system.

System context models and interaction models present complementary views of the relationships between a system and its environment:

- A system context model is a structural model that demonstrates the other systems in the environment of the system being developed.
- An interaction model is a dynamic model that shows how the system interacts with its environment as it is used.

The context model of a system may be represented using associations. Associations simply show that there are some relationships between the entities involved in the association. The nature of the relationships is now specified. You may therefore document the environment of the system using a simple block diagram, showing the entities in the system and their associations. This is illustrated in Figure 21, which shows that the systems in the environment of each weather station are a weather information system, an onboard satellite system, and a control system. The cardinality information on the link shows that there is one control system but several weather stations, one satellite, and one general weather information system.

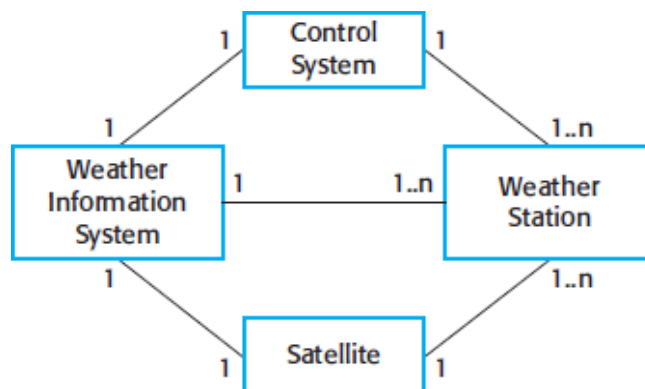


Figure 21: System context for the weather station

When you model the interactions of a system with its environment you should use an abstract approach that does not include too much detail. One way to do this is to use a use case model. The use case model for the weather station is shown in Figure 22. This shows that the weather station interacts with the weather information system to report weather data and the status of the weather station hardware. Other interactions are with a control system that can issue specific weather station control commands.

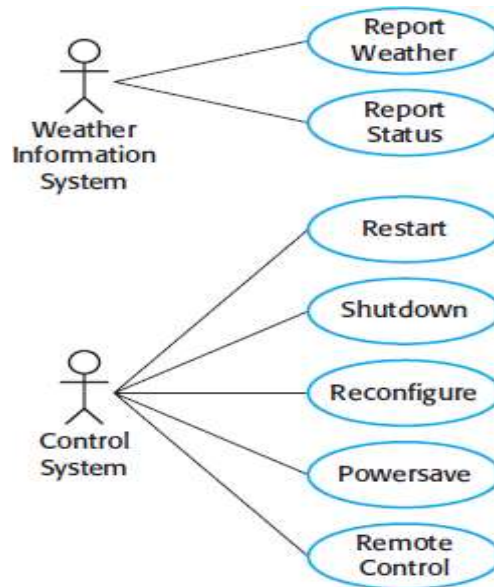


Figure 22: Weatherstation use cases

Architectural design

Once the interactions between the software system and the system's environment have been defined, you use this information as a basis for designing the system architecture. You identify the major components that make up the system and their interactions, and then may organize the components using an architectural pattern.

The high-level architectural design for the weather station software is shown in Figure 23. The weather station is composed of independent subsystems that communicate by broadcasting messages on a common infrastructure, shown as the Communication link in Figure 23. Each subsystem listens for messages on that infrastructure and picks up the messages that are intended for them.

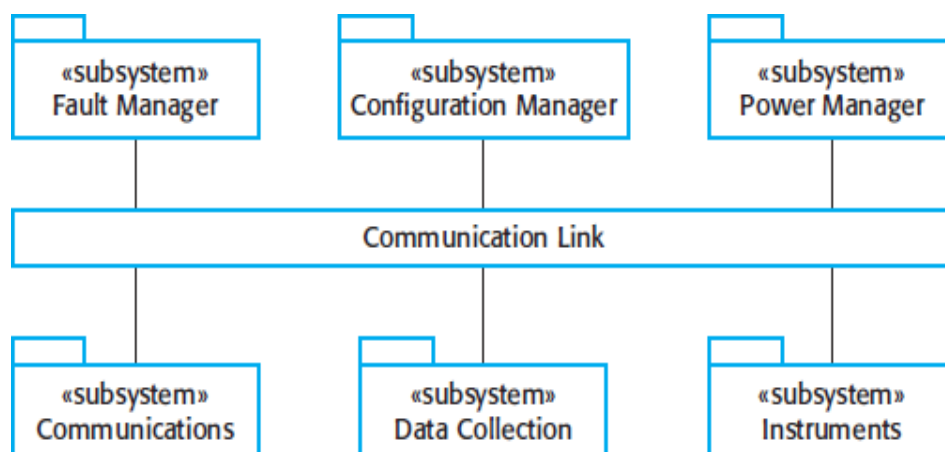


Figure 23: High-level architecture of the weather station

Figure 24 shows the architecture of the data collection subsystem, which is included in Figure 23. The Transmitter and Receiver objects are concerned with managing communications and the

WeatherDataobject encapsulates the information that is collected from the instruments and transmitted to the weather information system.

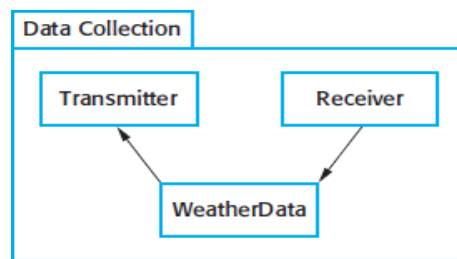


Figure 24: Architecture of data collection system

Object class identification

There have been various proposals made about how to identify object classes in object-oriented systems:

- Use a grammatical analysis of a natural language description of the system to be constructed.
- Use tangible entities (things) in the application domain such as aircraft, roles such as manager or doctor, events such as requests, interactions such as meetings, locations such as offices, organizational units such as companies, and so on.
- Use a scenario-based analysis where various scenarios of system use are identified and analyzed in turn. As each scenario is analyzed, the team responsible for the analysis must identify the required objects, attributes, and operations.

In the wilderness weather station, object identification is based on the tangible hardware in the system. Five object classes are shown in Figure 25. The Ground thermometer, Anemometer, and Barometer objects are application domain objects, and the WeatherStation and WeatherData objects have been identified from the system description and the scenario (use case) description:

- The WeatherStation object class provides the basic interface of the weather station with its environment. I use a single object class to encapsulate all of these interactions, but in other designs you could design the system interface as several different classes.
- The WeatherData object class is responsible for processing the report weather command. It sends the summarized data from the weather station instruments to the weather information system.
- The Ground thermometer, Anemometer, and Barometer object classes are directly related to instruments in the system. They reflect tangible hardware entities in the system and the operations are concerned with controlling that hardware. These objects operate autonomously to collect data at the specified frequency and store the collected data locally. This data is delivered to the WeatherData object on request.

You use knowledge of the application domain to identify other objects, attributes, and services. We know that weather stations are often located in remote places and include various instruments that sometimes go wrong. Instrument failures should be reported automatically. This implies that you need attributes and operations to check the correct functioning of the instruments. There are many remote weather stations so each weather station should have its own identifier.

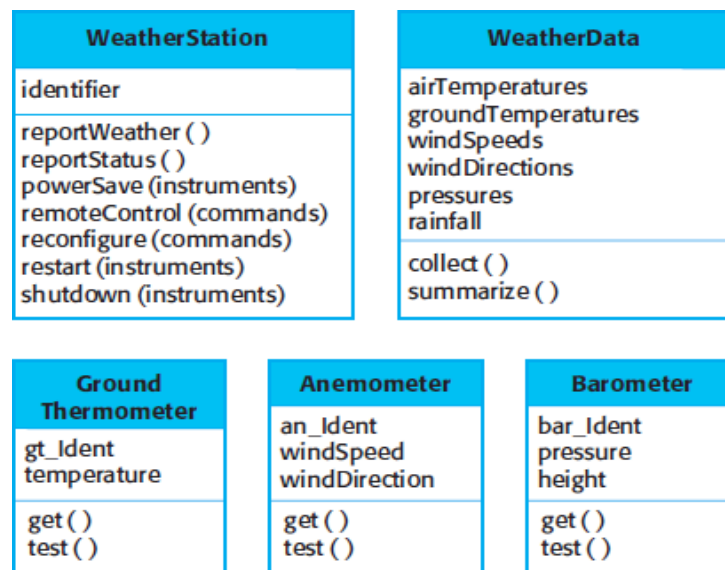


Figure 25: Weather station objects

Design models

Design or system models, show the objects or object classes in a system. They also show the associations and relationships between these entities. These models are the bridge between the system requirements and the implementation of a system. They have to be abstract so that unnecessary detail doesn't hide the relationships between them and the system requirements.

When you use the UML to develop a design, you will normally develop two kinds of design model:

- **Structural models:** which describe the static structure of the system using object classes and their relationships. Important relationships that may be documented at this stage are generalization (inheritance) relationships, uses/used-by relationships, and composition relationships.
- **Dynamic models:** which describe the dynamic structure of the system and show the interactions between the system objects. Interactions that may be documented include the sequence of service requests made by objects and the state changes that are triggered by these object interactions.

In the early stages of the design process, I think there are three models that are particularly useful for adding detail to use case and architectural models:

- **Subsystem models:** which that show logical groupings of objects into coherent subsystems. These are represented using a form of class diagram with each subsystem shown as a package with enclosed objects. Subsystem models are static (structural) models.
- **Sequence models:** which show the sequence of object interactions. These are represented using a UML sequence or a collaboration diagram. Sequence models are dynamic models.
- **State machine model:** which show how individual objects change their state in response to events. These are represented in the UML using state diagrams. State machine models are dynamic models.

A subsystem model is a useful static model as it shows how a design is organized into logically related groups of objects. I have already shown this type of model in Figure 23 to show the subsystems in the weather mapping system.

Sequence models are dynamic models that describe, for each mode of interaction, the sequence of object interactions that take place. When documenting a design, you should produce a sequence model for each significant interaction. Figure 26 is an example of a sequence model, shown as a UML sequence diagram.

This diagram shows the sequence of interactions that take place when an external system requests the summarized data from the weather station. You read sequence diagrams from top to bottom:

- The SatComms object receives a request from the weather information system to collect a weather report from a weather station. It acknowledges receipt of this request. The stick arrowhead on the sent message indicates that the external system does not wait for a reply but can carry on with other processing.
- SatComms sends a message to WeatherStation, via a satellite link, to create a summary of the collected weather data. Again, the stick arrowhead indicates that SatComms does not suspend itself waiting for a reply.
- WeatherStation sends a message to a Commslink object to summarize the weather data. In this case, the squared-off style of arrowhead indicates that the instance of the WeatherStation object class waits for a reply.
- Commslink calls the summarize method in the object WeatherData and waits for a reply.
- The weather data summary is computed and returned to WeatherStation via the Commslink object.
- WeatherStation then calls the SatComms object to transmit the summarized data to the weather information system, through the satellite communications system.

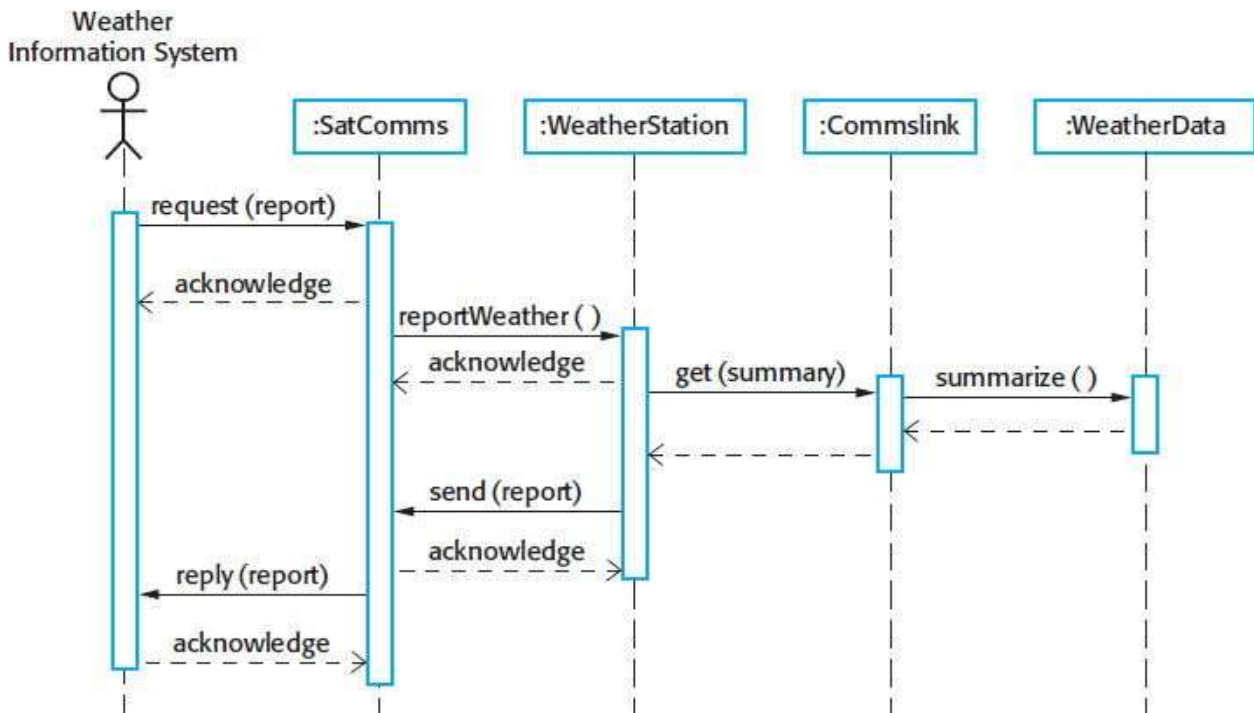


Figure 26: Sequence diagram describing data collection

Figure 27 is a state diagram for the weather station system that shows how it responds to requests for various services. You can read this diagram as follows:

- If the system state is Shutdown then it can respond to a restart(), a reconfigure(), or a powerSave() message. The unlabeled arrow with the black blob indicates that the Shutdown state is the initial state. A restart() message causes a transition to normal operation. Both the powerSave() and reconfigure() messages cause a transition to a state in which the system reconfigures itself. The state diagram shows that reconfiguration is only allowed if the system has been shut down.
- In the Running state, the system expects further messages. If a shutdown() message is received, the object returns to the shutdown state.

- If a reportWeather() message is received, the system moves to the Summarizing state. When the summary is complete, the system moves to a Transmitting state where the information is transmitted to the remote system. It then returns to the Running state.
- If a reportStatus() message is received, the system moves to the Testing state, then the Transmitting state, before returning to the Running state.
- If a signal from the clock is received, the system moves to the Collecting state, where it collects data from the instruments. Each instrument is instructed in turn to collect its data from the associated sensors.
- If a remoteControl() message is received, the system moves to a controlled state in which it responds to a different set of messages from the remote control room. These are not shown on this diagram.

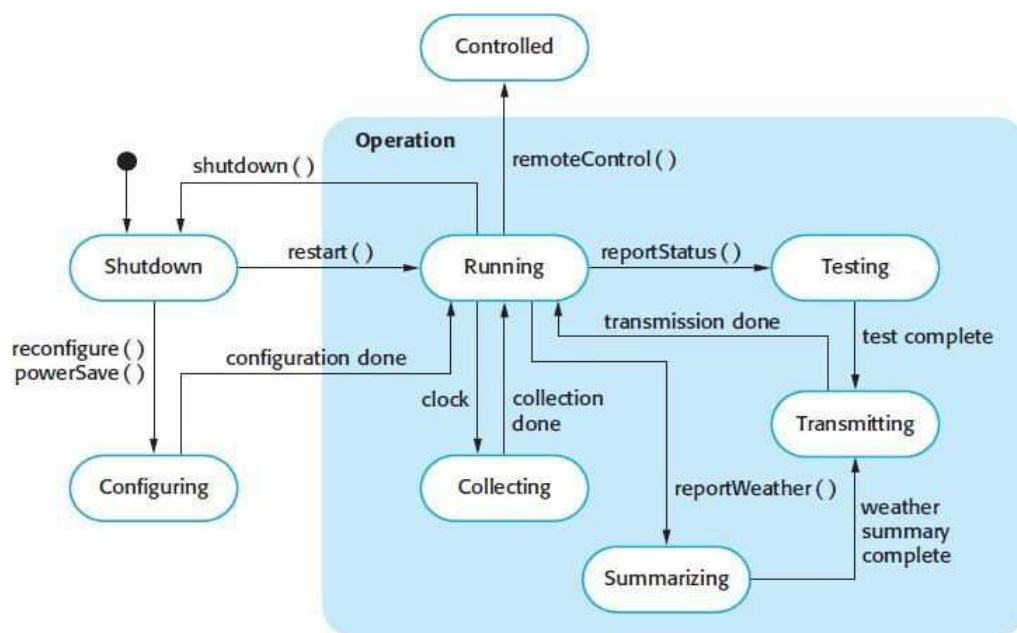


Figure 27: Weather station state diagram

Interface specification

An important part of any design process is the specification of the interfaces between the components in the design. You need to specify interfaces so that objects and subsystems can be designed in parallel. Once an interface has been specified, the developers of other objects may assume that interface will be implemented.

Interface design is concerned with specifying the detail of the interface to an object or to a group of objects. This means defining the signatures and semantics of the services that are provided by the object or by a group of objects. Interfaces can be specified in the UML using the same notation as a class diagram. However, there is no attribute section and the UML stereotype «interface» should be included in the name part. The semantics of the interface may be defined using the object constraint language (OCL).

Figure 28 shows two interfaces that may be defined for the weather station. The left-hand interface is a reporting interface that defines the operation names that are used to generate weather and status reports. These map directly to operations in the WeatherStation object. The remote control interface provides four operations, which map onto a single method in the WeatherStation object. In this case, the individual operations are encoded in the command string associated with the remoteControl method, shown in Figure 25.

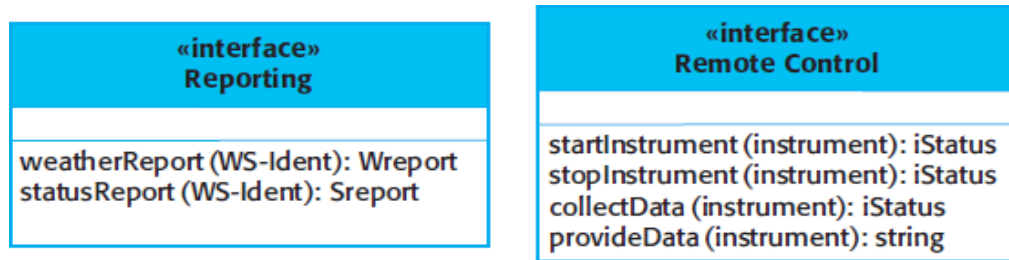


Figure 28: Weather station interfaces

2.12 Design patterns

The pattern is a description of the problem and the essence of its solution, so that the solution may be reused in different settings. The pattern is not a detailed specification. Rather, you can think of it as a description of accumulated wisdom and experience, a well-tried solution to a common problem.

Patterns and Pattern Languages are ways to describe best practices, good designs, and capture experience in a way that it is possible for others to reuse this experience.

The four essential elements of design patterns are:

- A name that is a meaningful reference to the pattern.
- A description of the problem area that explains when the pattern may be applied.
- A solution description of the parts of the design solution, their relationships, and their responsibilities.
- A statement of the consequences—the results and trade-offs—of applying the pattern. This can help designers understand whether or not a pattern can be used in a particular situation.

Pattern can be used in situations where different presentations of an object's state are required. It separates the object that must be displayed from the different forms of presentation. This is illustrated in Figure 29, which shows two graphical presentations of the same data set.

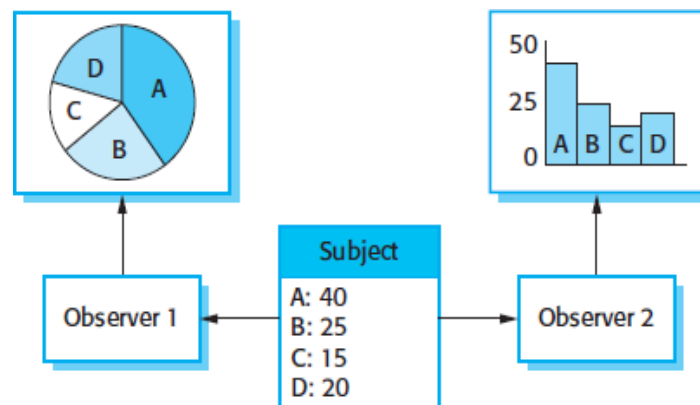


Figure 29: Multiple displays

2.13 Implementation issues

The three different issues that need to be considered during software implementation are:

- **Reuse:** Most modern software is constructed by reusing existing components or systems. When you are developing software, you should make as much use as possible of existing code.
- **Configuration management:** During the development process, many different versions of each software component are created. If you don't keep track of these versions in a configuration

management system, you are liable to include the wrong versions of these components in your system.

- **Host-target development:** Production software does not usually execute on the same computer as the software development environment. Rather, you develop it on one computer (the host system) and execute it on a separate computer (the target system). The host and target systems are sometimes of the same type but, often they are completely different.

Reuse

From the 1960s to the 1990s, most new software was developed from scratch, by writing all code in a high-level programming language. The only significant reuse of software was the reuse of functions and objects in programming language libraries. Consequently, an approach to development based around the reuse of existing software emerged and is now generally used for business systems, scientific software, and, increasingly, in embedded systems engineering.

Software reuse is possible at a number of different levels:

- **The abstraction level:** At this level, you don't reuse software directly but rather use knowledge of successful abstractions in the design of your software.
- **The object level:** At this level, you directly reuse objects from a library rather than writing the code yourself. To implement this type of reuse, you have to find appropriate libraries and discover if the objects and methods offer the functionality that you need.
- **The component level:** Components are collections of objects and object classes that operate together to provide related functions and services. You often have to adapt and extend the component by adding some code of your own.
- **The system level:** At this level, you reuse entire application systems. This usually involves some kind of configuration of these systems. This may be done by adding and modifying code or by using the system's own configuration interface.

As the reused software has been tested in other applications, it should be more reliable than new software. However, there are costs associated with reuse:

- The costs of the time spent in looking for software to reuse and assessing whether or not it meets your needs.
- Where applicable, the costs of buying the reusable software.
- The costs of adapting and configuring the reusable software components or systems to reflect the requirements of the system that you are developing.
- The costs of integrating reusable software elements with each other (if you are using software from different sources) and with the new code that you have developed.

How to reuse existing knowledge and software should be the first thing you should think about when starting a software development project.

Configuration management

Configuration management is the name given to the general process of managing a changing software system. The aim of configuration management is to support the system integration process so that all developers can access the project code and documents in a controlled way, find out what changes have been made, and compile and link components to create a system.

There are, therefore, three fundamental configuration management activities:

- **Version management:** where support is provided to keep track of the different versions of software components. Version management systems include facilities to coordinate development by several

programmers. They stop one developer overwriting code that has been submitted to the system by someone else.

- **System integration:** where support is provided to help developers define what versions of components are used to create each version of a system. This description is then used to build a system automatically by compiling and linking the required components.
- **Problem tracking:** where support is provided to allow users to report bugs and other problems, and to allow all developers to see who is working on these problems and when they are fixed.

Host-target development

Most software development is based on a host-target model. Software is developed on one computer (the host), but runs on a separate machine (the target). More generally, we can talk about a development platform and an execution platform. A platform is more than just hardware. It includes the installed operating system plus other supporting software such as a database management system or, for development platforms, an interactive development environment.

Sometimes, the development and execution platforms are the same, making it possible to develop the software and test it on the same machine. More commonly, however, they are different so that you need to either move your developed software to the execution platform for testing or run a simulator on your development machine.

A software development platform should provide a range of tools to support software engineering processes. These may include:

- An integrated compiler and syntax-directed editing system that allows you to create, edit, and compile code.
- A language debugging system.
- Graphical editing tools, such as tools to edit UML models.
- Testing tools, such as JUnit that can automatically run a set of tests on a new version of a program.
- Project support tools that help you organize the code for different development projects.

However, for distributed systems, you need to decide on the specific platforms where the components will be deployed. Issues that you have to consider in making this decision are:

- **The hardware and software requirements of a component:** If a component is designed for specific hardware architecture, or relies on some other software system, it must obviously be deployed on a platform that provides the required hardware and software support.
- **The availability requirements of the system:** High-availability systems may require components to be deployed on more than one platform. This means that, in the event of platform failure, an alternative implementation of the component is available.
- **Component communications:** If there is a high level of communications traffic between components, it usually makes sense to deploy them on the same platform or on platforms that are physically close to one other. This reduces communications latency, the delay between the time a message is sent by one component and received by another.

2.14 Open source development

Open source development is an approach to software development in which the source code of a software system is published and volunteers are invited to participate in the development process. Its roots are in the Free Software Foundation (<http://www.fsf.org>), which advocates that source code should not be proprietary but rather should always be available for users to examine and modify as they wish. There was

an assumption that the code would be controlled and developed by a small core group, rather than users of the code.

Open source software extended this idea by using the Internet to recruit a much larger population of volunteer developers. Many of them are also users of the code. In principle at least, any contributor to an open source project may report and fix bugs and propose new features and functionality. However, in practice, successful open source systems still rely on a core group of developers who control changes to the software.

Advantages of open source development

- It is usually fairly cheap or free to acquire open source software. You can normally download open source software without charge.
- The other key benefit of using open source products is that mature open source systems are usually very reliable. The reason for this is that they have a large population of users who are willing to fix problems themselves rather than report these problems to the developer and wait for a new release of the system.
- Bugs are discovered and repaired more quickly than is usually possible with proprietary software.

For a company involved in software development, there are two open source issues that have to be considered:

- Should the product that is being developed make use of open source components?
- Should an open source approach be used for the software's development?

The answers to these questions depend on the type of software that is being developed and the background and experience of the development team.

Open source licensing

Although a fundamental principle of open-source development is that source code should be freely available, this does not mean that anyone can do as they wish with that code. Legally, the developer of the code (either a company or an individual) still owns the code. They can place restrictions on how it is used by including legally binding conditions in an open source software license.

Most open source licenses are derived from one of three general models:

- **The GNU General Public License (GPL):** This is a so-called 'reciprocal' license that, simplistically, means that if you use open source software that is licensed under the GPL license, then you must make that software open source.
- **The GNU Lesser General Public License (LGPL):** This is a variant of the GPL license where you can write components that link to open source code without having to publish the source of these components.
- **The Berkley Standard Distribution (BSD) License:** This is a non-reciprocal license, which means you are not obliged to republish any changes or modifications made to open source code. You can include the code in proprietary systems that are sold. If you use open source components, you must acknowledge the original creator of the code.

Licensing issues are important because if you use open-source software as part of a software product, then you may be obliged by the terms of the license to make your own product open source.

The companies managing projects that use open source should:

- Establish a system for maintaining information about open source components that are downloaded and used. You have to keep a copy of the license for each component that was valid at the time the

component was used. Licenses may change so you need to know the conditions that you have agreed to.

- Be aware of the different types of licenses and understand how a component is licensed before it is used.
- Be aware of evolution pathways for components. You need to know a bit about the open source project where components are developed to understand how they might change in future.
- Educate people about open source. It's not enough to have procedures in place to ensure compliance with license conditions. You also need to educate developers about open source and open source licensing.
- Have auditing systems in place. Developers, under tight deadlines, might be tempted to break the terms of a license. If possible, you should have software in place to detect and stop this.
- Participate in the open source community. If you rely on open source products, you should participate in the community and help support their development.

2.15 Assignment Questions

1. With a neat diagram explain RUP.
2. Explain generalization and aggregation.
3. Explain context model with neat diagram.
4. Write short notes on open source development.
5. Explain event driven modeling with diagram.

2.15 Outcome

- An idea about why different types of model are required and the fundamental system modeling perspectives of context, interaction, structure, and behaviour
- Familiarized with some of the diagram types in the Unified Modeling Language (UML) and how these diagrams may be used in system modelling
- An idea about how the Rational Unified Process integrates good software engineering practice to create adaptable software processes

2.16 Further Reading

1. Ian Sommerville: Software Engineering, 9th Edition, Pearson Education, 2012. (Listed topics only from Chapters 1,2,3,4, 5, 7, 8, 9, 23, and 24)
2. The SCRUM Primer, Ver 2.0, <http://www.goodagile.com/scrumprimer/scrumprimer20.pdf>
3. Pressman: Software Engineering-A Practitioners approach, 7th Edition, Tata McGraw Hill.
4. Pankaj Jalote: An Integrated Approach to Software Engineering, WileyIndia
5. <http://agilemanifesto.org/>
6. <http://www.jamesshore.com/Agile-Book/>

MODULE 4

STRUCTURE

- 3.1 Introduction
- 3.2 Objective
- 3.3 Overview
- 3.4 Development testing
- 3.5 Test-driven development
- 3.6 Release testing
- 3.7 User testing
- 3.8 Software Evolution
- 3.9 Evolution processes
- 3.10 Program evolution dynamics
- 3.11 Software maintenance
- 3.12 Legacy system management
- 3.13 Assignment Questions
- 3.14 Outcome
- 3.15 Further Reading

3.1 Introduction

This module gives the overview of different testing process such as development testing, release testing, user testing. The test driven development, software evolution process, programming evolution dynamics is discussed in detail. Software maintenance and legacy system management is highlighted.

3.2 Objective

- Understand the stages of testing from testing, during development to acceptance testing by system customers
- Know the important differences between component, system, and release testing and be aware of user testing processes and techniques
- Have learned about different types of software maintenance and the factors that affect maintenance costs
- Understand software evolution processes and influences on these processes

3.2 Overview

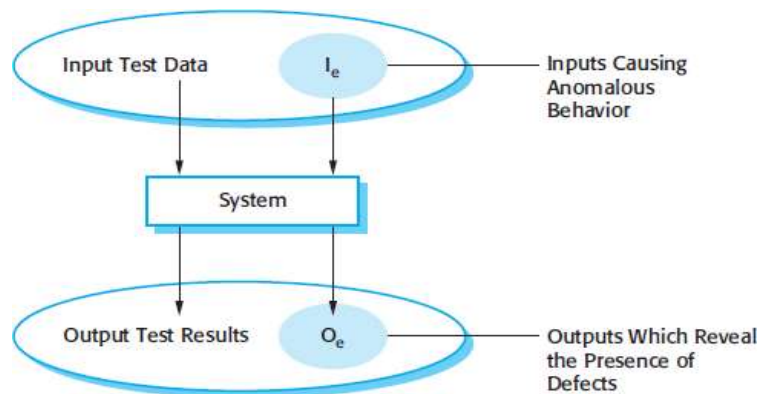
Testing is intended to show that a program does what it is intended to do and to discover program defects before it is put into use.

The testing process has two distinct goals:

- To demonstrate to the developer and the customer that the software meets its requirements.
- To discover situations in which the behavior of the software is incorrect, undesirable, or does not conform to its specification. These are a consequence of software defects.

The first goal leads to validation testing, where you expect the system to perform correctly using a given set of test cases that reflect the system's expected use. The second goal leads to defect testing, where the test cases are designed to expose defects.

The diagram shown in below figure may help to explain the differences between validation testing and defect testing. Think of the system being tested as a black box. The system accepts inputs from some input set I and generates outputs in an output set O . Some of the outputs will be erroneous. These are the outputs in set O_e that are generated by the system in response to inputs in the set I_e . The priority in defect testing is to find those inputs in the set I_e because these reveal problems with the system. Validation testing involves testing with correct inputs that are outside I_e . These stimulate the system to generate the expected correct outputs.



An input-output model of program testing

- ‘Validation: Are we building the right product?’
- ‘Verification: Are we building the product right?’

The ultimate goal of verification and validation processes is to establish confidence that the software system is ‘fit for purpose’. The level of required confidence depends on the system’s purpose, the expectations of the system users, and the current marketing environment for the system:

- **Software purpose** The more critical the software, the more important that it is reliable.
- **User expectations** Because of their experiences with buggy, unreliable software, many users have low expectations of software quality.
- **Marketing environment** When a system is marketed, the sellers of the system must take into account competing products, the price that customers are willing to pay for a system, and the required schedule for delivering that system.

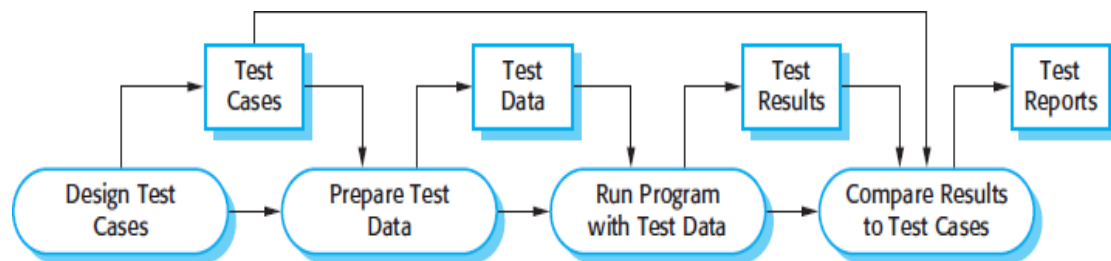
As well as software testing, the verification and validation process may involve software inspections and reviews. Inspections and reviews analyze and check the system requirements, design models, the program source code, and even proposed system tests.

Inspections mostly focus on the source code of a system but any readable representation of the software, such as its requirements or a design model, can be inspected.

There are three advantages of software inspection over testing:

- During testing, errors can mask (hide) other errors. When an error leads to unexpected outputs, you can never be sure if later output anomalies are due to a new error or are side effects of the original error. Because inspection is a static process, you don’t have to be concerned with interactions between errors.
- Incomplete versions of a system can be inspected without additional costs.
- As well as searching for program defects, an inspection can also consider broader quality attributes of a program, such as compliance with standards, portability, and maintainability.

The below figure is an abstract model of the ‘traditional’ testing process, as used in plan driven development. Test cases are specifications of the inputs to the test and the expected output from the system (the test results), plus a statement of what is being tested. Test data are the inputs that have been devised to test a system. Test data can sometimes be generated automatically, but automatic test case generation is impossible, as people who understand what the system is supposed to do must be involved to specify the expected test results. However, test execution can be automated. The expected results are automatically compared with the predicted results so there is no need for a person to look for errors and anomalies in the test run.



A model of the software testing process

Typically, a commercial software system has to go through three stages of testing:

- Development testing, where the system is tested during development to discover bugs and defects.
- Release testing, where a separate testing team tests a complete version of the system before it is released to users. The aim of release testing is to check that the system meets the requirements of system stakeholders.
- User testing, where users or potential users of a system test the system in their own environment.

3.3 Development testing

Development testing includes all testing activities that are carried out by the team developing the system.

During development, testing may be carried out at three levels of granularity:

- Unit testing, where individual program units or object classes are tested. Unit testing should focus on testing the functionality of objects or methods.
- Component testing, where several individual units are integrated to create composite components. Component testing should focus on testing component interfaces.
- System testing, where some or all of the components in a system are integrated and the system is tested as a whole. System testing should focus on testing component interactions.

Unit testing

Unit testing is the process of testing program components, such as methods or object classes. Individual functions or methods are the simplest type of component.

When you are testing object classes, you should design your tests to provide coverage of all of the features of the object. This means that you should:

- Test all operations associated with the object.
- Set and check the value of all attributes associated with the object.
- Put the object into all possible states. This means that you should simulate all events that cause a state change.

Consider, for example, the weather station object. The interface of this object is shown in below figure. It has a single attribute, which is its identifier. This is a constant that is set when the weather station is installed. You therefore only need a test that checks if it has been properly set up. You need to define test cases for all of the methods associated with the object such as reportWeather, reportStatus, etc. Ideally, you should test methods in isolation but, in some cases, some test sequences are necessary.

WeatherStation
identifier
reportWeather ()
reportStatus ()
powerSave (instruments)
remoteControl (commands)
reconfigure (commands)
restart (instruments)
shutdown (instruments)

The weather station object interface

To test the states of the weather station, you use a state model. Using this model, you can identify sequences of state transitions that have to be tested and define event sequences to force these transitions. In principle, you should test every possible state transition sequence, although in practice this may be too expensive.

Whenever possible, you should automate unit testing. In automated unit testing, you make use of a test automation framework to write and run your program tests.

An automated test has three parts:

- A setup part, where you initialize the system with the test case, namely the inputs and expected outputs.
- A call part, where you call the object or method to be tested.
- An assertion part where you compare the result of the call with the expected result. If the assertion evaluates to true, the test has been successful; if false, then it has failed.

Sometimes the object that you are testing has dependencies on other objects that may not have been written or which slow down the testing process if they are used.

Choosing unit test cases

Testing is expensive and time consuming, so it is important that you choose effective unit test cases. Effectiveness, in this case, means two things:

- The test cases should show that, when used as expected, the component that you are testing does what it is supposed to do.
- If there are defects in the component, these should be revealed by test cases.

Two possible strategies here that can be effective in helping to choose test cases.

- **Partition testing**, where you identify groups of inputs that have common characteristics and should be processed in the same way. You should choose tests from within each of these groups.
- **Guideline-based testing**, where you use testing guidelines to choose test cases. These guidelines reflect previous experience of the kinds of errors that programmers often make when developing components.

Some of the most general guidelines for choosing unit test cases are:

- Choose inputs that force the system to generate all error messages.
- Design inputs that cause input buffers to overflow.
- Repeat the same input or series of inputs numerous times.
- Force invalid outputs to be generated.
- Force computation results to be too large or too small.

Component testing

Software components are often composite components that are made up of several interacting objects. You access the functionality of these objects through the defined component interface. Testing composite components should therefore focus on showing that the component interface behaves according to its specification. You can assume that unit tests on the individual objects within the component have been completed.

There are different types of interface between program components and, consequently, different types of interface error that can occur:

- **Parameter interfaces** These are interfaces in which data or sometimes function references are passed from one component to another. Methods in an object have a parameter interface.
- **Shared memory interfaces** These are interfaces in which a block of memory is shared between components. Data is placed in the memory by one subsystem and retrieved from there by other subsystems. This type of interface is often used in embedded systems, where sensors create data that is retrieved and processed by other system components.
- **Procedural interfaces** These are interfaces in which one component encapsulates a set of procedures that can be called by other components. Objects and reusable components have this form of interface.
- **Message passing interfaces** These are interfaces in which one component requests a service from another component by passing a message to it. A return message includes the results of executing the service.

These errors fall into three classes:

- **Interface misuse** A calling component calls some other component and makes an error in the use of its interface.
- **Interface misunderstanding** A calling component misunderstands the specification of the interface of the called component and makes assumptions about its behavior.
- **Timing errors** These occur in real-time systems that use a shared memory or a message-passing interface. The producer of data and the consumer of data may operate at different speeds.

Some general guidelines for interface testing are:

- Examine the code to be tested and explicitly list each call to an external component. Design a set of tests in which the values of the parameters to the external components are at the extreme ends of their ranges. These extreme values are most likely to reveal interface inconsistencies.
- Where pointers are passed across an interface, always test the interface with null pointer parameters.
- Where a component is called through a procedural interface, design tests that deliberately cause the component to fail. Differing failure assumptions are one of the most common specification misunderstandings.
- Use stress testing in message passing systems. This means that you should design tests that generate many more messages than are likely to occur in practice. This is an effective way of revealing timing problems.

- Where several components interact through shared memory, design tests that vary the order in which these components are activated. These tests may reveal implicit assumptions made by the programmer about the order in which the shared data is produced and consumed.

System testing

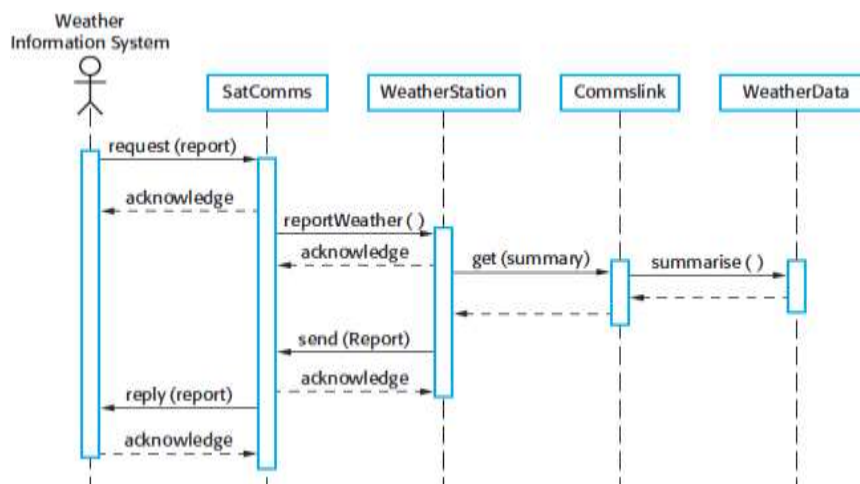
System testing during development involves integrating components to create a version of the system and then testing the integrated system. System testing checks that components are compatible, interact correctly and transfer the right data at the right time across their interfaces. It obviously overlaps with component testing but there are two important differences:

During system testing, reusable components that have been separately developed and off-the-shelf systems may be integrated with newly developed components. The complete system is then tested.

Components developed by different team members or groups may be integrated at this stage. System testing is a collective rather than an individual process.

Therefore system testing should focus on testing the interactions between the components and objects that make up a system. You may also test reusable components or systems to check that they work as expected when they are integrated with new components. This interaction testing should discover those component bugs that are only revealed when a component is used by other components in the system. Interaction testing also helps find misunderstandings, made by component developers, about other components in the system.

Because of its focus on interactions, use case-based testing is an effective approach to system testing. To illustrate this, I use an example from the wilderness weather station system where the weather station is asked to report summarized weather data to a remote computer. The below figure shows the sequence of operations in the weather station when it responds to a request to collect data for the mapping system. You can use this diagram to identify operations that will be tested and to help design the test cases to execute the tests.



Collect weather data sequence chart

The sequence diagram helps you design the specific test cases that you need as it shows what inputs are required and what outputs are created:

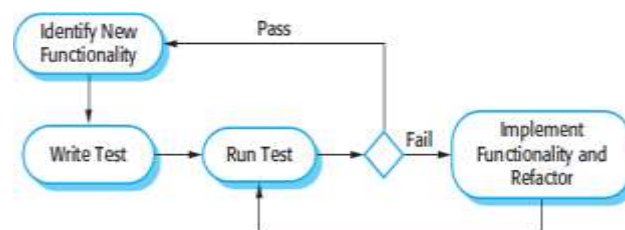
- An input of a request for a report should have an associated acknowledgment. A report should ultimately be returned from the request. During testing, you should create summarized data that can be used to check that the report is correctly organized.

- An input request for a report to WeatherStation results in a summarized report being generated. You can test this in isolation by creating raw data corresponding to the summary that you have prepared for the test of SatComms and checking that the WeatherStation object correctly produces this summary. This raw data is also used to test the WeatherData object.

3.4 Test-driven development

Test-driven development (TDD) is an approach to program development in which you interleave testing and code development. Essentially, you develop the code incrementally, along with a test for that increment. You don't move on to the next increment until the code that you have developed passes its test.

The fundamental TDD process is shown in below figure. The steps in the process are as follows:



Test-driven development

- Start by identifying the increment of functionality that is required. This should normally be small and implementable in a few lines of code.
- Write a test for this functionality and implement this as an automated test. This means that the test can be executed and will report whether or not it has passed or failed.
- Then run the test, along with all other tests that have been implemented. Initially, you have not implemented the functionality so the new test will fail. This is deliberate as it shows that the test adds something to the test set.
- Then implement the functionality and re-run the test. This may involve refactoring existing code to improve it and add new code to what's already there.
- Once all tests run successfully, you move on to implementing the next chunk of functionality.

Benefits of test-driven development are:

- **Code coverage:** In principle, every code segment that you write should have at least one associated test. Therefore, you can be confident that all of the code in the system has actually been executed. Code is tested as it is written so defects are discovered early in the development process.
- **Regression testing:** A test suite is developed incrementally as a program is developed. You can always run regression tests to check that changes to the program have not introduced new bugs.
- **Simplified debugging:** When a test fails, it should be obvious where the problem lies. The newly written code needs to be checked and modified.
- **System documentation:** The tests themselves act as a form of documentation that describe what the code should be doing. Reading the tests can make it easier to understand the code.

3.5 Release testing

Release testing is the process of testing a particular release of a system that is intended for use outside of the development team.

The primary goal of the release testing process is to convince the supplier of the system that it is good enough for use. Release testing is usually a black-box testing process where tests are derived from the

system specification. The system is treated as a black box whose behavior can only be determined by studying its inputs and the related outputs. Another name for this is ‘functional testing’, so-called because the tester is only concerned with functionality and not the implementation of the software.

Requirements-based testing

A general principle of good requirements engineering practice is that requirements should be testable; that is, the requirement should be written so that a test can be designed for that requirement. A tester can then check that the requirement has been satisfied. Requirements-based testing, therefore, is a systematic approach to test case design where you consider each requirement and derive a set of tests for it. Requirements-based testing is validation rather than defect testing—you are trying to demonstrate that the system has properly implemented its requirements.

Scenario testing

Scenario testing is an approach to release testing where you devise typical scenarios of use and use these to develop test cases for the system. A scenario is a story that describes one way in which the system might be used. Scenarios should be realistic and real system users should be able to relate to them.

Performance testing

Once a system has been completely integrated, it is possible to test for emergent properties, such as performance and reliability. Performance tests have to be designed to ensure that the system can process its intended load. This usually involves running a series of tests where you increase the load until the system performance becomes unacceptable. This is known as ‘stress testing’.

This type of testing has two functions:

1. It tests the failure behavior of the system. Circumstances may arise through an unexpected combination of events where the load placed on the system exceeds the maximum anticipated load. In these circumstances, it is important that system failure should not cause data corruption or unexpected loss of user services. Stress testing checks that overloading the system causes it to ‘fail-soft’ rather than collapse under its load.
2. It stresses the system and may cause defects to come to light that would not normally be discovered. Although it can be argued that these defects are unlikely to cause system failures in normal usage, there may be unusual combinations of normal circumstances that the stress testing replicates.

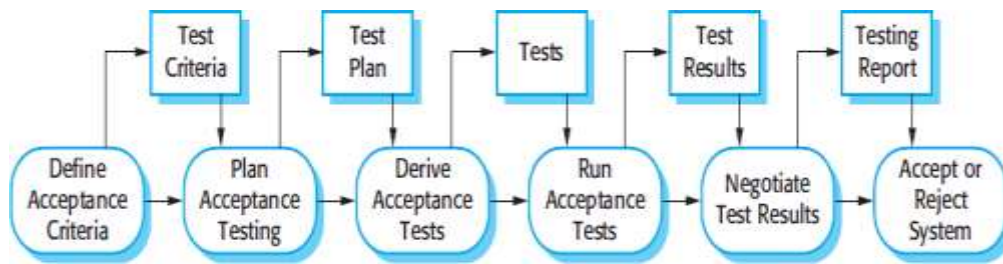
3.6 User testing

User or customer testing is a stage in the testing process in which users or customers provide input and advice on system testing.

In practice, there are three different types of user testing:

1. **Alpha testing**, where users of the software work with the development team to test the software at the developer’s site.
2. **Beta testing**, where a release of the software is made available to users to allow them to experiment and to raise problems that they discover with the system developers.
3. **Acceptance testing**, where customers test a system to decide whether or not it is ready to be accepted from the system developers and deployed in the customer environment.

There are six stages in the acceptance testing process, as shown in below Figure. They are:



The acceptance testing process

1. **Define acceptance criteria** This stage should, ideally, take place early in the process before the contract for the system is signed. The acceptance criteria should be part of the system contract and be agreed between the customer and the developer. In practice, however, it can be difficult to define criteria so early in the process. Detailed requirements may not be available and there may be significant requirements change during the development process.
2. **Plan acceptance testing** This involves deciding on the resources, time, and budget for acceptance testing and establishing a testing schedule. The acceptance test plan should also discuss the required coverage of the requirements and the order in which system features are tested. It should define risks to the testing process, such as system crashes and inadequate performance, and discuss how these risks can be mitigated.
3. **Derive acceptance tests** Once acceptance criteria have been established, tests have to be designed to check whether or not a system is acceptable. Acceptance tests should aim to test both the functional and non-functional characteristics (e.g., performance) of the system. They should, ideally, provide complete coverage of the system requirements. In practice, it is difficult to establish completely objective acceptance criteria. There is often scope for argument about whether or not a test shows that a criterion has definitely been met.
4. **Run acceptance tests** The agreed acceptance tests are executed on the system. Ideally, this should take place in the actual environment where the system will be used, but this may be disruptive and impractical. Therefore, a user testing environment may have to be set up to run these tests. It is difficult to automate this process as part of the acceptance tests may involve testing the interactions between end-users and the system. Some training of end-users may be required.
5. **Negotiate test results** It is very unlikely that all of the defined acceptance tests will pass and that there will be no problems with the system. If this is the case, then acceptance testing is complete and the system can be handed over. More commonly, some problems will be discovered. In such cases, the developer and the customer have to negotiate to decide if the system is good enough to be put into use. They must also agree on the developer's response to identified problems.
6. **Reject/accept system** This stage involves a meeting between the developers and the customer to decide on whether or not the system should be accepted. If the system is not good enough for use, then further development is required to fix the identified problems. Once complete, the acceptance testing phase is repeated.

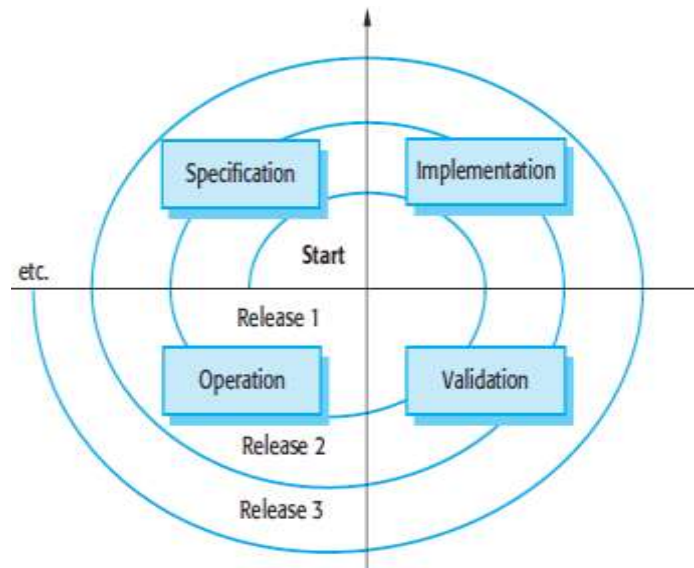
3.7 Software Evolution

Software development does not stop when a system is delivered but continues throughout the lifetime of the system. After a system has been deployed, it inevitably has to change if it is to remain useful. Business changes and changes to user expectations generate new requirements for the existing software. Parts of the software may have to be modified to correct errors that are found in operation, to adapt it for changes to its hardware and software platform, and to improve its performance or other non-functional characteristics.

Software evolution is important because organizations have invested large amounts of money in their software and are now completely dependent on these systems. Software evolution may be triggered by

changing business requirements, by reports of software defects, or by changes to other systems in a software system's environment.

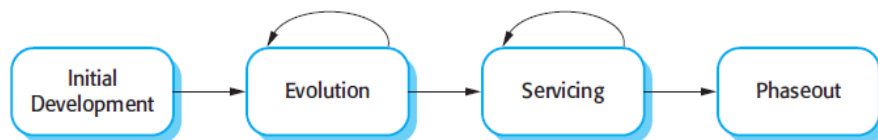
You should, therefore, think of software engineering as a spiral process with requirements, design, implementation, and testing going on throughout the lifetime of the system, it is shown in below Figure. You start by creating release 1 of the system. Once delivered, changes are proposed and the development of release 2 starts almost immediately. In fact, the need for evolution may become obvious even before the system is deployed so that later releases of the software may be under development before the current version has been released.



A spiral model of development and evolution

In this case, there are likely to be discontinuities in the spiral process. Requirements and design documents may not be passed from one company to another. Companies may merge or reorganize and inherit software from other companies, and then find that this has to be changed. When the transition from development to evolution is not seamless, the process of changing the software after delivery is often called 'software maintenance'.

Rajlich and Bennett (2000) proposed an alternative view of the software evolution life cycle, as shown in below Figure. In this model, they distinguish between evolution and servicing. Evolution is the phase in which significant changes to the software architecture and functionality may be made. During servicing, the only changes that are made are relatively small, essential changes.



Evolution and servicing

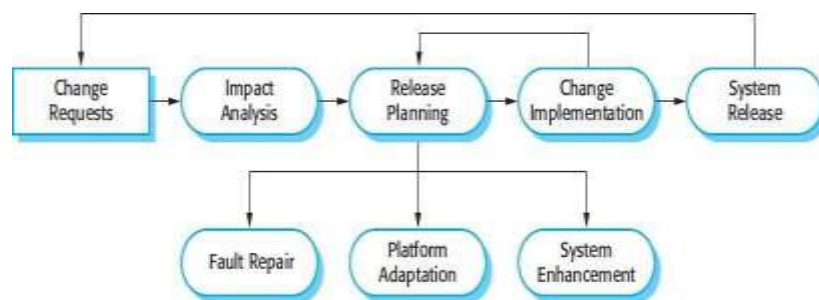
During evolution, the software is used successfully and there is a constant stream of proposed requirements changes. However, as the software is modified, its structure tends to degrade and changes become more and more expensive. This often happens after a few years of use when other environmental changes, such as hardware and operating systems, are also often required. At some stage in the life cycle, the software reaches a transition point where significant changes, implementing new requirements, become less and less cost effective.

At that stage, the software moves from evolution to servicing. During the servicing phase, the software is still useful and used but only small tactical changes are made to it. During this stage, the company is usually considering how the software can be replaced. In the final stage, phase-out, the software may still be used but no further changes are being implemented. Users have to work around any problems that they discover.

3.8 Evolution processes

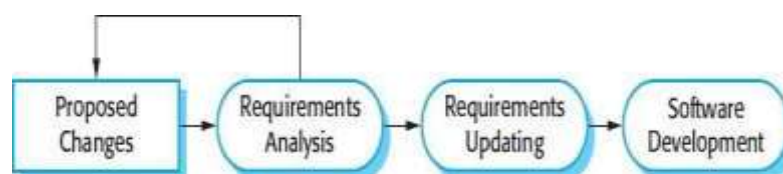
Software evolution processes vary depending on the type of software being maintained, the development processes used in an organization and the skills of the people involved. In some organizations, evolution may be an informal process where change requests mostly come from conversations between the system users and developers. In other companies, it is a formalized process with structured documentation produced at each stage in the process.

Below figure shows an overview of the evolution process. The process includes the fundamental activities of change analysis, release planning, system implementation, and releasing a system to customers. The cost and impact of these changes are assessed to see how much of the system is affected by the change and how much it might cost to implement the change. If the proposed changes are accepted, a new release of the system is planned. During release planning, all proposed changes (fault repair, adaptation, and new functionality) are considered. A decision is then made on which changes to implement in the next version of the system. The changes are implemented and validated, and a new version of the system is released. The process then iterates with a new set of changes proposed for the next release.



The software evolution process

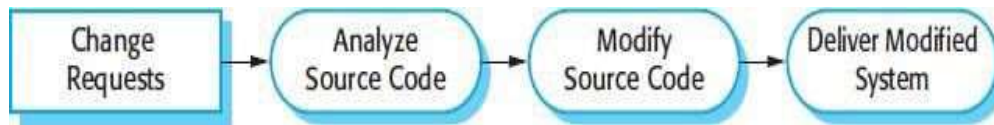
Ideally, the change implementation stage of this process should modify the system specification, design, and implementation to reflect the changes to the system (as shown in below Figure). New requirements that reflect the system changes are proposed, analyzed, and validated. System components are redesigned and implemented and the system is retested. If appropriate, prototyping of the proposed changes may be carried out as part of the change analysis process.



Change implementation

In these cases, the need to make the change quickly means that you may not be able to follow the formal change analysis process. Rather than modify the requirements and design, you make an emergency fix to the program to solve the immediate problem (It is as shown in below figure Figure). However, the danger is that the requirements, the software design, and the code become inconsistent. Although you may

intend to document the change in the requirements and design, additional emergency fixes to the software may then be needed. These take priority over documentation. Eventually, the original change is forgotten and the system documentation and code are never realigned.



The emergency repair process

3.9 Program evolution dynamics

Program evolution dynamics is the study of system change. In the 1970s and 1980s, Lehman and Belady (1985) carried out several empirical studies of system change with a view to understanding more about characteristics of software evolution. The work continued in the 1990s as Lehman and others investigated the significance of feedback in evolution processes (Lehman, 1996; Lehman et al., 1998; Lehman et al., 2001). From these studies, they proposed ‘Lehman’s laws’ concerning system change.

Law	Description
Continuing change	A program that is used in a real-world environment must necessarily change, or else become progressively less useful in that environment.
Increasing complexity	As an evolving program changes, its structure tends to become more complex. Extra resources must be devoted to preserving and simplifying the structure.
Large program evolution	Program evolution is a self-regulating process. System attributes such as size, time between releases, and the number of reported errors is approximately invariant for each system release.
Organizational stability	Over a program’s lifetime, its rate of development is approximately constant and independent of the resources devoted to system development.
Conservation of familiarity	Over the lifetime of a system, the incremental change in each release is approximately constant.
Continuing growth	The functionality offered by systems has to continually increase to maintain user satisfaction.
Declining quality	The quality of systems will decline unless they are modified to reflect changes in their operational environment.
Feedback system	Evolution processes incorporate multiagent, multiloop feedback systems and you have to treat them as feedback systems to achieve significant product improvement.

3.10 Software maintenance

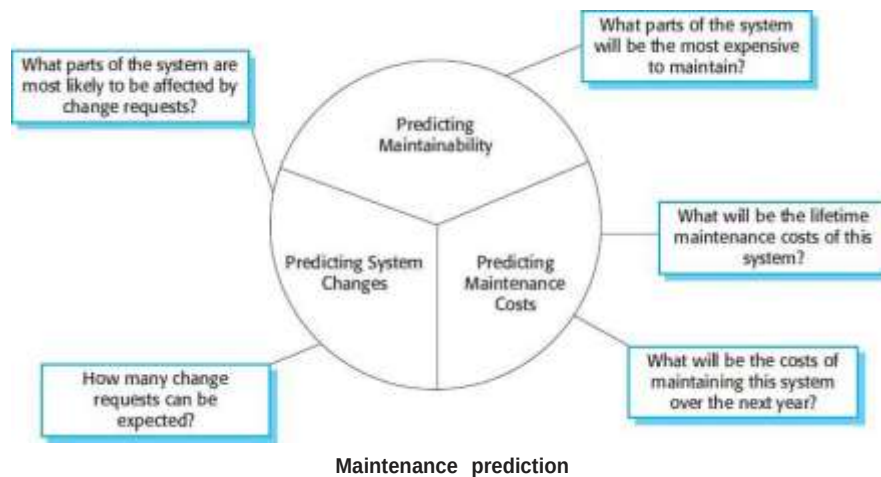
Software maintenance is the general process of changing a system after it has been delivered. The term is usually applied to custom software in which separate development groups are involved before and after delivery. The changes made to the software may be simple changes to correct coding errors, more extensive changes to correct design errors, or significant enhancements to correct specification errors or

accommodate new requirements. Changes are implemented by modifying existing system components and, where necessary, by adding new components to the system. There are three different types of software maintenance:

- **Fault repairs**Coding errors are usually relatively cheap to correct; design errors are more expensive as they may involve rewriting several program components. Requirements errors are the most expensive to repair because of the extensive system redesign which may be necessary.
- **Environmental adaptation**This type of maintenance is required when some aspect of the system's environment such as the hardware, the platform operating system, or other support software changes. The application system must be modified to adapt it to cope with these environmental changes.
- **Functionality addition**This type of maintenance is necessary when the system requirements change in response to organizational or business change. The scale of the changes required to the software is often much greater than for the other types of maintenance. It is usually more expensive to add functionality after a system is in operation than it is to implement the same functionality during development. The reasons for this are:
 - **Team stability**After a system has been delivered, it is normal for the development team to be broken up and for people to work on new projects. The new team or the individuals responsible for system maintenance do not understand the system or the background to system design decisions. They need to spend time understanding the existing system before implementing changes to it.
 - **Poor development practice**The contract to maintain a system is usually separate from the system development contract. The maintenance contract may be given to a different company rather than the original system developer. This factor, along with the lack of team stability, means that there is no incentive for a development team to write maintainable software. If a development team can cut corners to save effort during development it is worthwhile for them to do so, even if this means that the software is more difficult to change in the future.
 - **Staff skills**Maintenance staff are often relatively inexperienced and unfamiliar with the application domain. Maintenance has a poor image among software engineers. It is seen as a less-skilled process than system development and is often allocated to the most junior staff. Furthermore, old systems may be written in obsolete programming languages. The maintenance staff may not have much experience of development in these languages and must learn these languages to maintain the system.
 - **Program age and structure**As changes are made to programs, their structure tends to degrade. Consequently, as programs age, they become harder to understand and change. Some systems have been developed without modern software engineering techniques. They may never have been well structured and were perhaps optimized for efficiency rather than understandability. System documentation may be lost or inconsistent. Old systems may not have been subject to stringent configuration management so time is often wasted finding the right versions of system components to change.

Maintenance prediction

Managers hate surprises, especially if these result in unexpectedly high costs. You should therefore try to predict what system changes might be proposed and what parts of the system are likely to be the most difficult to maintain. You should also try to estimate the overall maintenance costs for a system in a given time period. The below figure shows these predictions and associated questions.



To evaluate the relationships between a system and its environment, you should assess:

- **The number and complexity of system interfaces** The larger the number of interfaces and the more complex these interfaces, the more likely it is that interface changes will be required as new requirements are proposed.
- **The number of inherently volatile system requirements** requirements that reflect organizational policies and procedures are likely to be more volatile than requirements that are based on stable domain characteristics.
- **The business processes in which the system is used** As business processes evolve, they generate system change requests. The more business processes that use a system, the more the demands for system change. Metrics that can be used for assessing maintainability are as follows:
- **Number of requests for corrective maintenance** An increase in the number of bug and failure reports may indicate that more errors are being introduced into the program than are being repaired during the maintenance process. This may indicate a decline in maintainability.
- **Average time required for impact analysis** This reflects the number of program components that are affected by the change request. If this time increases, it implies more and more components are affected and maintainability is decreasing.
- **Average time taken to implement a change request** This is not the same as the time for impact analysis although it may correlate with it. This is the amount of time that you need to modify the system and its documentation, after you have assessed which components are affected. An increase in the time needed to implement a change may indicate a decline in maintainability.
- **Number of outstanding change requests** An increase in this number over time may imply a decline in maintainability.

Software reengineering

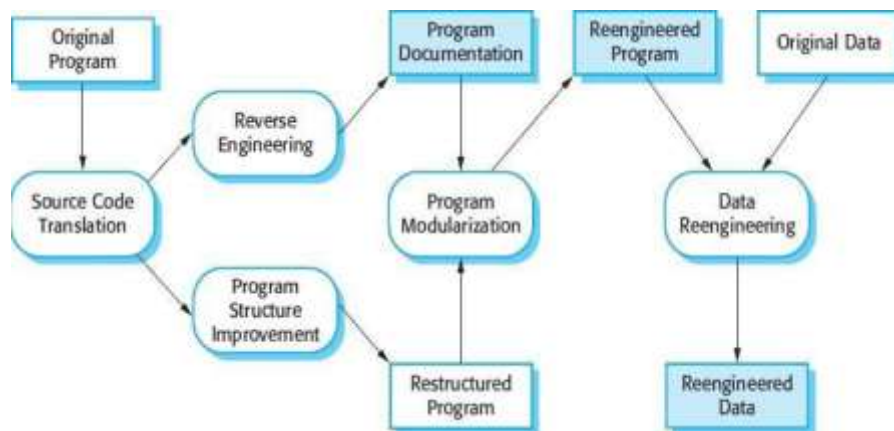
Reengineering may involve redocumenting the system, refactoring the system architecture, translating programs to a modern programming language, and modifying and updating the structure and values of the system's data. The functionality of the software is not changed and, normally, you should try to avoid making major changes to the system architecture.

There are two important benefits from reengineering rather than replacement:

- **Reduced risk** There is a high risk in redeveloping business-critical software. Errors may be made in the system specification or there may be development problems. Delays in introducing the new software may mean that business is lost and extra costs are incurred.
- **Reduced cost** The cost of reengineering may be significantly less than the cost of developing new software. Ulrich (1990) quotes an example of a commercial system for which the reimplementation costs were estimated at \$50 million. The system was successfully reengineered for \$12 million. I

suspect that, with modern software technology, the relative cost of reimplementaion is probably less than this but will still considerably exceed the costs of reengineering.

The below figure is a general model of the reengineering process. The input to the process is a legacy program and the output is an improved and restructured version of the same program. The activities in this reengineering process are as follows:



The reengineering process

- **Source code translation** Using a translation tool, the program is converted from an old programming language to a more modern version of the same language or to a different language.
- **Reverse engineering** The program is analyzed and information extracted from it. This helps to document its organization and functionality. Again, this process is usually completely automated.
- **Program structure improvement** The control structure of the program is analyzed and modified to make it easier to read and understand. This can be partially automated but some manual intervention is usually required.
- **Program modularization** Related parts of the program are grouped together and, where appropriate, redundancy is removed. In some cases, this stage may involve architectural refactoring (e.g., a system that uses several different data stores may be refactored to use a single repository). This is a manual process.
- **Data reengineering** The data processed by the program is changed to reflect program changes. This may mean redefining database schemas and converting existing databases to the new structure. You should usually also clean up the data. This involves finding and correcting mistakes, removing duplicate records, etc. Tools are available to support data reengineering.

Preventative maintenance by refactoring

Refactoring is the process of making improvements to a program to slow down degradation through change (Opdyke and Johnson, 1990). It means modifying a program to improve its structure, to reduce its complexity, or to make it easier to understand. Refactoring is sometimes considered to be limited to object-oriented development but the principles can be applied to any development approach. When you refactor a program, you should not add functionality but should concentrate on program improvement. You can therefore think of refactoring as 'preventative maintenance' that reduces the problems of future change.

Bad smells that can be improved through refactoring include:

- **Duplicate code** The same or very similar code may be included at different places in a program. This can be removed and implemented as a single method or function that is called as required.
- **Long methods** If a method is too long, it should be redesigned as a number of shorter methods.
- **Switch (case) statements** These often involve duplication, where the switch depends on the type of some value. The switch statements may be scattered around a program. In object-oriented languages, you can often use polymorphism to achieve the same thing.

- **Data clumping**Data clumps occur when the same group of data items (fields in classes, parameters in methods) reoccur in several places in a program. These can often be replaced with an object encapsulating all of the data.
- **Speculative generality**This occurs when developers include generality in a program in case it is required in future. This can often simply be removed.

3.11 Legacy system management

Most organizations usually have a portfolio of legacy systems that they use, with a limited budget for maintaining and upgrading these systems. They have to decide how to get the best return on their investment. This involves making a realistic assessment of their legacy systems and then deciding on the most appropriate strategy for evolving these systems. There are four strategic options:

- **Scrap the system completely**This option should be chosen when the system is not making an effective contribution to business processes. This commonly occurs when business processes have changed since the system was installed and are no longer reliant on the legacy system.
- **Leave the system unchanged and continue with regular maintenance**This option should be chosen when the system is still required but is fairly stable and the system users make relatively few change requests.
- **Reengineer the system to improve its maintainability**This option should be chosen when the system quality has been degraded by change and where a new change to the system is still being proposed. This process may include developing new interface components so that the original system can work with other, newer systems.
- **Replace all or part of the system with a new system**This option should be chosen when factors, such as new hardware, mean that the old system cannot continue in operation or where off-the-shelf systems would allow the new system to be developed at a reasonable cost. In many cases, an evolutionary replacement strategy can be adopted in which major system components are replaced by off-the-shelf systems with other components reused wherever possible.

There are four clusters of systems:

- **Low quality, low business value**Keeping these systems in operation will be expensive and the rate of the return to the business will be fairly small. These systems should be scrapped.
- **Low quality, high business value**These systems are making an important business contribution so they cannot be scrapped. However, their low quality means that it is expensive to maintain them. These systems should be reengineered to improve their quality. They may be replaced, if a suitable off-the-shelf system is available.
- **High quality, low business value**These are systems that don't contribute much to the business but which may not be very expensive to maintain. It is not worth replacing these systems so normal system maintenance may be continued if expensive changes are not required and the system hardware remains in use. If expensive changes become necessary, the software should be scrapped.
- **High quality, high business value**These systems have to be kept in operation. However, their high quality means that you don't have to invest in transformation or system replacement. Normal system maintenance should be continued.

3.12 Assignment Questions

1. Explain test driven development with neat diagram.
2. Explain software evolution process with neat diagram.
3. Explain legacy system management.
4. List and explain different types of release testing.

3.13 Outcome

- Familiarized with the stages of testing from testing, during development to acceptance testing by system customers
- Idea about differences between component, system, and release testing and awareness of user testing processes and techniques
- Have learned about different types of software maintenance and the factors that affect maintenance costs
- Familiarized with software evolution processes and influences on these processes

3.14 Further Reading

1. Ian Sommerville: Software Engineering, 9th Edition, Pearson Education, 2012. (Listed topics only from Chapters 1,2,3,4, 5, 7, 8, 9, 23, and 24)
2. The SCRUM Primer, Ver 2.0, <http://www.goodagile.com/scrumpriener/scrumpriener20.pdf>
3. Pressman: Software Engineering-A Practitioners approach, 7th Edition, Tata McGraw Hill.
4. Pankaj Jalote: An Integrated Approach to Software Engineering, WileyIndia
5. <http://agilemanifesto.org/>
6. <http://www.jamesshore.com/Agile-Book/>

MODULE 5

STRUCTURE

- 4.1 Introduction
- 4.2 Objective
- 4.3 Project Planning
- 4.4 Software pricing
- 4.5 Plan-driven development
- 4.6 Project scheduling
- 4.7 Estimation techniques
- 4.8 The COCOMO II model
- 4.9 Quality management
- 4.10 Software quality
- 4.11 Software standards
- 4.12 Reviews and inspections
- 4.13 Software measurement and metrics
- 4.14 Software component analysis
- 4.15 Assignment Questions
- 4.16 Outcome
- 4.17 Further Reading

4.1 Introduction

This module gives the overview of project planning, software pricing and plan driven development. Project scheduling, estimation technique are discussed in detail with an example of COCOMO II. Software quality management and software measurement and metrics are highlighted.

4.2 Objective

- understand the fundamentals of software costing and reasons why the price of the software may not be directly related to its development cost
- know what sections should be included in a project plan that is created within a plan-driven development process
- understand what is involved in project scheduling and the use of bar charts to present a project schedule
- have been introduced to the ‘planning game’, which is used to support project planning in extreme programming

4.3 Project Planning

Project planning is one of the most important jobs of a software project manager. As a manager, you have to break down the work into parts and assign these to project team members, anticipate problems that might arise, and prepare tentative solutions to those problems. The project plan, which is created at the start of a project, is used to communicate how the work will be done to the project team and customers, and to help assess progress on the project.

Project planning takes place at three stages in a project life cycle:

- At the proposal stage, when you are bidding for a contract to develop or provide a software system. You need a plan at this stage to help you decide if you have the resources to complete the work and to work out the price that you should quote to a customer.
- During the project startup phase, when you have to plan who will work on the project, how the project will be broken down into increments, how resources will be allocated across your company, etc. Here, you have more information than at the proposal stage, and can therefore refine the initial effort estimates that you have prepared.
- Periodically throughout the project, when you modify your plan in light of experience gained and information from monitoring the progress of the work. You learn more about the system being implemented and capabilities of your development team. This information allows you to make more accurate estimates of how long the work will take. Furthermore, the software requirements are likely to change and this usually means that the work breakdown has to be altered and the schedule extended.

4.4 Software pricing

In principle, the price of a software product to a customer is simply the cost of development plus profit for the developer. In practice, however, the relationship between the project cost and the price quoted to the customer is not usually so simple.

Factor	Description
Market opportunity	A development organization may quote a low price because it wishes to move into a new segment of the software market. Accepting a low profit on one project may give the organization the opportunity to make a greater profit later. The experience gained may also help it develop new products.
Cost estimate uncertainty	If an organization is unsure of its cost estimate, it may increase its price by a contingency over and above its normal profit.
Contractual terms	A customer may be willing to allow the developer to retain ownership of the source code and reuse it in other projects. The price charged may then be less than if the software source code is handed over to the customer.
Requirements volatility	If the requirements are likely to change, an organization may lower its price to win a contract. After the contract is awarded, high prices can be charged for changes to the requirements.
Financial health	Developers in financial difficulty may lower their price to gain a contract. It is better to make a smaller than normal profit or break even than to go out of business. Cash flow is more important than profit in difficult economic times.

Factors affecting software pricing

4.5 Plan-driven development

Plan-driven or plan-based development is an approach to software engineering where the development process is planned in detail. A project plan is created that records the work to be done, who will do it, the development schedule, and the work products. Managers use the plan to support project decision making and as a way of measuring progress. Plan-driven development is based on engineering project management

techniques and can be thought of as the ‘traditional’ way of managing large software development projects. This contrasts with agile development, where many decisions affecting the development are delayed and made later, as required, during the development process.

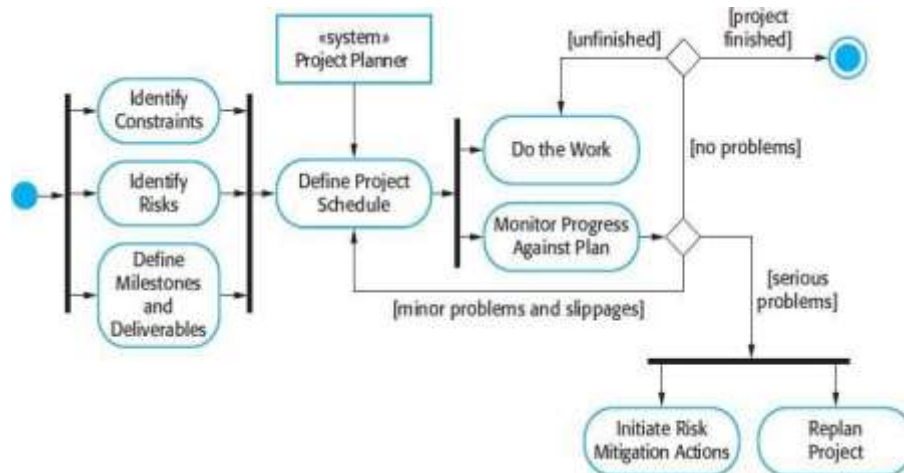
Project plans

In a plan-driven development project, a project plan sets out the resources available to the project, the work breakdown, and a schedule for carrying out the work. The plan should identify risks to the project and the software under development, and the approach that is taken to risk management. Although the specific details of project plans vary depending on the type of project and organization, plans normally include the following sections:

- **Introduction**This briefly describes the objectives of the project and sets out the constraints (e.g., budget, time, etc.) that affect the management of the project.
- **Project organization**This describes the way in which the development team is organized, the people involved, and their roles in the team.
- **Risk analysis**This describes possible project risks, the likelihood of these risks arising, and the risk reduction strategies that are proposed.
- **Hardware and software resource requirements**This specifies the hardware and support software required to carry out the development. If hardware has to be bought, estimates of the prices and the delivery schedule may be included.
- **Work breakdown**This sets out the breakdown of the project into activities and identifies the milestones and deliverables associated with each activity. Milestones are key stages in the project where progress can be assessed; deliverables are work products that are delivered to the customer.
- **Project schedule**This shows the dependencies between activities, the estimated time required to reach each milestone, and the allocation of people to activities.
- **Monitoring and reporting mechanisms**This defines the management reports that should be produced, when these should be produced, and the project monitoring mechanisms to be used.

The planning process

Project planning is an iterative process that starts when you create an initial project plan during the project startup phase. Below figure shows the UML activity diagram that shows a typical workflow for a project planning process.



The project planning process

Plan changes are inevitable. As more information about the system and the project team becomes available during the project, you should regularly revise the plan to reflect requirements, schedule, and risk changes. Changing business goals also leads to changes in project plans. As business goals change, this could affect all projects, which may then have to be replanned.

At the beginning of a planning process, you should assess the constraints affecting the project. These constraints are the required delivery date, staff available, overall budget, available tools, and so on. In conjunction with this, you should also identify the project milestones and deliverables. Milestones are points in the schedule against which you can assess progress, for example, the handover of the system for testing. Deliverables are work products that are delivered to the customer (e.g., a requirements document for the system).

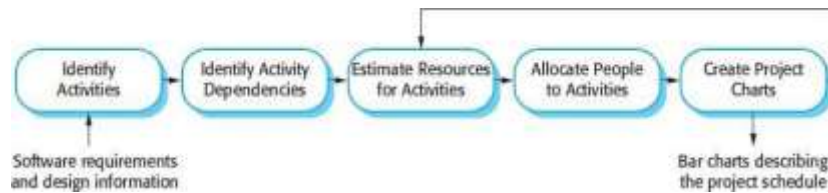
The outcome of a review may be a decision to cancel a project. This may be a result of technical or managerial failings but, more often, is a consequence of external changes that affect the project. The development time for a large software project is often several years. During that time, the business objectives and priorities inevitably change. These changes may mean that the software is no longer required or that the original project requirements are inappropriate. Management may then decide to stop software development or to make major changes to the project to reflect the changes in the organizational objectives.

4.6 Project scheduling

Project scheduling is the process of deciding how the work in a project will be organized as separate tasks, and when and how these tasks will be executed. You estimate the calendar time needed to complete each task, the effort required, and who will work on the tasks that have been identified. You also have to estimate the resources needed to complete each task, such as the disk space required on a server, the time required on specialized hardware, such as a simulator, and what the travel budget will be. In terms of the planning stages that I discussed in the introduction of this chapter, an initial project schedule is usually created during the project startup phase. This schedule is then refined and modified during development planning.

Scheduling in plan-driven projects (As shown in below figure) involves breaking down the total work involved in a project into separate tasks and estimating the time

required to complete each task. Tasks should normally last at least a week, and no longer than 2 months. Finer subdivision means that a disproportionate amount of time must be spent on replanning and updating the project plan. The maximum amount of time for any task should be around 8 to 10 weeks. If it takes longer than this, the task should be subdivided for project planning and scheduling.



The project scheduling process

Schedule representation

Project schedules may simply be represented in a table or spreadsheet showing the tasks, effort, expected duration, and task dependencies (As shown in below figure). However, this style of representation makes it difficult to see the relationships and dependencies between the different activities. For this reason, alternative graphical representations of project schedules have been developed that are often easier to read and understand.

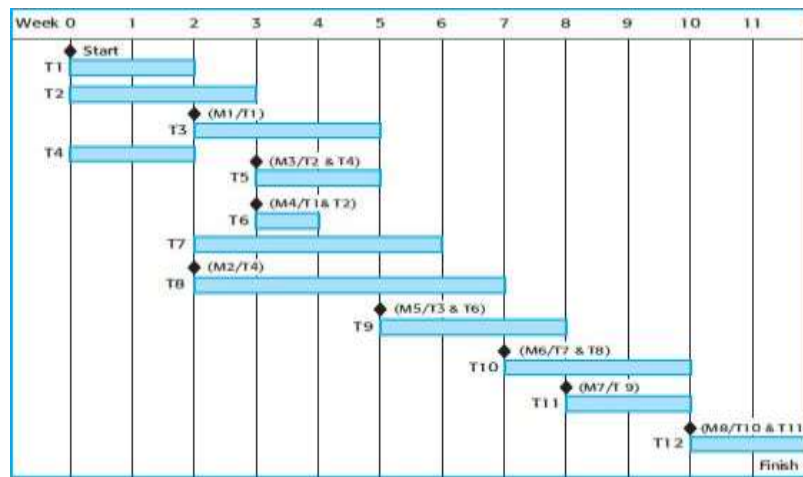
There are two types of representation that are commonly used:

- Bar charts, which are calendar-based, show who is responsible for each activity, the expected elapsed time, and when the activity is scheduled to begin and end. Bar charts are sometimes called ‘Gantt charts’, after their inventor, Henry Gantt.
- Activity networks, which are network diagrams, show the dependencies between the different activities making up a project.

Task	Effort (person-days)	Duration (days)	Dependencies
T1	15	10	
T2	8	15	
T3	20	15	T1 (M1)
T4	5	10	
T5	5	10	T2, T4 (M3)
T6	10	5	T1, T2 (M4)
T7	25	20	T1 (M1)
T8	75	25	T4 (M2)
T9	10	15	T3, T6 (M5)
T10	20	15	T7, T8 (M6)
T11	10	10	T9 (M7)
T12	20	10	T10, T11 (M8)

Tasks, durations, and dependencies

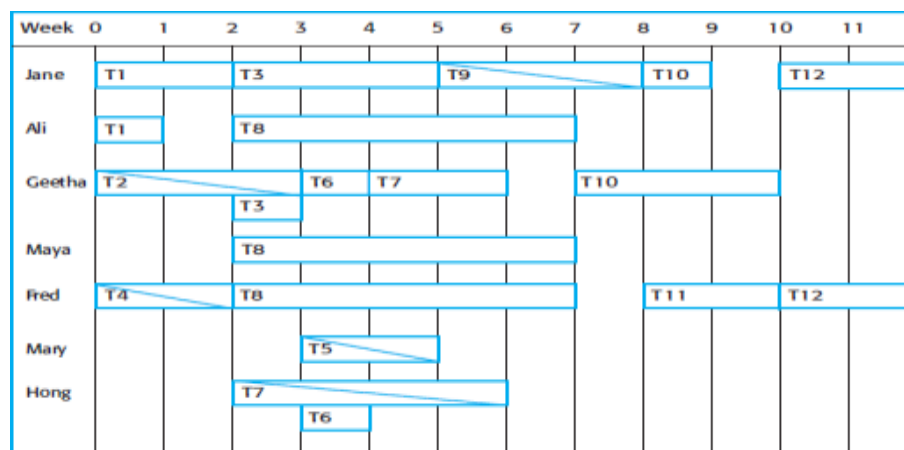
The below figure takes the information from above figure and presents the project schedule in a graphical format. It is a bar chart showing a project calendar and the start and finish dates of tasks. Reading from left to right, the bar chart clearly shows when tasks start and end. The milestones (M1, M2, etc.) are also shown on the bar chart. Notice that tasks that are independent are carried out in parallel (e.g., tasks T1, T2, and T4 all start at the beginning of the project).



Activity bar chart

As well as planning the delivery schedule for the software, project managers have to allocate resources to tasks. The key resource is, of course, the software engineers who will do the work, and they have to be assigned to project activities. The resource allocation can also be input to project management tools and a bar chart generated, which shows when staff is working on the project (As shown in below figure). People may be working on more than one task at the same time and, sometimes, they are not working on the project. They may be on holiday, working on other projects, attending training courses, or engaging in some other activity. I show part-time assignments using a diagonal line crossing the bar.

Large organizations usually employ a number of specialists who work on a project when needed. In below figure, you can see that Mary is a specialist, who works on only a single task in the project. This can cause scheduling problems. If one project is delayed while a specialist is working on it, this may have a knock-on effect on other projects where the specialist is also required. These may then be delayed because the specialist is not available.



Staff allocation chart

If a task is delayed, this can obviously affect later tasks that are dependent on it. They cannot start until the delayed task is completed. Delays can cause serious problems with staff allocation, especially when people are working on several projects at the same time. If a task (T) is delayed, the people allocated may be assigned to other work (W). To complete this may take longer than the delay but, once assigned, they cannot simply be

reassigned back to the original task, T. This may then lead to further delays in T as they complete W.

4.7 Estimation techniques

Project schedule estimation is difficult. You may have to make initial estimates on the basis of a high-level user requirements definition. The software may have to run on unfamiliar computers or use new development technology. The people involved in the project and their skills will probably not be known. There are so many uncertainties that it is impossible to estimate system development costs accurately during the early stages of a project.

There are two types of technique that can be used to do this:

- **Experience-based techniques** The estimate of future effort requirements is based on the manager's experience of past projects and the application domain. Essentially, the manager makes an informed judgment of what the effort requirements are likely to be.
- **Algorithmic cost modeling** In this approach, a formulaic approach is used to compute the project effort based on estimates of product attributes, such as size, and process characteristics, such as experience of staff involved.

Experience-based techniques rely on the manager's experience of past projects and the actual effort expended in these projects on activities that are related to software development. Typically, you identify the deliverables to be produced in a project and the different software components or systems that are to be developed. You document these in a spreadsheet, estimate them individually, and compute the total effort required. It usually helps to get a group of people involved in the effort estimation and to ask each member of the group to explain their estimate. This often reveals factors that others have not considered and you then iterate towards an agreed group estimate.

The difficulty with experience-based techniques is that a new software project may not have much in common with previous projects. Software development changes very quickly and a project will often use unfamiliar techniques such as web services, COTS-based development, or AJAX. If you have not worked with these techniques, your previous experience may not help you to estimate the effort required, making it more difficult to produce accurate costs and schedule estimates.

Algorithmic cost modeling

Algorithmic cost modeling uses a mathematical formula to predict project costs based on estimates of the project size; the type of software being developed; and other team, process, and product factors. An algorithmic cost model can be built by analyzing the costs and attributes of completed projects, and finding the closest-fit formula to actual experience.

Algorithmic models for estimating effort in a software project are mostly based on a simple formula:

$$\text{Effort} = A \times \text{Size}^B \times M$$

A is a constant factor which depends on local organizational practices and the type of software that is developed. Size may be either an assessment of the code size of the software or a functionality estimate expressed in function or application points. The value of exponent B usually lies between 1 and 1.5. M is a multiplier made by combining

process, product, and development attributes, such as the dependability requirements for the software and the experience of the development team.

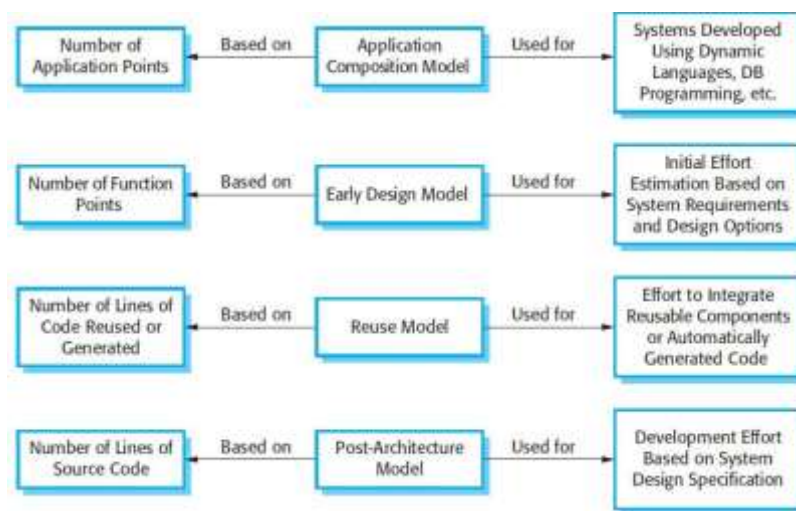
All algorithmic models have similar problems:

- It is often difficult to estimate Size at an early stage in a project, when only the specification is available. Function-point and application-point estimates (see later) are easier to produce than estimates of code size but are still often inaccurate.
- The estimates of the factors contributing to B and M are subjective. Estimates vary from one person to another, depending on their background and experience of the type of system that is being developed.

4.8 The COCOMO II model

Several similar models have been proposed to help estimate the effort, schedule, and costs of a software project. The model that I discuss here is the COCOMO II model. This is an empirical model that was derived by collecting data from a large number of software projects. These data were analyzed to discover the formulae that were the best fit to the observations. These formulae linked the size of the system and product, project and team factors to the effort to develop the system. COCOMO II is a well-documented and nonproprietary estimation model.

The submodels (As shown in below figure) that are part of the COCOMO II model are:



COCOMO estimation models

- **An application-composition model** This models the effort required to develop systems that are created from reusable components, scripting, or database programming. Software size estimates are based on application points, and a simple size/productivity formula is used to estimate the effort required.
- **An early design model** This model is used during early stages of the system design after the requirements have been established. The estimate is based on the standard estimation formula that I discussed in the introduction, with a simplified set of seven multipliers. Estimates are based on function points, which are then converted to number of lines of source code.

- **A reuse model** This model is used to compute the effort required to integrate reusable components and/or automatically generated program code. It is normally used in conjunction with the post-architecture model.
- **A post-architecture model** Once the system architecture has been designed, a more accurate estimate of the software size can be made. Again, this model uses the standard formula for cost estimation discussed above.

4.9 Quality management

Software quality management for software systems has three principal concerns:

- At the organizational level, quality management is concerned with establishing a framework of organizational processes and standards that will lead to high-quality software. This means that the quality management team should take responsibility for defining the software development processes to be used and standards that should apply to the software and related documentation, including the system requirements, design, and code.
- At the project level, quality management involves the application of specific quality processes, checking that these planned processes have been followed, and ensuring that the project outputs are conformant with the standards that are applicable to that project.
- Quality management at the project level is also concerned with establishing a quality plan for a project. The quality plan should set out the quality goals for the project and define what processes and standards are to be used.

The terms ‘quality assurance’ and ‘quality control’ are widely used in manufacturing industry. Quality assurance (QA) is the definition of processes and standards that should lead to high-quality products and the introduction of quality processes into the manufacturing process. Quality control is the application of these quality processes to weed out products that are not of the required level of quality.

Quality planning is the process of developing a quality plan for a project. The quality plan should set out the desired software qualities and describe how these are to be assessed. It therefore defines what ‘high-quality’ software actually means for a particular system.

Outline structure for a quality plan. This includes:

Product introduction A description of the product, its intended market, and the quality expectations for the product.

Product plans The critical release dates and responsibilities for the product, along with plans for distribution and product servicing.

Process descriptions The development and service processes and standards that should be used for product development and management.

Quality goals The quality goals and plans for the product, including an identification and justification of critical product quality attributes.

Risks and risk management The key risks that might affect product quality and the actions to be taken to address these risks.

4.10 Software quality

Software quality is not directly comparable with quality in manufacturing. The idea of tolerances is not applicable to digital systems and, for the following reasons, it may be impossible to come to an objective conclusion about whether or not a software system meets its specification:

- Software developers and customers may interpret the requirements in different ways and it may be impossible to reach agreement on whether or not software conforms to its specification.
- Specifications usually integrate requirements from several classes of stakeholders. These requirements are inevitably a compromise and may not include the requirements of all stakeholder groups. The excluded stakeholders may therefore perceive the system as a poor quality system, even though it implements the agreed requirements.
- It is impossible to measure certain quality characteristics (e.g., maintainability) directly and so they cannot be specified in an unambiguous way.

The below figure illustrates this process-based approach to achieving product quality.



Process based quality

There is a clear link between process and product quality in manufacturing because the process is relatively easy to standardize and monitor. Once manufacturing systems are calibrated, they can be run again and again to output high-quality products. However, software is not manufactured—it is designed. In software development, therefore, the relationship between process quality and product quality is more complex. Software development is a creative rather than a mechanical process, so the influence of individual skills and experience is significant. External factors, such as the novelty of an application or commercial pressure for an early product release, also affect product quality irrespective of the process used.

4.11 Software standards

Software standards play a very important role in software quality management. As I have discussed, an important part of quality assurance is the definition or selection of standards that should apply to the software development process or software product. As part of this QA process, tools and methods to support the use of these standards may also be chosen. Once standards have been selected for use, project-specific processes have to be defined to monitor the use of the standards and check that they have been followed.

Software standards are important for three reasons:

- Standards capture wisdom that is of value to the organization. They are based on knowledge about the best or most appropriate practice for the company. This knowledge is often only acquired after a great deal of trial and error. Building it

into a standard helps the company reuse this experience and avoid previous mistakes.

- Standards provide a framework for defining what ‘quality’ means in a particular setting.
- Standards assist continuity when work carried out by one person is taken up and continued by another. Standards ensure that all engineers within an organization adopt the same practices. Consequently, the learning effort required when starting new work is reduced.

There are two related types of software engineering standard that may be defined and used in software quality management:

- **Product standards** These apply to the software product being developed. They include document standards, such as the structure of requirements documents, documentation standards, such as a standard comment header for an object class definition, and coding standards, which define how a programming language should be used.
- **Process standards** These define the processes that should be followed during software development. They should encapsulate good development practice. Process standards may include definitions of specification, design and validation processes, process support tools, and a description of the documents that should be written during these processes.

Standards have to deliver value, in the form of increased product quality. There is no point in defining standards that are expensive in terms of time and effort to apply that only lead to marginal improvements in quality. Product standards have to be designed so that they can be applied and checked in a cost-effective way, and process standards should include the definition of processes that check that product standards have been followed.

To minimize dissatisfaction and to encourage buy-in to standards, quality managers who set the standards should therefore take the following steps:

- **Involve software engineers in the selection of product standards** If developers understand why standards have been selected, they are more likely to be committed to these standards. Ideally, the standards document should not just set out the standard to be followed, but should also include commentary explaining why standardization decisions have been made.
- **Review and modify standards regularly to reflect changing technologies** Standards are expensive to develop and they tend to be enshrined in a company standards handbook. Because of the costs and discussion required, there is often a reluctance to change them. A standards handbook is essential but it should evolve to reflect changing circumstances and technology.
- **Provide software tools to support standards** Developers often find standards to be a bugbear when conformance to them involves tedious manual work that could be done by a software tool. If tool support is available, very little effort is required to follow the software development standards.

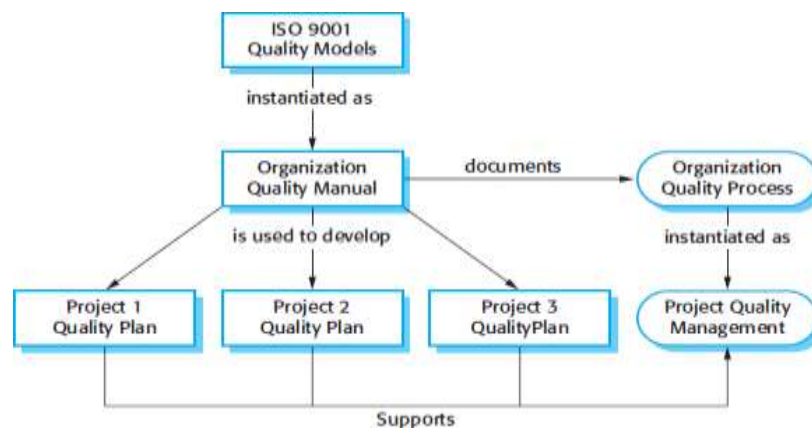
The ISO 9001 standards framework

The ISO 9001 standard is not itself a standard for software development but is a framework for developing software standards. It sets out general quality principles, describes quality processes in general, and lays out the organizational standards and

procedures that should be defined. These should be documented in an organizational quality manual.

The ISO 9001 standard does not define or prescribe the specific quality processes that should be used in a company. To be conformant with ISO 9001, a company must have defined the types of process shown in Figure 24.5 and have procedures in place that demonstrate that its quality processes are being followed. This allows flexibility across industrial sectors and company sizes. Quality standards can be defined that are appropriate for the type of software being developed. Small companies can have unbureaucratic processes and still be ISO 9001 compliant. However, this flexibility means that you cannot make assumptions about the similarities or differences between the processes in different ISO 9001-compliant companies. Some companies may have very rigid quality processes that keep detailed records, whereas others may be much less formal, with minimal additional documentation.

The relationships between ISO 9001, organizational quality manuals, and individual project quality plans are shown in Figure 24.6. This diagram has been derived from a model given by Ince (1994), who explains how the general ISO 9001 standard can be used as a basis for software quality management processes.



ISO 9001 and quality management

Some software customers demand that their suppliers should be ISO 9001 certified. The customers can then be confident that the software development company has an approved quality management system in place. Independent accreditation authorities examine the quality management processes and process documentation and decide if these processes cover all of the areas specified in ISO 9001. If so, they certify that a company's quality processes, as defined in the quality manual, conform to the ISO 9001 standard.

4.12 Reviews and inspections

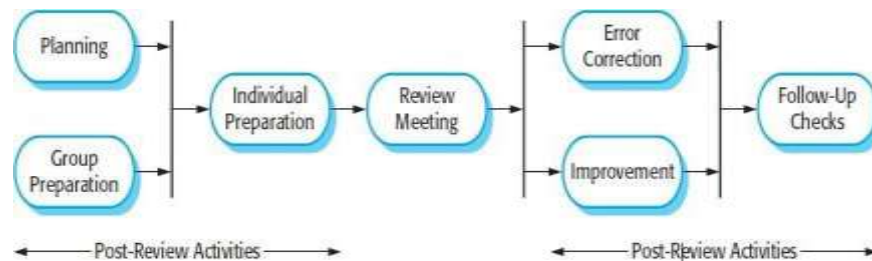
Reviews and inspections are QA activities that check the quality of project deliverables. This involves examining the software, its documentation and records of the process to discover errors and omissions and to see if quality standards have been followed. As I discussed in Chapters 8 and 15, reviews and inspections are used alongside program testing as part of the general process of software verification and validation.

Quality reviews are based on documents that have been produced during the software

development process. As well as software specifications, designs, or code, process models, test plans, configuration management procedures, process standards, and user manuals may all be reviewed. The review should check the consistency and completeness

of the documents or code under review and make sure that quality standards have been followed.

Although a quality review provides information for management about the software being developed, quality reviews are not the same as management progress reviews. Their prime concern is whether or not the project will deliver useful software on time and on budget. Progress reviews take external factors into account, and changed circumstances may mean that software under development is no longer required or has to be radically changed.



The software review process

The review process

Pre-review activities These are preparatory activities that are essential for the review to be effective. Typically, pre-review activities are concerned with review planning and review preparation. Review planning involves setting up a review team, arranging a time and place for the review, and distributing the documents to be reviewed. During review preparation, the team may meet to get an overview of the software to be reviewed. Individual review team members read and understand the software or documents and relevant standards. They work independently to find errors, omissions, and departures from standards. Reviewers may supply written comments on the software if they cannot attend the review meeting.

The review meeting During the review meeting, an author of the document or program being reviewed should ‘walk through’ the document with the review team. The review itself should be relatively short—two hours at most. One team member should chair the review and another should formally record all review decisions and actions to be taken. During the review, the chair is

responsible for ensuring that all written comments are considered. The review chair should sign a record of comments and actions agreed during the review.

Post-review activities After a review meeting has finished, the issues and problems raised during the review must be addressed. This may involve fixing software bugs, refactoring software so that it conforms to quality standards, or rewriting documents. Sometimes, the problems discovered in a quality review are such that a management review is also necessary to decide if more resources should be made available to correct them. After changes have been made, the review chair may check that the review comments have all been taken into account. Sometimes, a further review will be required to check that the changes made cover all of the previous review comments.

Program inspections

Program inspections involve team members from different backgrounds who make a careful, line-by-line review of the program source code. They look for defects and problems and describe these at an inspection meeting. Defects may be logical errors,

anomalies in the code that might indicate an erroneous condition or features that have been omitted from the code. The review team examines the design models or the program code in detail and highlights anomalies and problems for repair.

During an inspection, a checklist of common programming errors is often used to focus the search for bugs. This checklist may be based on examples from books or from knowledge of defects that are common in a particular application domain. You use different checklists for different programming languages because each language has its own characteristic errors. Humphrey (1989), in a comprehensive discussion of inspections, gives a number of examples of inspection checklists.

Fault class	Inspection check
Data faults	• Are all program variables initialized before their values are used? • Have all constants been named? • Should the upper bound of arrays be equal to the size of the array or Size -1? • If character strings are used, is a delimiter explicitly assigned? • Is there any possibility of buffer overflow?
Control faults	• For each conditional statement, is the condition correct? • Is each loop certain to terminate? • Are compound statements correctly bracketed? • In case statements, are all possible cases accounted for? • If a break is required after each case in case statements, has it been included?
Input/output faults	• Are all input variables used? • Are all output variables assigned a value before they are output? • Can unexpected inputs cause corruption?
Interface faults	• Do all function and method calls have the correct number of parameters? • Do formal and actual parameter types match? • Are the parameters in the right order? • If components access shared memory, do they have the same model of the shared memory structure?
Storage management faults	• If a linked structure is modified, have all links been correctly reassigned? • If dynamic storage is used, has space been allocated correctly? • Is space explicitly deallocated after it is no longer required?
Exception management faults	• Have all possible error conditions been taken into account?

An inspection checklist

4.13 Software measurement and metrics

Software measurement is concerned with deriving a numeric value or profile for an attribute of a software component, system, or process. By comparing these values to each other and to the standards that apply across an organization, you may be able to draw conclusions about the quality of software, or assess the effectiveness of software processes, tools, and methods.

A software metric is a characteristic of a software system, system documentation, or development process that can be objectively measured. Examples of metrics include the size of a product in lines of code; the Fog index (Gunning, 1962), which is a measure of the readability of a passage of written text; the number of reported faults in a delivered software product; and the number of person-days required to develop a system component.

There are two ways in which measurements of a software system may be used:

- **To assign a value to system quality attributes** By measuring the characteristics of system components, such as their cyclomatic complexity, and then aggregating these measurements, you can assess system quality attributes, such as maintainability.
- **To identify the system components whose quality is substandard** **Measurements** can identify individual components with characteristics that deviate from the norm. For example, you can measure components to discover those with the highest complexity. These are most likely to contain bugs because the complexity makes them harder to understand.

Product metrics

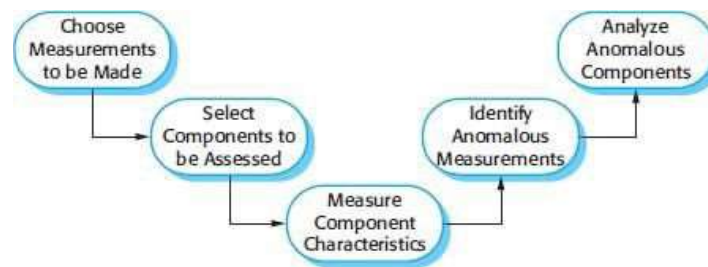
Product metrics fall into two classes:

- Dynamic metrics, which are collected by measurements made of a program in execution. These metrics can be collected during system testing or after the system has gone into use. An example might be the number of bug reports or the time taken to complete a computation.
- Static metrics, which are collected by measurements made of representations of the system, such as the design, program, or documentation. Examples of static metrics are the code size and the average length of identifiers used.

Software metric	Description
Fan-in/Fan-out	Fan-in is a measure of the number of functions or methods that call another function or method (say X). Fan-out is the number of functions that are called by function X. A high value for fan-in means that X is tightly coupled to the rest of the design and changes to X will have extensive knock-on effects. A high value for fan-out suggests that the overall complexity of X may be high because of the complexity of the control logic needed to coordinate the called components.
Length of code	This is a measure of the size of a program. Generally, the larger the size of the code of a component, the more complex and error-prone that component is likely to be. Length of code has been shown to be one of the most reliable metrics for predicting error-proneness in components.
Cyclomatic complexity	This is a measure of the control complexity of a program. This control complexity may be related to program understandability. I discuss cyclomatic complexity in Chapter 8.
Length of identifiers	This is a measure of the average length of identifiers (names for variables, classes, methods, etc.) in a program. The longer the identifiers, the more likely they are to be meaningful and hence the more understandable the program.
Depth of conditional nesting	This is a measure of the depth of nesting of if-statements in a program. Deeply nested if-statements are hard to understand and potentially error-prone.
Fog index	This is a measure of the average length of words and sentences in documents. The higher the value of a document's Fog index, the more difficult the document is to understand.

Static software product metrics

4.14 Software component analysis



The process of product measurement

- **Choose measurements to be made** The questions that the measurement is intended to answer should be formulated and the measurements required to answer these questions defined.
- **Select components to be assessed** You may not need to assess metric values for all of the components in a software system. Sometimes, you can select a representative selection of components for measurement, allowing you to make an overall assessment of system quality.
- **Measure component characteristics** The selected components are measured and the associated metric values computed.
- **Identify anomalous measurements** After the component measurements have been made, you then compare them with each other and to previous measurements that have been recorded in a measurement database.
- **Analyze anomalous components** When you have identified components that have anomalous values for your chosen metrics, you should examine them to decide whether or not these anomalous metric values mean that the quality of the component is compromised.

4.15 Assignment Questions

1. What is software pricing?
2. Explain plan driven development in detail.
3. What is project scheduling?
4. Explain project planning life cycle process.
5. Why software standards are important?

4.16 Outcome

- After studying this module students will be able to understand project planning, scheduling, and cost estimation understand what is involved in project scheduling

and the use of barcharts to present a project ,‘planning game’, which is used to supportproject planning in extreme programming

4.17 Further Reading

1. Ian Sommerville: Software Engineering, 9th Edition, Pearson Education, 2012. (Listed topics only from Chapters 1,2,3,4, 5, 7, 8, 9, 23, and 24)
2. The SCRUM Primer, Ver 2.0, <http://www.goodagile.com/scrumpprimer/scrumpprimer20.pdf>
3. Pressman: Software Engineering-A Practitioners approach, 7th Edition, Tata McGraw Hill.
4. Pankaj Jalote: An Integrated Approach to Software Engineering, WileyIndia
5. <http://agilemanifesto.org/>
6. <http://www.jamesshore.com/Agile-Book/>